Extending Intuitionistic Logic to a Probability Calculus (in Lean 4)

Anonymous Author(s)

Abstract

Kolmogorov's probability fixes the event algebra to be Boolean, so every event is decidable. This is a poor fit for the semidecidable properties, termination, reachability, that verification cares about. Following the Cox–Jaynes interpretation of probability as extended logic, we define a construct for extending a logic (treated as a parameter in our approach) to a calculus of probability, and specify criteria for when this works: (1) when the resulting calculus can reconstruct the original logic in the limiting case of certain (0– or 1– probability) events, (2) when the construction, evaluated at classical logic, agrees with classical probability theory, and (3) when every linear law that holds in all certain states holds throughout the calculus. Together with closure under mixtures, the criteria determine the calculus uniquely. Instantiating the construction at intuitionistic logic gives valuations on frames, complete Heyting algebras of open sets, the algebras of verifiable propositions. In this setting, we define slack, $1 - v(A) - v(\neg A)$, the classical complement rule's deficit and boundary mass in the measure model. Slack pinpoints exactly when Bayesian conditioning, which treats the evidence as fully decided, gives an incorrect update, disagreeing with Dempster's rule. A separate obstruction, a five-element counterexample, shows that Cox's sum rule cannot be derived from additivity on disjoint events over frames and must instead be posited. Classical probability credits the slack of a semidecidable property to its failure, an event no finite observation can confirm. Evaluated at linear logic, the construction agrees with probabilistic coherence spaces at first order, and at second order it gives the randomized sequential programs, a strict subset. For argument types built from $\mathbf{1}$ with & and $\oplus$ we characterize exactly when the two agree, by a direct proof that avoids perfect graph theory. The whole development is machine-checked in Lean 4 over mathlib, in roughly 15,000 lines with no `sorry`.

Keywords: intuitionistic logic, probability, Cox's theorem, belief functions, locales, Lean, mathlib, formalized mathematics

1 Introduction

Kolmogorov's probability [Kolmogorov 1933] assigns numerical values representing uncertainty to events. The event logic is classical, forming a Boolean σ-algebra. The assignments satisfy non-negativity, normalization, and σ-additivity. This commits the framework to the axioms of classical logic, including the law of excluded middle. Many properties of programs (e.g. "does this computation halt?", "does this process reach a given state?") are only *semidecidable*: a procedure confirms the property when it holds, and no procedure, in general, confirms its failure. The mismatch that results is not a contradiction. It is a failure of discrimination. A probability calculus built on a particular logic inherits that logic's identifications, so it cannot discriminate events the logic proves equal.

In the case of classical logic, this identification is the *law of excluded middle*: $\mathit{halts} \sqcup \neg\mathit{halts} = \top$, so the Boolean event algebra contains $\neg\mathit{halts}$. The complement rule $P(A^c) = 1 - P(A)$ then forces the remainder $1 - P(\mathit{halts})$ onto $\neg\mathit{halts}$, an event no finite observation can confirm. The events "not yet confirmed" and "refuted" are identified. For a program whose non-termination is independent of ZFC [Yedidia and Aaronson 2016], the classical value $P(\mathit{halts}) = 0$ is correct, yet neither running the program nor any ZFC proof ever establishes it. Uniformly over programs, computing these sharp values is exactly as hard as the halting problem.

Another failure of discrimination appears in *linear logic*, whose tensor $\otimes$ distinguishes a from $a \otimes a$, one use of a resource from two. Modularity with $\otimes$ in place of $\sqcap$, $v(a) + v(b) = v(a \sqcup b) + v(a \otimes b)$, inherits idempotence of disjunction, $a \sqcup a = a$, and at $b = a$ forces $v(a \otimes a) = v(a)$. So, no assignment satisfying this axiom distinguishes one use of a from two.

Probability is, as the Cox–Jaynes tradition has it, *extended logic* [Cox 1946; Jaynes 2003], and different logics can be extended through the same construction [Paris 2001; Williams 2012]. The logic of events, in its algebraic form the event algebra, is a *parameter* of the construction, and the choice of event algebra determines the probability calculus. An event algebra consists of a set of events together with operations interpreting the logic's connectives and structural rules, while a probability calculus is differentiated from an arbitrary grading of events by satisfying certain axioms. We specify how to construct such extensions, and propose three criteria for when the extension of a logic is a probability calculus:

(1) the valuations taking only the values 0 and 1 (the two degrees of complete certainty) must reconstruct the underlying event logic. This is what we take "extended logic" to mean.
(2) the construction evaluated at classical logic must return the classical theory, ensuring it is a genuine generalization of classical probability.

(3) every linear inequality between probabilities that holds in all states of complete certainty must hold in every state of the calculus, so that uncertainty is uncertainty about which certain state obtains.

Criteria (1) and (3) together say that the calculus obeys exactly the laws of the logic's certain states, so it neither conflates events the logic distinguishes nor misses a law the logic imposes. With closure under mixtures they determine the calculus uniquely (Section 10.2).

We propose two fixes for the identifications above. Both extend a logic to a probability calculus, and on finite frames the second reduces to the first. The first fix we propose is one that accepts the standard axioms imposed on a grading to make it a probability calculus, while choosing an event algebra that makes the required distinctions. The extension of the event algebra to a probability calculus is then given by the *valuations*, i.e. the maps from events to numbers satisfying monotonicity, normalization, and modularity [Paris 2001; Weatherson 2003]. The resulting extension does not guarantee defining a probability calculus, and we specify the conditions under which it does: the $\{0,1\}$-valued valuations are the prime filters of the event algebra, and in a classical metatheory they reconstruct its order when it is distributive. Intuitionistic logic, in particular, is a case for which this approach yields an extension, and we work it out in detail. Its event algebras are the Heyting algebras, and we work with frames (the complete Heyting algebras). The open sets of any topological space form a frame. On a frame the pseudocomplement $\neg a$ need not satisfy $a \sqcup \neg a = \top$, so the complement rule weakens to $v(a) + v(\neg a) \leq 1$. The gap $\mathrm{slack}(a) = 1 - v(a) - v(\neg a)$ is the mass of the part of a neither confirmed nor refuted. For the valuation that a measure μ induces on the open sets of a space, the slack of an open set is the μ-measure of its boundary.

The second fix also applies to logics whose connectives are not lattice operations, such as linear logic. It generalizes the axioms of the probability calculus in addition to varying the event algebra. It applies to any logic presented by a set of *certain states*, its $\{0,1\}$-valued assignments. The calculus is the set of finite mixtures of certain states, the construction of de Finetti that generalized probabilism applies to non-classical logics [Paris 2001; Williams 2012]. A mixture that takes only the values 0 and 1 is one of the certain states it mixes, because every certain state with positive weight must agree with it at every event. So the $\{0,1\}$-valued elements of the calculus are exactly the certain states, which present the logic, and criterion (1) holds. For finitely many events, the axioms of the calculus are exactly the affine laws that hold on every certain state. When the certain states respect $\wedge$ and $\vee$, Paris shows that these laws are monotonicity, normalization, and modularity [Paris 2001], so on finite frames and finite Boolean algebras the calculus is the set of valuations.

For linear logic, the event algebra is a quantale, a complete lattice with an associative tensor $\otimes$ that need not be idempotent. The first fix fails there because of the probability axioms, not the event algebra: the modularity axiom identifies a with $a \otimes a$. Probabilistic coherence spaces [Danos and Ehrhard 2011] avoid this by assigning weights to the points of each formula's web instead of to formulas, with $a \otimes a$ interpreted on a product web. Their elements are not valuations in the sense of the modularity axiom. For every multiplicative–additive formula over finite data types, the calculus of the second fix is contained in the probabilistic coherence space of the formula, and the certain elements of that space are cliques of Girard's coherence space [Girard 1987].

The calculus and the probabilistic coherence space coincide at first-order linear logic. At higher order they can differ: for a program with two functional arguments the calculus is the set of randomized sequential programs, a strict subset of the probabilistic coherence space (Section 11). The probabilistic coherence space, the standard model of probabilistic programs at this type, therefore admits behaviours that no sequential program flipping coins can produce in a linear probabilistic language for this type. At this type these behaviours diverge with positive probability, and with three functional arguments the gap occurs among programs that always halt. Reasoning in it can certify only weaker guarantees than the programs satisfy. The gap has the same numerical form as the gap between classical and non-classical correlations in quantum foundations [Cabello et al. 2014]. At !Bool the calculus is also a strict subset.

Quantum logic, whose event algebras are orthomodular lattices [Birkhoff and von Neumann 1936], is a logic to which neither fix applies. Both rely on $\{0,1\}$-valued assignments: the first on $\{0,1\}$-valued valuations, which need not exist on non-distributive lattices and do not exist on M_3, and the second relies on having certain states. A certain state of quantum logic would assign 1 to exactly one ray of every orthogonal basis, and by the Kochen–Specker theorem [Kochen and Specker 1967] there is none in dimension at least 3. We prove this for an 18-ray configuration in $\mathbb{R}^4$ [Cabello et al. 1996], so the calculus of the second fix is empty there. Quantum states, which by Gleason's theorem [Gleason 1957] are given by density matrices, nevertheless exist, and they fail criterion (1). Requiring certainty only within each context, and reading criterion (1) through the events of probability 1 and 0, extends the construction to quantum logic (Section 12.1).

Contributions.

1. **Characterizations** (Sections 2, 3 and 5). Dempster's rule on evidence b with $v(b) \neq 0$ agrees with Bayesian conditioning $v(a \sqcap b)/v(b)$ at every a iff the slack at b is zero, the slack splits exactly into a double-negation part and a De Morgan part.

2. **Cox's theorem** (Section 6). Darst's five-element lattice of sets [Darst 1970], which carries an assignment additive on disjoint joins without being modular, is a frame satisfying weak excluded middle, so the derivation of the sum rule from additivity on disjoint joins, which Knuth and Skilling state for distributive lattices [Knuth and Skilling 2012], does not extend to frames, and the sum rule must be posited.
3. **Beyond frames** (Section 9). For a lattice with a least element, every element of the commutative monoid through which its modular maps factor is a sum over a chain, which is not unique at M_3, and a function satisfying the modular law with $\otimes$ in place of $\sqcap$ gives a and $a \otimes a$ the same finite value.
4. **Linear logic** (Sections 10.4 and 11.3). At every multiplicative–additive formula over finite data types, every $\{0,1\}$-valued element of the probabilistic coherence space is the indicator of a clique of Girard's coherence space, and at $(T \otimes T)^{\perp}$, for $T = ((\mathbf{2} \multimap \mathbf{2}) \otimes (\mathbf{2} \multimap \mathbf{2}))^{\perp}$, the converse fails.
5. **Second-order linear types** (Section 11). For a program with two functional arguments, which may diverge, the calculus is the stable-set polytope of a graph whose clique-constrained polytope is the probabilistic coherence space, and for X and Y built from $\mathbf{1}$ with $\&$ and $\oplus$ a mechanized proof that avoids perfect graph theory shows that the calculus and the probabilistic coherence space of $(X \otimes Y)^{\perp}$ coincide iff the coherence graph of X or of Y has no induced 4-cycle.
6. **Four-cycles** (Sections 11.2 and 11.4). Two formulas whose coherence graphs have induced 4-cycles give a gap at every formula over nonempty data types in which their tensor occurs negatively, and $(\mathbf{2} \,\&\, \mathbf{1}) \multimap ((\mathbf{2} \multimap \mathbf{2}) \otimes (\mathbf{2} \multimap \mathbf{2}))$ has a gap although no tensor occurring negatively in it has an induced 4-cycle in both factors.
7. **Local certainty** (Sections 12.1 and 13.2). The probabilistic coherence space of every multiplicative–additive formula lies between its calculus and its local calculus, the functions of total weight at most 1 on the support of every $\{0,1\}$-valued element of the dual, linear negation exchanges the two, and at $T^{\perp}$ the Popescu–Rohrlich box lies in the local calculus and not in the probabilistic coherence space, so the gap at T and Bell's theorem at its dual are equivalent.
8. **A first formal treatment.** Every result above is machine-checked in Lean 4 over mathlib, except the steps the text marks as not formalized, and to our knowledge it is the first mechanization of valuations on arbitrary frames without a continuity assumption.

Mechanization. The development is carried out in Lean 4 [de Moura and Ullrich 2021] over mathlib [mathlib Community 2020], in roughly 15,000 lines with no `sorry`, and is included in full as anonymous supplementary material.[1] Two points are specific to a proof assistant. We separate *object-level* classicality, excluded middle in the frame Ω, from the *meta-level* classicality mathlib inherits through `Classical.choice`, identify where the meta-level dependence is essential (Table 1), and give proofs without choice of the finite results that admit them. The product-rule half of Cox's theorem needs an Aczél/Hölder regraduation absent from mathlib, which we build from scratch.

2 Valuations on Frames

2.1 The Event Logic

Definition 1 (frame)**.** A *frame* is a complete lattice Ω in which finite meets distribute over arbitrary joins,

$$a \sqcap \Big(\bigsqcup_{i \in I} b_i\Big) \;=\; \bigsqcup_{i \in I} (a \sqcap b_i)$$

for every $a \in \Omega$ and every family $\{b_i\}_{i \in I} \subseteq \Omega$. Equivalently, it is a complete Heyting algebra. By $x \Rightarrow y$ we denote the Heyting implication in a frame, and $\neg a := a \Rightarrow \bot$ is the pseudocomplement of $a \in \Omega$. Also, a^c denotes a complement of a, if it exists, i.e. an element such that $a \sqcap a^c = \bot$ and $a \sqcup a^c = \top$.

Frames are the algebraic semantics of intuitionistic logic and the point-free abstraction of topological spaces. The same objects with morphisms reversed form the category of *locales*, so the distinction is invisible at the level of a single algebra. We say *frame* throughout and reserve *locale* for the spatial and categorical statements (points of a locale, classifying locales) where that word is the standard one. The motivating example throughout is $\Omega = \mathcal{O}(X)$, the opens of a space, with $\sqcap, \sqcup$ as intersection and union and the pseudocomplement $\neg a$ of an open given by the *interior* of its set complement. In mathlib this is the typeclass `Order.Frame`, and the Heyting operations come with the key asymmetry already in place: $a \sqcap \neg a = \bot$ holds in any Heyting algebra (`inf_compl_eq_bot`), while $a \sqcup \neg a = \top$ is available only in Boolean algebras. An element with $a \sqcup \neg a = \top$ is *complemented*: it satisfies its own instance of excluded middle.

2.2 The Valuation Structure

The central object assigns a degree of certainty to each element of the frame, with inclusion–exclusion as the only interaction axiom:

Definition 2 (valuation^{Lean})**.** A *valuation* on a frame Ω is a *monotone* function $v : \Omega \to [0, \infty]$ with $v(\bot) = 0$, $v(\top) = 1$ (*normalization*), and, for all $a, b \in \Omega$, *modularity*:

$$v(a) + v(b) \;=\; v(a \sqcup b) + v(a \sqcap b).$$

[1] Claude (Anthropic) was used to assist with drafting and debugging the Lean formalization.

```
structure Valuation (Ω : Type*) [Order.
    Frame Ω] where
  toFun : Ω → ℝ≥0∞
  map_bot' : toFun ⊥ = 0
  map_top' : toFun ⊤ = 1
  mono'    : Monotone toFun
  modular' : ∀ a b, toFun a + toFun b
             = toFun (a ⊔ b) + toFun (a ⊓
                 b)
```

This axiom set is not ours: Weatherson's axioms for intuitionistic probability are exactly these [Weatherson 2003], they are Paris's axioms for logics whose two-valued valuations respect $\wedge$ and $\vee$ [Paris 2001], and the valuations of constructive measure theory [Coquand and Spitters 2009; Vickers 2008] are their Scott-continuous instances (Section 15).

Values live in the extended non-negative reals $[0, \infty]$, and normalization confines them to $[0, 1]$[Lean]. The conventions this choice carries, unconditional summation and truncated subtraction, are an artifact of the mechanization and are recorded in Appendix A.

2.3 Slack

Modularity applied to the disjoint pair $(a, \neg a)$ collapses to $v(a) + v(\neg a) = v(a \sqcup \neg a)$[Lean]. So, the join need not be $\top$:

Theorem 3 (the complement bound[Lean])**.** *For every valuation v on a frame Ω and every $a \in \Omega$, $v(a) + v(\neg a) \leq 1$.*

```
theorem Valuation.add_compl_le_one (v :
    Valuation Ω) (a : Ω) :
    v a + v aᶜ ≤ 1
```

This is a *derived* Dempster–Shafer sub-additivity inequality. The deficit

$$\mathrm{slack}(a) \;=\; 1 - \big(v(a) + v(\neg a)\big) \;=\; 1 - v(a \sqcup \neg a)$$

is the valuation of the undecided region. It decomposes canonically. The pair $(\neg\neg a, \neg a)$, double negation against negation, is always disjoint. However, it is a partition only under *weak* excluded middle ($\neg a \sqcup \neg\neg a = \top$). Applying modularity to this disjoint pair splits the slack into two logically independent gaps:

Definition 4 (regular element)**.** An element $a \in \Omega$ is *regular* (or $\neg\neg$-stable) if $a = \neg\neg a$.

Theorem 5 (slack decomposition[Lean])**.** *For every valuation v on a frame Ω and every $a \in \Omega$,* $\mathrm{slack}(a) = \mathrm{dnGap}(a) + \mathrm{deMorganGap}(a)$*, where* $\mathrm{dnGap}(a) = v(\neg\neg a) - v(a)$ *is* 0 *if a is $\neg\neg$-stable (regular), and* $\mathrm{deMorganGap}(a) = 1 - v(\neg\neg a \sqcup \neg a)$ *is* 0 *if weak excluded middle holds at a.*

The single Dempster–Shafer "ignorance" number is thus resolved into a regularity defect and a De Morgan defect: the first measures failure of double-negation stability, the second corresponds to a named intermediate logic (weak excluded middle). Each part detects one step of the chain of logics from intuitionistic logic through weak excluded middle to classical logic:

Theorem 6 (slack and intermediate logics[Lean])**.** *For every frame Ω, in a classical metatheory,* $\mathrm{deMorganGap}(a) = 0$ *for every valuation and every a iff Ω satisfies weak excluded middle, and* $\mathrm{dnGap}(a) = 0$ *for every valuation and every a iff every element of Ω is regular, that is, iff Ω is Boolean.*

The forward directions are the vanishing conditions of the slack decomposition. The converses use a sharp valuation that is 1 at $\top$ and 0 at $\neg\neg a \sqcup \neg a$, or 1 at $\neg\neg a$ and 0 at a, which the prime ideal theorem provides as for Theorem 11. Intuitionistically, contraposition holds in one direction only: from $\neg b \to \neg a$ one obtains $a \to \neg\neg b$, not $a \to b$. Hempel's raven paradox [Hempel 1945] uses the equivalence of a hypothesis with its contrapositive. Where being black is not regular, an instance of the contrapositive, the white sock found indoors, bears on the double-negated hypothesis and not on the hypothesis itself, and dnGap measures the difference. Where both colours can be observed, being black is regular, the equivalence returns, and this diagnosis does not apply. Section 7.2 treats that case.

Because dnGap and deMorganGap are themselves defined from v, the apparent three unknowns $v(a)$, $v(\neg a)$, and $v(\neg\neg a)$ are not independent: they are part of a three-constraint system, yielding the following result:

Theorem 7 (slack in closed form[Lean])**.** *For every valuation v on a frame Ω and every $a \in \Omega$,*

$$\mathrm{slack}(a) \;=\; \big(v(\neg\neg a) - v(a)\big) \;+\; \big(1 - v(\neg\neg a) - v(\neg a)\big),$$

where $v(a) \leq v(\neg\neg a)$.

The inequality follows from monotonicity applied to $a \leq \neg\neg a$, a fact of every Heyting algebra, and shows the first summand is exactly the difference, not the floor produced by $[0, \infty]$'s truncated subtraction. The split into a regularity defect and a De Morgan defect follows from monotonicity and modularity alone, with no further axiom about ignorance required. This contrasts with the usual Dempster–Shafer picture, where the mass assigned to non-singleton, ambiguous sets is an extra primitive.

Theorem 7 expresses slack pointwise in terms of v. The converse, whether slack in turn determines v, is more delicate. One constraint is automatic:

Theorem 8 (slack is submodular[Lean])**.** *For every valuation v on a frame Ω and every $a, b \in \Omega$,*

$$\mathrm{slack}(a \sqcup b) + \mathrm{slack}(a \sqcap b) \;\leq\; \mathrm{slack}(a) + \mathrm{slack}(b).$$

The proof runs v's own modularity twice, once at (a, b) and once at $(\neg a, \neg b)$, and combines the two results with the De Morgan laws: the equality $\neg(a \sqcup b) = \neg a \sqcap \neg b$, which holds in every Heyting algebra, and the inequality $\neg a \sqcup \neg b \leq \neg(a \sqcap b)$, monotone under v but not in general an equality.

Any function proposed as the slack of some valuation must therefore be submodular.

Remark 9 (non-recovery[Lean]). Submodularity is necessary, not sufficient: Section 7.3 exhibits two valuations on the same frame with identical slack functions.

Remark 10 (slack under weak excluded middle[Lean]). The two gaps, dnGap and deMorganGap, computed at a single element, are logically independent. However, if weak excluded middle is imposed on the frame for every a ($\neg\neg a \sqcup \neg a = \top$[Lean]), then deMorganGap vanishes identically and $\mathrm{slack}(a) = \mathrm{dnGap}(a)$ for every valuation and every a. This sits strictly between plain intuitionistic logic, where both gaps can be positive, and full excluded middle, where both vanish[Lean]. A concrete instance appears in Section 7.

2.4 What Survives without Excluded Middle

Three classical pillars survive intact, and one fails in a precisely delimited way. *Disjoint additivity* is unconditional: $a \sqcap b = \bot$ implies $v(a \sqcup b) = v(a) + v(b)$. *Conditioning* is native: for $v(b) \neq 0$ the posterior $v(a \mid b) = v(a \sqcap b)/v(b)$ is again a valuation (the proof is frame distributivity), and the product rule $v(a \mid b) \cdot v(b) = v(a \sqcap b)$ and Bayes symmetry (and therefore Bayesian updating) $v(a \mid b)\, v(b) = v(b \mid a)\, v(a)$ hold[Lean]. *Total probability* holds over any genuine partition, meaning $a \sqcap a' = \bot$ and $a \sqcup a' = \top$: $v(b) = v(a \sqcap b) + v(a' \sqcap b)$. What fails is total probability over $\{a, \neg a\}$. That family need not join to $\top$, and the conditional masses fall short of $v(b)$ by a conditional slack[Lean].

The classical fragment. We characterize the elements of Ω on which the classical complement rule holds. The $\neg\neg$-stable (*regular*) elements, which form a Boolean algebra (by Glivenko's theorem), are a natural candidate, but they do not suffice. Modularity constrains the frame join $\sqcup$, whereas the Boolean join of regular elements is the regularized $\neg\neg(a \sqcup b)$, for which only the inequality $v(a) + v(b) \leq v(\neg\neg(a \sqcup b)) + v(a \sqcap b)$ survives (i.e. the 2-monotonicity of Section 5). A regular but uncomplemented element, which is an open ray $(-\infty, 0)$ in the opens of $\mathbb{R}$, under an atomic measure at 0, still carries slack. The correct classical fragment is the *complemented* elements, where $v(a) + v(a^c) = 1$ holds exactly[Lean]. On a connected locale that fragment can be as small as $\{\bot, \top\}$. In the Boolean limit, every element is complemented, the slack vanishes identically, and classical Kolmogorov axioms are recovered[Lean].

Mixtures. Valuations are closed under finite convex combination[Lean], with every axiom inherited termwise, because modularity is linear. This innocuous fact becomes a characterization in Section 4.

3 Classicality in Theory and Metatheory

Classicality enters this work twice: in the metatheory, the logic of the proof assistant itself, and in the object theory, the frame's own logic.

The metatheory. Lean's kernel is constructive, and its core library declares `Classical.choice` as an axiom, from which excluded middle follows. A theorem is constructive when its proof does not use this axiom, which `#print axioms` reports. mathlib uses the axiom throughout. The order on $\mathbb{R}$ and $[0, \infty]$ has no computable `Decidable` instance, so every library lemma that case-splits on $x \leq y$ for such numbers uses `Classical.propDecidable`, and so do the library's finite sums and finite sets. A theorem that never invokes excluded middle for frame elements $a \in \Omega$ can therefore still depend on the axiom through the library. Almost every theorem of this development does. For most of them this dependence is incidental: it comes from the library, not from the argument. Table 1 lists the arguments that use a classical principle themselves, and what a constructive version would require. We call a result constructive only in this metatheoretic sense, and we call the logic of the frame intuitionistic. A theorem about an intuitionistic frame can have a classical proof.
The arguments for M_3, the pentagon, the perfect-graph criterion, the Kochen–Specker set, and Theorem 80 use none of these principles, although their Lean proofs depend on `Classical.choice` through the library. The closure statements of Section 11.4 and the results built on them use the bipolar theorem. The Cox model of Section 6 is given an explicit computable `Decidable` instance.

Object-level classicality: the R3 axiom. Van Horn's repaired Cox theorem [Horn 2003] axiomatizes plausibility over classical propositions. Its negation axiom, called *R3*, posits that the plausibility of $\neg A$ is a fixed non-increasing function S of the plausibility of A. Consistency forces the involution $S \circ S = \mathrm{id}$, and after regraduation the complement rule $p \mapsto 1 - p$. We show that the involutive negation *is* double-negation elimination, so R3 is where classical logic enters Cox's derivation and Van Horn's. The lattice derivation of Knuth and Skilling [Knuth and Skilling 2012] avoids R3, and is classical through its decompositions into disjoint elements (Section 6). To do this, we first isolate the exact content of R3's conclusion as a predicate on valuations, then prove equivalence to excluded middle.

That predicate is *classical negation*: a valuation v has classical negation if $v(a) + v(\neg a) = 1$ for every $a \in \Omega$.

Theorem 11 (the R3 characterization[Lean]). *For every frame Ω, every valuation on Ω has a classical negation* iff $a \sqcup \neg a = \top$ *for every $a \in \Omega$. The converse is proved in a classical metatheory.*

The forward direction is a calculation, $v(a) + v(\neg a) = v(a \sqcup \neg a) + v(a \sqcap \neg a) = v(\top) + v(\bot)$, which is valid intuitionistically. Gaines states it for valuations on lattices of

Result	Classical principle	Constructive version
Converse of Theorem 11, separation (Sec. 9)	prime ideal theorem, proof by contradiction	finite lattices: proved without choice[Lean]. Infinite frames: only for frames with enough points
Sharp valuations as prime ideals (Thm. 13)	decidable membership in an ideal	for ideals with decidable membership. Finite lattices with decidable order: proved without choice[Lean]
Closure under $\otimes$, &, $\oplus$, and Thms. 92(ii), 99	bipolar theorem, by Hahn–Banach over $\mathbb{R}$	finite rational data: Fourier–Motzkin elimination, not done
Completeness (Thm. 70)	Hahn–Banach over $\mathbb{R}$	for rational points and integer certain states, proved without choice by Fourier–Motzkin elimination over $\mathbb{Z}$[Lean]
Barycenters (Thm. 72)	compactness of probability measures	finite E and rational values: the completeness row. In general the fan theorem is needed
Sharp readout (Sec. 7.4)	excluded middle for halting	not definable, replaced by a synthetic undecidability statement [Forster et al. 2019]
Non-uniqueness of products (Thm. 51)	free ultrafilter on $\mathbb{N}$	not shown on Set($\mathbb{N} \times \mathbb{N}$). Proved without choice on the algebra of sets eventually determined by the order of the coordinates[Lean]
Sierpiński model (Sec. 7.4)	bivalence of propositions	an explicit two-point space, proved without choice[Lean]
Slack is boundary mass (Thm. 33)	classical measures and set complements	finite spaces, proved without choice[Lean]. In general constructive valuations [Coquand and Spitters 2009]
Kraft bound for Ω (Sec. 7.4)	product measure on Cantor space	finite Kraft inequality, proved without choice[Lean]. Ω is the supremum of its partial sums, a lower real
Stonean characterization (Sec. 7.1)	Stone duality for spaces	none, the localic statement is the definition
Values in $[0, \infty]$, infinite sums	classical extended reals	rationals for finite results, lower reals in general, in which slack is an upper real
Aczél regraduation (Sec. 6)	classical real analysis	not attempted

Table 1. Classical principles used by the arguments. "Proved without choice" means that `#print axioms` lists at most `propext` and `Quot.sound`. These proofs use core Lean, basic mathlib order files, and mathlib's `ring` tactic on $\mathbb{Z}$, and avoid mathlib's $\mathbb{Q}$ and finite sums, which depend on `Classical.choice`.

propositions [Gaines 1978], and Weatherson for intuitionistic probability [Weatherson 2003]. The converse is, in substance, Nachbin's theorem. A sharp valuation with classical negation is the indicator of a prime filter that contains a or $\neg a$ for every a, that is, of a maximal filter, and a distributive lattice whose prime filters are all maximal is Boolean [Nachbin 1947]. The proof below is direct: wherever excluded middle fails, it produces a valuation with genuine slack. Its proof is classical in three places. It argues by contradiction, it uses the prime ideal theorem, and the valuation it builds is a $\{0, 1\}$-valued function only because membership in an ideal is classically decidable. Suppose $a \sqcup \neg a \neq \top$.

Definition 12 (ideal, prime ideal)**.** An *ideal* $J \subseteq \Omega$ is a downward closed set which is also closed under finite joins. An ideal J is *prime* if $\top \notin J$ and $a \sqcap b \in J$ implies $a \in J$ or $b \in J$.

The prime ideal separation theorem (mathlib's `DistribLattice.prime_ideal_of_disjoint_filter_ideal`) yields a prime ideal J containing $a \sqcup \neg a$ but not $\top$. The indicator of J's complement,

$$v(x) = \begin{cases} 0 & x \in J \\ 1 & x \notin J, \end{cases}$$

is a valuation. Modularity in the only nontrivial case ($x, y \notin J$) is *exactly* primeness, $x \sqcap y \in J \Rightarrow x \in J \vee y \in J$. And this valuation assigns $v(a) + v(\neg a) = 0 \neq 1$.

Sharp valuations are prime filters. The construction above is one half of an exact correspondence. Call v *sharp* if it is $\{0, 1\}$-valued, a state of full certainty. Sharpness characterizes exactly the prime-ideal indicators:

Theorem 13 (sharp valuations are points[Lean])**.** *For every valuation v on a frame Ω, v is sharp iff v is the complement-indicator of a prime ideal.*

The correspondence is given by two mutually inverse maps: $v \mapsto Z(v) = \{x \in \Omega : v(x) = 0\}$, which computes the zero-set of a sharp valuation, and the map taking J to its complement-indicator (the valuation of Theorem 11's construction). $Z(v)$ is prime exactly because v is sharp, and the complement-indicator of $Z(v)$ recovers v itself. The theorem pairs sharp valuations with the prime ideals whose membership is decidable, which classically are all of them. Their complements are the prime filters of Ω, not yet the points of the locale. A prime ideal is *completely prime* when it is also closed under arbitrary (not just finite) suprema of its members. The finitely prime points of Theorem 13 sharpen to the completely prime points exactly where Scott continuity is added, recovering points of the locale in the sense used by [Coquand and Spitters 2009; Vickers 2008].

Definition 14 (directed set, Scott continuity)**.** A subset $D \subseteq \Omega$ is *directed* if, for any finite subset $F \subseteq D$, there exists an element $d \in D$ that is an upper bound of F (taking $F = \emptyset$

shows a directed set is nonempty). A valuation v is *Scott-continuous* if $v(\bigsqcup D) = \sup_{a \in D} v(a)$ for every directed D.

Theorem 15 (Scott-continuous sharp valuations[Lean]). *For every valuation v on a frame Ω, v is sharp and Scott-continuous iff v is the complement-indicator of a completely prime ideal.*

The characterization and the two correspondences together locate the object-level classicality precisely: whenever excluded middle fails at some a, the failure occurs at a point of the locale.

Points and the adjunction with spaces. Classically the initial frame is $[1] = \{0 < 1\}$. Intuitionistically it is the frame of truth values, a frame homomorphism into it need not be $\{0,1\}$-valued, and the $\{0,1\}$-valued ones are the points with decidable membership. mathlib defines points as frame homomorphisms into `Prop`, and the comparison below is classical. For a lattice P, monotone maps $P \to [1]$ correspond to upper sets (the preimage of 1), bounded lattice homomorphisms to prime filters, and frame homomorphisms to completely prime filters, those F with $\bigsqcup S \in F$ only if $S \cap F \neq \emptyset$. The complements are the lower sets, prime ideals, and completely prime ideals of Theorem 13 and the Scott-continuous characterization above. The points of a frame P, $\mathrm{pt}(P) = \mathrm{Frm}(P, [1])$, carry the topology of the sets $\Sigma_a = \{f \mid f(a) = 1\}$, and Σ preserves finite meets and arbitrary joins. With $\mathcal{O}$ sending a space to its frame of opens, this gives the adjunction $\mathcal{O} \dashv \mathrm{pt}$ between spaces and locales [Johnstone 1982], in mathlib as `Locale.adjunctionTopToLocalePT`. Its units send a point x to $U \mapsto [x \in U]$ and an element a to Σ_a. A prime filter need not be completely prime: in $[0,\infty]$, the filter $\{\infty\}$ is prime but contains $\bigsqcup_n n = \infty$ and no n. The corresponding sharp valuation is not Scott-continuous, as for the $\top$-indicator on $\mathrm{Low}(\mathbb{N})$ of Section 4[Lean].

Finite frames without choice. The prime ideal theorem is a choice principle, and it is not available constructively. Without it, a nontrivial distributive lattice can have no prime filters, and then it has no sharp valuations. Independently of choice, a frame can have no points, that is, no Scott-continuous sharp valuations [Johnstone 1982]. So the converse of Theorem 11 and the statement that sharp valuations reconstruct the logic (the first criterion of Section 1) are proved here only in a classical metatheory. On a finite distributive lattice with decidable order the needed prime filter is found by search: an element below a and not below b that is minimal for the size of its downset is join-prime, and the indicator of its principal filter is a sharp valuation. This proves the finite separation theorem and the finite form of the converse of Theorem 11 without `Classical.choice`[Lean Lean], for $\mathbb{N}$-valued valuations. In a constructive metatheory we therefore have the first criterion for finite event algebras with decidable order. For an infinite frame it would require enough points with decidable membership, which the open sets of the real line do not offer, since constructively it is not decidable whether a real number lies in an open interval.

A development in Rocq. Rocq's kernel, like Lean's, does not assume excluded middle or choice, so the two differ in their libraries more than in their logic. The differences would be the following.

- *Finite results.* The Mathematical Components library [Mahboubi and Tassi 2021] works with types with decidable equality and finite types, and avoids classical axioms in its algebraic core. We expect the finite theorems of this paper (the finite representation, the frame recipe, M_3, the pentagon, the perfect-graph criterion, the Kochen–Specker set, and the axioms of finite frames and of the second-order type over Bool in Section 14) to be provable there without classical axioms and to extract to programs. We have not checked this.
- *Values.* $[0,\infty]$ is a classical object. A constructive development would use rational values for finite results and lower reals, approximable from below, for valuations in general [Coquand and Spitters 2009; Vickers 2008]. The value of the halting event, Chaitin's Ω, is a lower real of this kind: it is approximated from below by the halting programs found so far, and no better. Finite values would also remove the restriction to finite-valued functions in Section 14.
- *Polytopes.* Section 14 does not formalize that the axioms of a calculus are its facet laws, and it computes the facets at the second-order type without a proof. The Coq-Polyhedra library, built on Mathematical Components, contains proofs of Farkas's lemma, linear programming duality and Minkowski's theorem [Allamigeon and Katz 2019] and of the structure of the face lattice of a polyhedron [Allamigeon et al. 2022], and computational certificates of this kind have been checked for polytopes with tens of thousands of vertices [Allamigeon et al. 2023]. Both steps would be within reach in Rocq.
- *Analysis.* MathComp-Analysis [Affeldt et al. 2018] assumes classical axioms, as mathlib does. The completeness theorem for rational points is the Fourier–Motzkin argument above, which carries over directly. Completeness over real points would require the constructive analysis of Bishop's school, with locatedness hypotheses for hyperplane separation. The barycenter theorem for infinitely many events requires compactness of $\{0,1\}^{\mathbb{N}}$, which constructively is the fan theorem, a principle that would have to be added as an axiom.
- *Computability.* Without excluded middle the sharp classical readout cannot be defined. The uncomputability statement would be phrased synthetically [Forster et al. 2019]: no function from

codes to Booleans decides halting, which requires an axiom such as Church's thesis to state as a negative result about all functions.
- *Points.* The separation results would be stated for frames with enough points, or for finite lattices as above.

Lean was chosen for mathlib's measure theory, order theory and topology, which the infinite results use. The cost is that constructive content is a property to be checked theorem by theorem with `#print axioms`, rather than a default.

A natural direction for future work is to develop the extension of logics to probability calculi in a constructive metatheory throughout. This would give algorithms in place of existence proofs, for instance the weights of a convex combination, the separating laws of Theorem 70, and the axioms of Section 14, computed from the certain states by facet enumeration, and would match the semidecidable reading of events with value types such as lower reals. It would also restrict the results that depend on points existing to frames with enough points.

4 Valuation Representation

Every valuation on a finite frame is a probability mass function on points. On infinite frames, this can fail, and Scott continuity marks the boundary between where it still holds and where it does not.

4.1 Finite Frames Representation

Theorem 16 (Birkhoff duality [Birkhoff 1937])**.** *Every finite distributive lattice is isomorphic to the lattice of lower sets of a finite poset: its sup-irreducible elements form a poset P whose lower sets reconstruct the lattice, and every finite poset arises this way from its own lattice of lower sets.*

Let P be the finite poset of sup-irreducible elements of a finite frame. Then, by Birkhoff duality (`OrderIso.lowerSetSupIrred`), every finite frame is the lattice of *lower sets* of P. We fix the following notation, used throughout the rest of the paper. A *lower set* $L \subseteq P$ satisfies $p \in L$, $q \le p \Rightarrow q \in L$, and an *upper set* is defined dually. Lower sets are always denoted L, and upper sets U (opens of a topological space are still U, V, W, as in traditional usage). The *principal lower set* at p is $\downarrow p = \{q : q \le p\}$, the *strict principal lower set* at p is $\Downarrow p = \{q : q < p\}$, and the *principal upper set* at p is $\uparrow p = \{q : q \ge p\}$. We prove the finite representation theorem below on the lower sets of P. Transporting it across the isomorphism above then gives the theorem for every finite frame, not just lower sets[Lean]. Let us define the *mass* of a point as the discrete derivative of v[Lean]:

$$\mathrm{mass}(p) \;=\; v(\downarrow p) - v(\Downarrow p).$$

The masses recover v and sum to 1:

Theorem 17 (finite representation[Lean])**.** *For a finite P, any valuation v on $\mathrm{Low}(P)$, and any $L \in \mathrm{Low}(P)$, $v(L) = \sum_{p \in L} \mathrm{mass}(p)$, and $\sum_p \mathrm{mass}(p) = 1$.*

The proof works by induction on $|L|$: repeatedly remove a maximal point p from L, and use modularity applied to $(L \setminus \uparrow p,\ \downarrow p)$ to show each removal changes v by exactly $\mathrm{mass}(p)$. The mass function is a classical probability mass function on P[Lean]. And since lower sets are exactly the opens of the Alexandrov topology, where interior is the identity, the theorem also reads $v = \mu \circ \mathrm{int}$, i.e. every valuation on a finite frame is the classical measure μ applied after taking the interior. In the finite case, this is the converse of the measure-theoretic bridge of Section 7. The intuitionistic mass also agrees with the classical notion of the mass of a point: reading $v = \mu \circ \mathrm{int}$,

$$\mathrm{mass}(p) \;=\; \mu(\downarrow p) - \mu(\Downarrow p) \;=\; \mu(\{p\}),$$

since $\downarrow p$ is the disjoint union of $\Downarrow p$ and $\{p\}$ and μ is defined on all subsets of P.

Theorem 17 represented v as a sum of point masses. The dual view represents v as a mixture of sharp *point-valuations*. For each $p \in P$, let δ_p be the valuation with $\delta_p(L) = 1$ if $p \in L$, and $\delta_p(L) = 0$ otherwise:

Theorem 18 (mixture characterization[Lean])**.** *On a finite frame, every valuation v satisfies $v = \sum_p \mathrm{mass}(p) \cdot \delta_p$.*

Together with the fact that any mixture of valuations is again a valuation[Lean], this theorem characterizes the valuations on a finite frame exactly: they are the finite convex mixtures of point-valuations, closed under mixing because modularity is linear. Section 6 shows modularity cannot instead be *derived* from more primitive axioms. It must be posited outright, and doing so is exactly what makes every valuation a mixture of sharp, all-or-nothing certainties. The characterization is an instance of a theorem of Paris: for a propositional logic whose two-valued valuations respect $\wedge$ and $\vee$, such as intuitionistic logic, the functions that give tautologies 1 and contradictions 0, are monotone for the logic's consequence relation, and are modular are, on every finite set of sentences, exactly the mixtures of valuations [Paris 2001].

4.2 Infinite Boundary Representation via Scott Continuity

Both halves of the finite theorem fail for infinite frames:

Example 19 (obstruction[Lean])**.** On $\mathrm{Low}(\mathbb{N})$, the indicator v of $\top$ is a valuation with $\mathrm{mass}(p) = 0$ for every point p, and $v(\top) = 1$.

No point carries any of the mass. The failure is one of approximation: $v(\top)$ is not the supremum of $v(\downarrow n)$ over the finite stages. The $\top$-indicator of Example 19 is not Scott-continuous[Lean], and Scott continuity is exactly what restores the representation:

Example 20 (recovery[Lean]). Let v be a valuation on $\mathrm{Low}(\mathbb{N})$. If $v(\top) = \sup_n v(\downarrow n)$ then $v(\top) = \sum_n \mathrm{mass}(n)$.

In general, on any poset all of whose principal lower sets are finite, Scott continuity implies zero diffuse part without additional preconditions. However, subtraction in $[0,\infty]$ is truncated at 0, so if $\sum_p \mathrm{mass}(p)$ were greater than $v(\top)$, the expression $v(\top) - \sum_p \mathrm{mass}(p)$ would still evaluate to 0. Theorem 21 rules out $\sum_p \mathrm{mass}(p) > v(\top)$ unconditionally, so the subtraction below is never truncated:

Theorem 21 (general decomposition[Lean]). *For every valuation v on $\mathrm{Low}(P)$ and an arbitrary poset P, $\sum_p \mathrm{mass}(p) \le v(\top)$.*

By Theorem 21 the remainder $v(\top) - \sum_p \mathrm{mass}(p)$ is a genuine difference. Every valuation thus splits unconditionally into an *atomic* part carried by points ($\sum_p \mathrm{mass}(p)$, which is always a sub-probability), and a *diffuse* (nonnegative) remainder $v(\top) - \sum_p \mathrm{mass}(p)$. This splits the total mass, not the valuation itself: the remainder is a number, and it is not shown to be the total of a valuation without point masses. The inequality also follows from the extension of v to a finitely additive measure on the Boolean algebra generated by $\mathrm{Low}(P)$ [Pettis 1951], whose singletons carry the point masses. The finite and Scott cases are the purely atomic extremes, and the $\top$-indicator is the purely diffuse one. The proof of Theorem 21 re-runs the maximal-point induction on each finite subset of P, bounding its partial sum by $v(\top)$, then takes the supremum over all finite subsets to bound $\sum_p \mathrm{mass}(p)$ itself. Its proof does not directly invoke an excluded-middle case split. However, it relies on the (classical) decidability of the ordering of the reals (a default in mathlib). Representing the diffuse part on non-spatial frames remains the open frontier. The obstruction there is again that the frame has *too few points*, the same one as behind the undecidability of Section 7.

5 The Dempster–Shafer Reading

Sections 2–4 independently arrive at the same structure as Dempster–Shafer (DS) theory [Dempster 1967; Shafer 1976], including sub-additive complements, a belief/plausibility interval, and mass functions. This section makes the identification exact in both statics and dynamics. In each case, it locates the precise point where excluded middle makes a difference.

5.1 Total Monotonicity via Intuitionistic Inclusion–Exclusion

The $\neg\neg$-stable (*regular*) elements of a frame form a Boolean algebra with meet $\sqcap$ and join $\neg\neg(a \sqcup b)$ (Glivenko). Let us call the resulting algebra the *Booleanization* of this set.

Definition 22 (belief function, plausibility function[Lean]). Let B be a Boolean algebra. A DS *belief function* on B is a monotone map $\mathrm{Bel} : B \to [0,1]$ with $\mathrm{Bel}(\bot) = 0$ and $\mathrm{Bel}(\top) = 1$ that is *n-monotone* for every $n \ge 2$: for all $A_1, \ldots, A_n \in B$,

$$\mathrm{Bel}\Big(\bigvee_{i=1}^{n} A_i\Big) \;\ge\; \sum_{\emptyset \ne T \subseteq \{1,\ldots,n\}} (-1)^{|T|+1}\, \mathrm{Bel}\Big(\bigwedge_{i\in T} A_i\Big).$$

Its *plausibility function* is the map $\mathrm{Pl} : B \to [0,1]$ given by $\mathrm{Pl}(A) = 1 - \mathrm{Bel}(\neg A)$.

We show v, restricted to its Booleanization, is such a belief function. Belief functions have also been developed directly on distributive lattices [Zhou 2013]. The 2-monotone case is immediate from modularity [Lean]. The full tower follows from the following result:

Theorem 23 (inclusion–exclusion[Lean]). *For every valuation on every frame Ω, every finite family $a : \iota \to \Omega$, and every finite $s \subseteq \iota$, inclusion–exclusion holds with equality for the frame join:*

$$v\Big(\bigsqcup_{i\in s} a_i\Big) \;=\; \sum_{\emptyset \ne T \subseteq s} (-1)^{|T|+1}\, v\Big(\bigsqcap_{i\in T} a_i\Big).$$

We state this in Lean additively, with the odd-cardinality half of the signed sum set against the even-cardinality half, so that no $[0,\infty]$ subtraction occurs. The proof is by induction with modularity and frame distributivity.

Replacing the frame join $\bigsqcup_i a_i$ on the equality's other side with its double negation $\neg\neg\big(\bigsqcup_i a_i\big)$, the Booleanization's own join, yields the following inequality:

Theorem 24 (total monotonicity[Lean]). *For every valuation on every frame Ω, every finite family $a : \iota \to \Omega$, and every finite $s \subseteq \iota$:*

$$\sum_{\emptyset \ne T \subseteq s} (-1)^{|T|+1} v\Big(\bigsqcap_{i\in T} a_i\Big) \;\le\; v\Big(\neg\neg\Big(\bigsqcup_{i\in s} a_i\Big)\Big).$$

The Booleanization's join is given by $\neg\neg \circ \sqcup$, and its meets of regular elements are again regular. Interpreted in this way, Theorem 24 states exactly the ∞-monotone tower: v restricted to its Booleanization is a belief function.

The proof of Theorem 24 from Theorem 23 follows from the monotonicity along $a \le \neg\neg a$. Here, inclusion–exclusion does *not* fail on a frame. It is exact for the intuitionistic disjunction. Applying the double-negation nucleus (a closure operator preserving finite meets) $\neg\neg$ to the join on the equality's right-hand side can only enlarge it. This follows from the fact that $a \le \neg\neg a$ and that v is monotone, which turns the equality into the inequality of Theorem 24. The amount by which the right-hand side is enlarged is exactly the double-negation gap of Section 2.

Definition 22 is the classical notion, stated on an abstract Boolean algebra. In our setting each of its ingredients is already present: the Boolean algebra is the Booleanization, the belief function is v itself, restricted to the regular elements and evaluated against the Boolean join $\neg\neg \circ \sqcup$, and the plausibility function is $\mathrm{Pl}(a) = 1 - v(\neg a)$. Additive probability does not become a belief function by weakening an axiom, since

modularity for v is never relaxed. What changes is only the join used to state the tower. The belief–plausibility interval is governed by the slack:

Theorem 25 (interval width[Lean]). *For every valuation v on a frame Ω and every $a \in \Omega$, $\mathrm{Pl}(a) - v(a) = \mathrm{slack}(a)$.*

5.2 The Conditioning Characterization

DS theory carries two update rules that coincide classically. *Geometric* conditioning renormalizes belief, and under the DS dictionary it *is* the localic posterior $v(a \sqcap b)/v(b)$ of Section 2.

Definition 26 (Dempster's rule of conditioning[Lean]). For a valuation v on a frame Ω, elements $a, b \in \Omega$, and evidence of nonzero belief, $v(b) \neq 0$, *Dempster's rule of conditioning* [Shafer 1976] states that

$$v(a \parallel b) \;=\; \frac{v(a \sqcup \neg b) - v(\neg b)}{1 - v(\neg b)}.$$

The hypothesis $v(b) \neq 0$ makes the denominator nonzero, since $v(b) + v(\neg b) \leq 1$ gives $v(b) \leq 1 - v(\neg b)$.

That is, the DS conditioning rule discards the mass incompatible with the evidence and renormalizes by its plausibility. Both the geometric posterior of Section 2 and Dempster's rule of Definition 26 are again valuations. Their modularity is the frame distributivity $(a \sqcup \neg b) \sqcap (a' \sqcup \neg b) = (a \sqcap a') \sqcup \neg b$. Their disagreement is given by the *excluded-middle gap* $\mathrm{emGap}(a, b) = v(a) - v(a \sqcap (b \sqcup \neg b))$, i.e. the mass of a outside b's instance of excluded middle:

Lemma 27 (the join–complement identity[Lean]). *For every valuation v on every frame Ω and every $a, b \in \Omega$, $v(a \sqcup \neg b) = v(\neg b) + v(a \sqcap b) + \mathrm{emGap}(a, b)$.*

Dempster's numerator therefore exceeds Bayes's by exactly $\mathrm{emGap}(a, b)$. This gap depends on a, but whether it vanishes at *every* a depends only on the slack at the evidence b:

Theorem 28 (the conditioning characterization[Lean]). *For every valuation v on every frame Ω and every $b \in \Omega$ with $v(b) \neq 0$, $\big(v(a \parallel b) = v(a \sqcap b)/v(b)$ for every $a \in \Omega\big)$ iff $\mathrm{slack}(b) = 0$.*

The hypothesis $\mathrm{slack}(b) = 0$ eliminates the gap $\mathrm{emGap}(a, b)$ for every a and makes the two normalizers coincide, since $1 - v(\neg b) = v(b)$. Conversely, testing at $a := b$, geometric conditioning gives certainty 1, while Dempster gives $v(b)/(1 - v(\neg b))$, which is 1 only if the slack vanishes. Theorem 28 is the valuation-relative analogue of Theorem 11: both are iff statements about the same proposition, $b \sqcup \neg b = \top$.

On a finite frame $\mathrm{Low}(P)$ the two rules have a reading in certain states. By Theorem 71, $v = \sum_p w_p\, \delta_p$ is a mixture of point valuations, and δ_p verifies b if $p \in b$, refutes it if $p \in \neg b$, and leaves it undecided otherwise. Expanding the definitions,

$$\frac{v(a \sqcap b)}{v(b)} = \frac{\sum_{p \in a \cap b} w_p}{\sum_{p \in b} w_p}, \qquad v(a \parallel b) = \frac{\sum_{p \in a \setminus \neg b} w_p}{\sum_{p \notin \neg b} w_p}.$$

Geometric conditioning averages over the certain states that verify the evidence, and Dempster's rule over those that do not refute it. The undecided states carry total weight $\mathrm{slack}(b)$, so the rules agree iff there are none of positive weight, which is Theorem 28 on finite frames. This reading is not formalized. Neither rule follows from criteria (1)–(3), which constrain the calculus and not its updates. The choice between them is the choice of which certain states remain possible after the evidence.

We call the frame-level fact *static* and the valuation-level fact *dynamic*: the former is an equation in the frame itself, between $b \sqcup \neg b$ and $\top$, with no valuation involved, while the latter tests the same proposition through v, since $\mathrm{slack}(b) = 0$ is exactly $v(b \sqcup \neg b) = v(\top)$. Dempster and Bayes therefore coincide at b whenever v cannot distinguish $b \sqcup \neg b$ from $\top$, even if the two differ in the frame itself. In the measure model, Dempster and Bayes disagree precisely when the evidence's boundary carries mass.

6 Cox's Theorem

Cox's theorem [Cox 1946; Jaynes 2003] derives the probability calculus from conditions on plausibility. After Halpern's counterexample [Halpern 1999], its repaired forms [Arnborg and Sjödin 2000; Horn 2003] fix the calculus up to monotone regraduation. Cox's derivation is classical through the negation axiom R3, which Section 3 shows *is* excluded middle. The lattice derivation of Knuth and Skilling [Knuth and Skilling 2012] avoids R3 and derives the sum rule from additivity on disjoint joins, and Example 30 shows that this step needs more than a distributive lattice. Dropping R3 leaves the product rule unaffected, since it is logic-free, and requires the sum rule to become an axiom rather than a theorem. Separately, the naive statement over frames, demanding a regraduation strictly monotone on all of $\mathbb{R}$, is unsatisfiable (Section 6.3).

6.1 A Cox Structure without Negation

Definition 29 (Cox model[Lean]). A *Cox model* on a frame Ω is a plausibility assignment $\mathrm{pl} : \Omega \times \Omega \to \mathbb{R}$, written $\mathrm{pl}(a, b)$ for the plausibility of a given b, together with a *conjunction functional* $F : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$, such that $\mathrm{pl}(\cdot, c)$ is monotone for every c, the boundary conditions $\mathrm{pl}(\bot, c) = 0$ and $\mathrm{pl}(\top, c) = 1$ hold for every c, the product rule $\mathrm{pl}(a \sqcap b, c) = F(\mathrm{pl}(a, b \sqcap c), \mathrm{pl}(b, c))$ holds, F is associative, and $x \mapsto F(x, y)$ is strictly monotone for every $y > 0$.

The missing R3 axiom can be observed in the omission of any constraint on the relation between $\mathrm{pl}(\neg a, b)$ and $\mathrm{pl}(a, b)$. The definition restricts the monotonicity of F to $0 < y$ to

avoid a contradiction with the fact that the boundary axioms force $F(x, 0) = 0$ for all x. An explicit model on the non-classical chain $[0, \infty]$ satisfies this constraint[Lean], as $a = \top$ is provably decidable.

6.2 The Sum Rule Is Irreducible

Classically, additivity is *derived*: R3 turns disjunction into negated conjunction via De Morgan, and the product rule does the rest. Intuitionistically that route is closed, since $a \sqcup b \neq \neg(\neg a \sqcap \neg b)$ in general:

Example 30 (the sum rule is irreducible[Lean])**.** Let Ω be the five-element frame of lower sets of the poset $o < a$, $o < b$ with a, b incomparable. Then there is a monotone, normalized assignment q on Ω that is additive on disjoint joins yet not modular, and no binary operation S on values satisfies $q(x \sqcup y) = S(q(x), q(y))$ for all x, y. Moreover, the pairs $(\downarrow a, \downarrow a)$ and $(\downarrow a, \downarrow b)$ have identical value-pairs $(\frac{1}{2}, \frac{1}{2})$ but joins valued $\frac{1}{2}$ and 1.

Darst gives this lattice, as the lattice of sets $\emptyset$, $\{1\}$, $\{1, 2\}$, $\{1, 3\}$, $\{1, 2, 3\}$, with a function that is additive on disjoint sets and not modular [Darst 1970]. The absence of a binary disjunction functional is not specific to frames: for a probability with $P(A) = \frac{1}{2}$ the pairs (A, A) and $(A, \neg A)$ behave in the same way. What is specific is the failure of modularity under additivity on disjoint joins. Knuth and Skilling derive the sum rule from the disjoint case and state it for distributive lattices [Knuth and Skilling 2012], and Goertzel applies their symmetries to Heyting algebras [Goertzel 2017]. The derivation decomposes elements into disjoint parts. In this distributive frame every two nonempty lower sets meet, the disjoint case carries no information, and modularity fails. So Cox's theorem over frames must posit the sum rule. It is posited as modularity, which says that the calculus is closed under mixing certainties, justified by Theorem 18. The failure persists under weak excluded middle: the counterexample frame satisfies it[Lean], since every nonempty lower set contains the bottom point o.

6.3 Cox Model Theorem

A regraduation is a map $g : \mathbb{R} \to [0, \infty]$ with $g(0) = 0$. No such map is strictly monotone on all of $\mathbb{R}$, since strictness would force $g(-1) < g(0) = 0$, impossible in $[0, \infty]$. The repair (strictness only on $[0, 1]$, where plausibilities live) is exactly the same domain restriction that Halpern's critique of the classical theorem turns on. The corrected statement is:

Theorem 31 (Cox's theorem over frames[Lean])**.** *For every frame Ω, every Cox model on Ω whose unconditional plausibility is modular regraduates to a valuation: there exist a valuation v on Ω and a strictly monotone function g on $[0, 1]$ with $g(0) = 0$, $g(1) = 1$, and for all $a \in \Omega$, $v(a) = g(\mathrm{pl}(a, \top))$.*

The regraduation for the sum-rule half is straightforward (g = `ENNReal.ofReal`). The product-rule half, showing F regraduates to multiplication, rests on Aczél's associativity theorem [Aczél 1966], a statement of pure analysis whose type never mentions Ω. mathlib has no Aczél theorem, no Hölder-style embedding for ordered semigroups on intervals, and no Cauchy functional equation under monotonicity, so we build the analytic core from scratch, obtaining a normalized, strictly monotone multiplicative generator on the positive cone[Lean]. The construction also weakens Hölder's classical hypotheses [Hölder 1901]: the Archimedean property is *derived* from continuity and positivity rather than assumed, and the interval form of divisibility follows from the intermediate value theorem[Lean] applied to a plain divisibility hypothesis[Lean]. The dependence on the logic of Ω is therefore *entirely* in the sum rule: nothing in the product-rule apparatus mentions Ω. That apparatus uses classical real analysis in the metatheory (Table 1). Appendix C gives the construction. The residual analytic gap is discussed in Section 16.

7 Two Grounding Models

Two concrete models demonstrate that our abstract theory is non-degenerate. One shows every classical measure *contains* an intuitionistic probability. The other shows the intuitionistic theory cannot collapse back without solving the halting problem.

7.1 Measures through the Interior Operator

Gödel–McKinsey–Tarski interpret intuitionistic logic in modal S4: propositions are *open* sets, and the modal necessity operator $\Box$ of S4 is interpreted as the topological interior, written $\mathrm{int}(\cdot)$ below [Gödel 1933; McKinsey and Tarski 1944]. Our valuations quantify that same translation:

Theorem 32 (measures as valuations[Lean])**.** *Let μ be a probability measure on a space X, $A \subseteq X$, and define $v(A) = \mu(\mathrm{int}\, A)$. Since $\mathrm{int}\, U = U$ for open U, v restricts on the frame $\mathcal{O}(X)$ to $U \mapsto \mu(U)$, and this restriction is a valuation.*

Modularity is $\mu(U \cup V) + \mu(U \cap V) = \mu(U) + \mu(V)$. The slack of an open U is then $1 - \mu(U) - \mu(\mathrm{int}\, U^c)$, which is the measure of the boundary ∂U. We can now prove that, in general, slack is exactly the boundary mass:

Theorem 33 (slack is boundary mass[Lean])**.** *For any probability measure μ on a topological space X and any open U,* $\mathrm{slack}(U) = \mu(\partial U)$.

The Heyting complement of U in $\mathcal{O}(X)$ is the interior of its set complement. So, the undecided region, the complement of $U \sqcup \neg U$, is exactly $\overline{U} \setminus U = \partial U$. The general sub-additivity inequality of Section 2 still holds here[Lean]. This identification of slack with boundary mass also exhibits the modal identification $\mathrm{Bel} = P(\Box\, \cdot)$ of the DS literature [Ruspini 1986] arising from the logic itself. Theorem 17 is the converse in

the finite case (in the general case, the converse is the open problem, see Section 4).

The same decomposition locates a failure of classical probability calculus in conditioning, that is, in updating a belief about an open A on evidence given by an open B. The Boolean complement of A splits into its Heyting complement and its boundary, $A^c = \operatorname{int} A^c \sqcup \partial A$, so a classical posterior that treats A as confirmed-or-refuted overstates the refutation by the boundary mass the evidence retains:

Theorem 34 (classical overconfidence in refutation[Lean])**.** *Let μ be a probability measure on a topological space X, let v be its restriction to $\mathcal{O}(X)$, and let A, B be opens with $\mu(B) \neq 0$. Then*

$$\frac{\mu(A \cap B)}{\mu(B)} = v(A \mid B) \qquad \text{and} \qquad \frac{\mu(A^c \cap B)}{\mu(B)} = v(\neg A \mid B) + \frac{\mu(\partial A \cap B)}{\mu(B)}.$$

The excess term is the conditional slack of A given B, the mass that the evidence leaves neither confirmed nor refuted. The classical posterior does not discard this mass. It credits all of it to A^c, an event no finite observation confirms. For the halting event of Section 7.4 with $B = \top$, the classical credence in non-termination is $1 - p$ while the intuitionistic one is 0, and the difference $1 - p$ is exactly the slack.

7.2 Universal Laws

Theorem 34 applies to universal laws such as "all ravens are black" [Hempel 1945]. Let I be an infinite set of objects and O a topological space of kinds of object, with an open set $bad \subseteq O$ of counterexamples, the non-black ravens. A world is a function $x : I \to O$, and the worlds carry the product topology, so an open set of worlds is a property verified by observing finitely many objects. The law is $H = \{x \mid x_i \notin bad \text{ for every } i\}$, and the counterexample event is its complement $C = \{x \mid x_i \in bad \text{ for some } i\}$, which is open. For $j \in I$ and $A \subseteq O$ let $B_A = \{x \mid x_j \in A\}$, and let C_j be the event of a counterexample at an object other than j.

Theorem 35 (universal laws[Lean])**.** *Let bad be nonempty. (i) $\neg C = \bot$ in $\mathcal{O}(I \to O)$. So every valuation v has $v(\neg C) = 0$, also after conditioning on any open, and $\operatorname{slack}(C) = 1 - v(C)$. (ii) For every probability measure μ on the worlds whose open sets are measurable, $\mu(H) = \operatorname{slack}(C)$, and $\mu(H \cap B)/\mu(B) = \mu(\partial C \cap B)/\mu(B)$ for every open B with $\mu(B) \neq 0$. (iii) Let $A \subseteq O$ be open with $\mu(B_A) \neq 0$, and let C_j be independent of B_A and of $B_{A \cap bad}$ under μ. Then*

$$\frac{\mu(C \cap B_A)}{\mu(B_A)} = \mu(C_j) + \frac{\mu(B_{A \cap bad})}{\mu(B_A)}\,\mu(C_j^c).$$

For (i), a neighbourhood of a world constrains finitely many objects, so it contains a world with a counterexample at another object. Hence C is dense and H has empty interior. No finite observation verifies H, and the intuitionistic credence in it is 0 on any evidence. Part (ii) follows from (i) and Theorems 33 and 34, since $\partial C = H$. The classical credence in the law, prior or posterior, is boundary mass of C, the mass of worlds the evidence has not refuted and cannot verify. Part (iii) splits $C \cap B_A$ at object j. When O is discrete, B_A is clopen, so $\operatorname{slack}(B_A) = 0$ and Dempster's rule gives the same posteriors (Theorem 28). The choice of update rule plays no part below.

Hempel's paradox arises because H is classically equivalent to "every non-black thing is a non-raven", so a white sock found indoors is an instance of H and classically confirms it. In $\mathcal{O}(I \to O)$ the law is not an event: its intuitionistic counterpart $\neg C$ is $\bot$, and evidence bears on it only through the open event C. Part (iii) measures that bearing. Let B_A say that object j is a raven, or that it is non-black. With O discrete, observing its full kind, a black raven or a white sock, is the case of A a single kind outside bad, and it gives the credence $\mu(C_j)$ in a counterexample in both cases. The difference from the credence after selection alone is $\mu(B_{A \cap bad})/\mu(B_A)$, the probability that the selected object is a counterexample, times $\mu(C_j^c)$. For a selected raven the first factor is the probability that a raven is non-black. For a selected non-black thing it is the probability that a non-black thing is a raven, which is small when non-black things far outnumber non-black ravens. So when $\mu(C_j^c) > 0$ the white sock lowers the credence in a counterexample, by much less than the black raven does. Both conditions matter. Without independence the direction can reverse: Good gives a prior under which a black raven lowers the credence in H [Good 1967]. And under an independent prior in which every object is a counterexample with probability at least some $\varepsilon > 0$, the second Borel–Cantelli lemma gives $\mu(C) = 1$, so $\mu(H) = 0$ and no observation moves either. This is the Bayesian resolution of Hosiasson-Lindenbaum and its successors, surveyed by Fitelson and Hawthorne [Fitelson and Hawthorne 2010; Hosiasson-Lindenbaum 1940]. What the verification topology changes is the reading of the decrease of $\mu(C)$. With finitely many objects and O discrete, every set of worlds is open, $\neg C = H$, and the two calculi coincide. With infinitely many objects they differ. Classically, a decrease of $\mu(C)$ is an equal increase of the credence $\mu(H) = 1 - \mu(C)$ in the law. Intuitionistically $v(\neg C)$ stays 0 by (i), so the same decrease is an increase of $\operatorname{slack}(C)$, the mass that is unrefuted and that no evidence turns into verification. The resolution uses only the open event C and never treats H as an event.

To characterize weak excluded middle spaces, we give the (standard) definitions required:

Definition 36 (extremally disconnected)**.** A topological space X is *extremally disconnected* if the closure of every open set is again open.

Definition 37 (Stone space, Stonean space)**.** The *Stone space* of a Boolean algebra B is the space of ultrafilters on B, topologized by the clopen basis $\{U : b \in U\}$ for $b \in B$. *Stone duality* [Johnstone 1982] identifies Boolean algebras with exactly the compact, Hausdorff, totally disconnected spaces arising this way. A *Stonean space* is a compact Hausdorff

space that is also extremally disconnected. Equivalently, it is the Stone space of a *complete* Boolean algebra.

Remark 38 (weak excluded middle on opens[Lean]). Section 2 noted that imposing weak excluded middle on the whole frame collapses slack to pure double-negation instability. For $\Omega = O(X)$, this global condition holds exactly when X is extremally disconnected [Johnstone 1980].

An immediate corollary gives the characterization:

Corollary 39 (Stonean characterization[Lean]). *Given a compact Hausdorff space X, weak excluded middle on $O(X)$ holds exactly when X is a Stonean space.*

7.3 Slack Does Not Determine the Valuation

Submodularity (Theorem 8) is a necessary condition on slack. It is not sufficient: distinct valuations, each satisfying every axiom a valuation must satisfy, can share a slack function.

Example 40 (boundaries in the three-point space[Lean]). Let ThreePt $= \{p_A, p_B, p_Z\}$ carry the topology generated by the two singletons $\{p_A\}$ and $\{p_B\}$, so its opens are exactly $\emptyset$, $\{p_A\}$, $\{p_B\}$, $\{p_A, p_B\}$, and the whole space. For every open U, $\partial U \subseteq \{p_Z\}$.

This is a fact about the topology alone, independent of any measure: p_Z has no proper open neighbourhood, so it is the only point any open's frontier can contain.

Example 41 (non-recovery[Lean]). There exist valuations $v \neq v'$ on O(ThreePt) whose slack functions are identical: $\mathrm{slack}_v = \mathrm{slack}_{v'}$ as functions.

For weights $a, b, c \geq 0$ with $a + b + c = 1$, the point measure $\mu_{a,b,c} = a\,\delta_{p_A} + b\,\delta_{p_B} + c\,\delta_{p_Z}$, read through Section 7.1's bridge, gives a valuation $v_{a,b,c}$ on O(ThreePt). By Example 40 and Theorem 33, its slack at an open U is $\mu_{a,b,c}(\partial U)$, equal to c when $p_Z \in \partial U$ and to 0 otherwise: a quantity depending only on c, not on how the remaining mass splits between a and b. So $v_{1/4,1/4,1/2}$ and $v_{1/8,3/8,1/2}$ have identical slack functions while disagreeing at $\{p_A\}$, where they take the values $1/4$ and $1/8$. Both are fully modular, monotone, and normalized: nothing in Theorem 8 or its proof distinguishes them.

Weak excluded middle does not rescue recoverability. If anything, the fully classical case is the worst one for it. When X is extremally disconnected, the De Morgan gap vanishes (Section 7.1) and slack degenerates to the double-negation gap alone, but on a space where excluded middle holds outright that gap vanishes too, so slack is identically 0 for *every* valuation. The zero slack function is then shared by every probability measure on X, which is the most extreme failure of recovery available.

7.4 The Computability Guard

The theory developed here is non-degenerate only if there are spaces whose slack does not vanish identically. Computability supplies them:

Definition 42 (Sierpiński topology, semidecidable proposition). The *Sierpiński space* $\mathbb{S}$ is the type `Prop` of propositions carrying the topology with exactly three opens: $\emptyset$, the set $\mathit{halts} = \{q \mid q\}$ of true propositions, and $\mathbb{S}$ itself [Escardó 2004; Rosolini 1986]. A proposition is *semidecidable* if it is an open in this topology.

The topology encodes verifiability. A point of $\mathbb{S}$ is a proposition, say "this computation halts." The one nontrivial observation available is membership in *halts*, established by producing a proof (running the computation to termination). No open captures the complementary observation, and on the frame $O(\mathbb{S}) = \{\bot, \mathit{halts}, \top\}$ that Σ_1 asymmetry is a theorem:

Theorem 43 (Σ_1 asymmetry[Lean]). $\neg\mathit{halts} = \bot$.

Every nonempty open of $\mathbb{S}$ contains True, so $\bot$ is the only open disjoint from *halts*. Thus, no nonempty open of $\mathbb{S}$ is evidence that the computation does not halt.

We now instantiate the measure-to-valuation construction of Theorem 32 at $X = \mathbb{S}$, allowing us to compute the slack:

Example 44 (halting slack[Lean]). Let $\mu = p\cdot\delta_{\mathsf{True}} + (1-p)\cdot\delta_{\mathsf{False}}$, and let v be the resulting valuation. Then

$$\mathrm{slack}(\mathit{halts}) \;=\; 1 - p \;=\; \mu\big(\partial\{q \mid q\}\big) \;=\; \mu\{\mathsf{False}\}.$$

Since *halts* is already open, $v(\mathit{halts}) = \mu(\mathit{halts}) = p$. The DS "ignorance" of Section 5 is here the mass of the single point False, the one outcome no finite observation can ever resolve. This probability is a fact about the model, not about anyone's ignorance.

***Chaitin's* Ω.** The same bridge scales to algorithmic information theory. Fix a universal prefix-free machine U, *prefix-free* in that no halting program of U is a proper prefix of another, and *universal* in that it simulates every other prefix-free machine. Give 2^ω, the space of infinite bit strings, the fair-coin measure μ. The event H of having a halting program of U as a prefix is open, since halting is established by a finite prefix (Definition 42), and prefix-freeness makes H a disjoint union of cylinders, so $v(H) = \mu(H) = \sum_{p\ \mathrm{halts}} 2^{-|p|}$: exactly Chaitin's halting probability Ω [Chaitin 1975]. Its slack $\mu(\partial H)$ is the mass of strings approximated arbitrarily well by halting strings while never having a halting program as a prefix, and Ω's classical uncomputability is the aggregate form of the pointwise obstruction above. The measure-theoretic core is mechanized on Cantor space with mathlib's infinite product measure: basic cylinders are clopen with measure $2^{-|w|\,\mathrm{Lean}}$, and a *finite* prefix-free set of words has pairwise disjoint cylinders of total measure $\sum_w 2^{-|w|\,\mathrm{Lean}}$, the Kraft-inequality content underlying Ω. The universal machine itself, and the countable extension of the finite Kraft sum, are computability theory beyond our scope.

Definition 45 (halting problem). For a code c and input n, the *halting problem* is the predicate $H(c, n)$ = "the machine with code c halts on input n," (`ComputablePred`).

Slack cannot be eliminated by simply declaring a classical answer for each code. The family of valuations defined below attempts to do this:

Definition 46 (sharp classical readout[Lean])**.** For a given n, the *sharp classical readout* is the family of $\{0,1\}$-valued valuations $\rho_n(c) \in \mathrm{Valuation}(\mathcal{O}(\mathbb{S}))$, with $\rho_n(c)(\mathit{halts}) = 1$ if $H(c,n)$ holds, and $\rho_n(c)(\mathit{halts}) = 0$ otherwise.

The readout is sharp but not slack-free. Since $\neg\mathit{halts} = \bot$ (Theorem 43), a code that does not halt receives slack $1-0=1$, the maximum. Sharp here means that the classical answer is known, not that the complement rule holds on $\mathcal{O}(\mathbb{S})$. However, this family is provably not computable:

Theorem 47 (the sharp readout is uncomputable[Lean])**.** *For every n, the predicate $c \mapsto \big(\rho_n(c)(\mathit{halts}) = 1\big)$ is not computable.*

The proof identifies the predicate with $H(\cdot, n)$ and applies the undecidability of the halting problem (`ComputablePred.halting_problem`).

No route back to a classical, decidable treatment survives contact with computability. Excluded middle guarantees that $H(c,n)$ is true or false for any fixed c, but that guarantee is free: it names no algorithm and exhibits no witness for which disjunct holds. Demanding an actual witness, uniformly in c, is exactly ρ_n's sharp readout, and Theorem 47 shows no algorithm computes it. Aggregating over the whole family of halting programs, rather than reading off one instance, does not help either: Ω is a perfectly well-defined classical probability, yet Chaitin showed it uncomputable by an independent argument turning on algorithmic randomness [Chaitin 1975]. Neither the pointwise nor the aggregate route gives classical probability a way to compute a solution to the halting problem, and each is blocked by its own uncomputability result.

We can conclude that slack is not a defect of the intuitionistic theory. On $\mathcal{O}(\mathbb{S})$, forcing the complement rule at *halts* leaves the single value $v(\mathit{halts}) = 1$, so every code is assigned certain termination. That assignment is computable but false of every non-halting code. Any assignment that avoids this error, giving $v(\mathit{halts}) < 1$ to each non-halting code, has positive slack at every such code, and the one that is moreover sharp is the uncomputable readout of Theorem 47. For the sum rule, this guard plays the role the explicit Cox-model instance plays for the product rule (Section 6): each axiom set in the paper comes with non-trivial models satisfying it.

8 Product Valuations and Kernel Composition

The structure through which classical probability supports sequential statistical reasoning is the Giry monad:

Definition 48 (Giry monad [Giry 1982])**.** The *Giry monad* G sends a measurable space X to the space $G(X)$ of probability measures on X. Its unit sends a point $x \in X$ to the point mass at x, and its multiplication sends a measure M on $G(X)$ to the measure $A \mapsto \int m(A)\,\mathrm{d}M(m)$.

Probabilistic reasoning then composes in stages, and, when the monad is commutative, independent stages compose in either order. The question is whether the intuitionistic calculus supports the same structure.

A *frame homomorphism* is a map preserving finite meets and arbitrary joins. Pushforward along a frame homomorphism $f : \Omega \to \Omega'$ sends a valuation v on Ω' to $a \mapsto v(f(a))$ on Ω, and this preserves the valuation axioms, making $\mathrm{Val}(-)$ a functor on locales, the localic counterpart of the functor part of the Giry monad [Giry 1982]. A frame homomorphism is the opposite of a locale map, so the functor laws are checked against identity and composition of frame homomorphisms[Lean].

Definition 49 (valued frames[Lean])**.** The *valuation functor* $\mathrm{Val} : \mathrm{Frm}^{\mathrm{op}} \to \mathrm{Set}$ sends a frame to its set of valuations and a frame homomorphism $\varphi : P \to Q$ to $w \mapsto w \circ \varphi$. The category SpVal of *valued frames* is its category of elements: objects are pairs (P, v) with v a valuation on P, and a morphism $(P, v) \to (Q, w)$ is a frame homomorphism $\varphi : P \to Q$ with $w \circ \varphi = v$.

A morphism of valued frames is a measure-preserving map of locales in the opposite direction. The point valuations δ_p (Section 4) provide the candidate (monad) unit. Theorem 17 shows that, on finite frames, Val agrees (on objects) with the classical finite-distribution monad on points. Flattening a valuation of valuations is integration, and integration in two variables requires product valuations. A *kernel* is a map assigning a valuation to each point of a locale, giving a conditional distribution. By Kock's identification of commutative monads with Fubini theorems [Kock 2012], kernels compose independently of order exactly when Val is a commutative monad.

A valuation u on a product frame is a *product* of valuations v and w when $u(A \times B) = v(A) \cdot w(B)$ for all rectangles $A \times B$. Let $\mathrm{Low}(\mathbb{N} \times \mathbb{N})$ be the frame of lower sets of the grid $\mathbb{N} \times \mathbb{N}$, ordered component-wise. On this frame, products are unique:

Theorem 50 (product uniqueness[Lean])**.** *Let v, w be valuations on $\mathrm{Low}(\mathbb{N})$. Any two valuations on $\mathrm{Low}(\mathbb{N} \times \mathbb{N})$ that are products of v and w are equal.*

By the inclusion–exclusion equality, two valuations agreeing on a $\sqcap$-closed family agree on all finite joins of its members[Lean]. The rectangles are closed under $\sqcap$, and in $\mathrm{Low}(\mathbb{N} \times \mathbb{N})$ every lower set is a *finite* union of rectangles[Lean], so agreement on rectangles is agreement everywhere. Valuations carry no continuity axiom (Section 4), so the σ-additivity driving the classical uniqueness argument for product measures has no analogue here. The order structure of the frame does that work instead.

Theorem 51 (product non-uniqueness[Lean]). *On the powerset frame* $\mathrm{Set}(\mathbb{N}\times\mathbb{N})$ *there exist valuations* v, w *and two* distinct *products of* v *and* w.

An *ultrafilter* on $\mathbb{N}$ is a collection of subsets, closed under intersection and superset, containing exactly one of S and its complement for every $S \subseteq \mathbb{N}$. It is *free* when it contains no finite set. The map assigning 1 to the members of an ultrafilter and 0 to every other subset is a valuation[Lean]. A rectangle $S \times T$ belongs to an ultrafilter on the product exactly when S and T belong to its images under the two coordinate projections[Lean]. A free ultrafilter $\mathcal{U}$ on $\mathbb{N}$ yields two ultrafilters on $\mathbb{N}\times\mathbb{N}$. A set belongs to the former when, for $\mathcal{U}$-most first coordinates, its slice contains $\mathcal{U}$-most second coordinates. A set belongs to the latter under the same test with the coordinates in the opposite order.

The two ultrafilters' valuations agree on *every* rectangle yet differ on the upper triangle $\{(m,m') \mid m < m'\}$[Lean]. Integrating the same marginal data in the two orders thus disagrees. Fubini fails for valuations, as it does for non-additive integration [Denneberg 1994; Ghirardato 1997]. By Kock's correspondence, no canonical commutative multiplication exists for unrestricted valuations. This half of the dichotomy is not specific to intuitionism, since powerset-frame valuations are exactly the classical finitely additive probability measures [Rao and Rao 1983]. Dropping continuity is what admits the non-uniqueness. For continuous valuations on continuous domains the product is unique and Fubini's theorem holds [Jones 1990], and they form the probabilistic powerdomain [Jones and Plotkin 1989]. A monad therefore requires restricting either the frames or the valuations.

A first step in the second direction is to restrict the mixing measures to countable ones. The components of a mixture are arbitrary valuations, so this restricts how valuations are combined, not which valuations occur. With countable weights the order of mixing does not matter, by the interchange of double series. In Theorem 51 the mixing measures are ultrafilter valuations, which are not countable mixtures:

Proposition 52 (countable kernel composition[Lean]).
1. *A countable mixture of valuations is a valuation*[Lean].
2. *A mixture of mixtures flattens to a mixture with product weights, which is discrete Chapman–Kolmogorov*[Lean].
3. *The one-point mixture is the identity*[Lean].
4. *Iterated countable mixing is order-independent*[Lean].

Restricted to countable-mixture presentations, the data of a monad are all in place: the category of locales, Val restricted to countable mixtures as the functor, the point valuations as the unit, and countable mixing as the multiplication. Proposition 52 verifies exactly the laws this needs: item 1 keeps the multiplication inside the fragment, item 2 is its associativity, item 3 its unit law, and item 4 its commutativity. What is not yet settled is whether this descends to valuations themselves, since a valuation can admit more than one countable-mixture presentation, and the diffuse part of Section 4 is not captured by any countable presentation at all.

9 Valuations Beyond Frames

Definition 1 fixes the event algebra to be a frame: a complete lattice in which finite meets distribute over arbitrary joins $\bigsqcup_{i\in I} a_i$, the operation Heyting implication $\Rightarrow$ (the right adjoint to $a \sqcap (-)$, so that $a \Rightarrow b$ is the largest c with $a \sqcap c \le b$) is built from. Definition 2's four axioms, normalization at $\bot$ and $\top$, monotonicity, and modularity $v(a)+v(b) = v(a\sqcup b)+v(a\sqcap b)$, use only the *binary* join $\sqcup$. None of the four mentions the *arbitrary* join $\bigsqcup_{i\in I}$. Completeness is used only where later sections evaluate a valuation on an infinite join, as in the Scott-continuity results of Section 4, or build $\Rightarrow$ and $\neg$ from it, as in Section 3's slack. Section 16 leaves open how much of the construction is tied to frames specifically, and whether some ambient category sends an event algebra to a probability calculus in general. This section settles both questions for one fragment of the construction, the sharp/point correspondence of Section 3: it isolates the exact algebraic hypothesis that correspondence needs and shows the hypothesis cannot be dropped. It does not recover $\Rightarrow$ or $\neg$ without completeness: the adjoint defining $\Rightarrow$ is built from an arbitrary join of witnesses, so it is unavailable once completeness goes, even if only finite joins are missing. The valuations studied below are negation-free by construction, not frame valuations with completeness quietly removed.

The functor without completeness. A *bounded lattice* is a lattice with a least element $\bot$ and a greatest element $\top$. A *bounded lattice homomorphism* preserves $\bot$, $\top$, $\sqcup$, and $\sqcap$. Definition 2's axioms make sense over any bounded lattice Ω, complete or not:

Definition 53 (lattice valuation[Lean]). A *lattice valuation* on a bounded lattice Ω is a monotone function $v : \Omega \to [0,\infty]$ with $v(\bot) = 0$, $v(\top) = 1$, and, for all $a, b \in \Omega$, $v(a)+v(b) = v(a\sqcup b)+v(a\sqcap b)$.

```
structure LValuation (Ω : Type*) [Lattice
    Ω] [BoundedOrder Ω] where
  toFun : Ω → ℝ≥0∞
  map_bot' : toFun ⊥ = 0
  map_top' : toFun ⊤ = 1
  mono'    : Monotone toFun
  modular' : ∀ a b, toFun a + toFun b
             = toFun (a ⊔ b) + toFun (a ⊓
                 b)
```

A lattice valuation on a frame is exactly a valuation in the sense of Definition 2. The two notions differ only in how much structure the ambient Ω carries.

Section 8 pulls a valuation back along a frame homomorphism and checks the assignment is functorial (identity, composition). That argument pattern-matches on $\sqcup$, $\sqcap$, $\bot$, $\top$ alone and never touches $\bigsqcup$:

Proposition 54 (pullback without completeness[Lean]). *For bounded lattices Γ, Ω, a bounded lattice homomorphism $f : \Gamma \to \Omega$, and a lattice valuation v on Ω, $v \circ f$ is a lattice valuation on Γ. This assignment sends the identity homomorphism to the identity and composition of homomorphisms to composition of the induced maps.*

$\mathrm{Val}(-)$ is therefore a functor on bounded lattices with bounded lattice homomorphisms, of which the frame-level functor of Section 8 is the restriction to the subcategory of frames and frame homomorphisms. Completeness contributes nothing to functoriality.

The valuation monoid. The modularity axiom can be moved from the valuation into its domain.

Definition 55 (valuation monoid[Lean]). For a lattice P with least element $\bot$, $V(P)$ is the free commutative monoid on P (finite formal sums of elements of P) modulo the congruence (equivalence relation compatible with +) generated by $[a] + [b] \sim [a \sqcup b] + [a \sqcap b]$ for all a, b and $[\bot] \sim 0$, where $[a]$ is the class of the generator a.

Theorem 56 (universal property[Lean]). *For every commutative monoid M, the maps $f : P \to M$ with $f(\bot) = 0$ and $f(a) + f(b) = f(a \sqcup b) + f(a \sqcap b)$ are exactly the composites of $a \mapsto [a]$ with monoid homomorphisms $V(P) \to M$, and the homomorphism is unique. On a bounded lattice, the lattice valuations are therefore the monoid homomorphisms $h : V(P) \to [0, \infty]$ with $h([\top]) = 1$ and $a \mapsto h([a])$ monotone.*

So modularity becomes a property of the domain $V(P)$, and normalization and monotonicity are conditions on the homomorphism.

Theorem 57 (flags[Lean]). *In every lattice with a least element, every element of $V(P)$ equals $[\alpha_1] + \cdots + [\alpha_n]$ for some chain $\alpha_1 \le \cdots \le \alpha_n$.*

The proof inserts one generator at a time into a chain $c_1 \le \cdots \le c_n$: $[x] + [c_1] = [x \sqcap c_1] + [x \sqcup c_1]$, and $x \sqcup c_1$ is inserted into $c_2 \le \cdots \le c_n$. Every common lower bound of x and the chain lies below every element of the result, which keeps the result a chain. Distributivity is not used. For distributive lattices the monoid goes back to Tarski and to Horn and Tarski, who give a normal form of the same kind, a sum of the joins of the k-fold meets of the generators [Horn and Tarski 1948; Tarski 1938]. Coquand and Spitters use it for valuations on distributive lattices [Coquand and Spitters 2009], and Geissinger studies the universal valuation [Geissinger 1973]. On a finite distributive lattice $\mathrm{Low}(Q)$ the chain is unique up to copies of $\bot$, and $V(P)$ is cancellative ($x + y = x' + y$ implies $x = x'$): evaluating against the point valuations δ_q gives the number $k(q)$ of elements of the chain containing q, the j-th largest element of the chain is $\{q \mid k(q) \ge j\}$, and $x \mapsto k$ is an injective homomorphism into $\mathbb{N}^Q$. This argument is not formalized. Without distributivity both can fail.

Proposition 58 (flags at M_3[Lean]). *In $V(M_3)$, for the lattice of Definition 61, $[a] + [o] = [c] + [o]$, and $[a] + [b] = [a] + [c]$ while $[b] \ne [c]$. So the chain of Theorem 57 is not unique, and $V(M_3)$ is not cancellative.*

Both equations follow from $[x] + [y] = [o]$ for distinct atoms x, y. The map sending z and c to 0 and a, b, o to ∞ satisfies the hypotheses of Theorem 56 and separates $[b]$ from $[c]$.

Separation needs only distributivity. Theorem 13 characterizes what a sharp valuation *is*, a complement-indicator of a prime ideal, but not whether there are enough of them to distinguish elements of Ω. That is a separate fact:

Theorem 59 (sharp valuations separate points[Lean]). *For every frame Ω and $a, b \in \Omega$ with $a \not\le b$, there is a sharp valuation v with $v(a) = 1$ and $v(b) = 0$.*

The proof runs the same prime-ideal separation theorem that built the counterexample of Theorem 11 (`DistribLattice.prime_ideal_of_disjoint_filter_ideal`), on the principal filter of a (the elements above a) and the principal ideal of b (the elements below b) rather than on $a \sqcup \neg a$ and $\top$, so it is classical, as in Table 1. As a corollary, any two distinct $a, b \in \Omega$ are distinguished by some sharp valuation, by a classical step: Ω need not be linear, so $a \ne b$ alone does not hand back $a \le b$ or $b \le a$, but it forces at least one failure of $\le$, and Theorem 59 applies in whichever direction fails. This is the cash-out of the first qualifying condition of Section 16: the certain valuations do not merely characterize a single point of the locale in isolation, together they reconstruct the order on Ω. Reconstructing the order is not the same as reconstructing the full Heyting structure: $\neg$ and $\Rightarrow$, as just noted, are adjoints built from an arbitrary join and carry no meaning over a plain distributive lattice. What this theorem recovers is the order-theoretic core of the first qualifying condition, not the negation fragment of intuitionistic logic, which stays frame-specific.

Tracing the proof of Theorem 59 shows it never invokes $\bigsqcup$. The only place frame structure enters is `DistribLattice.prime_ideal_of_disjoint_filter_ideal` itself, which needs distributivity and nothing beyond it:

Theorem 60 (separation over distributive lattices[Lean]). *For every distributive bounded lattice Ω and $a, b \in \Omega$ with $a \not\le b$, there is a sharp lattice valuation v on Ω with $v(a) = 1$ and $v(b) = 0$.*

Theorem 60 reproves Theorem 59 with the hypothesis `Order.Frame` weakened to `DistribLattice`, and nothing else about the argument changes. The condition doing the

work in the sharp/point correspondence is distributivity, and completeness is not load-bearing for either half of it.

Distributivity is not a proof artifact. The natural question is whether Theorem 60's hypothesis is a genuine dividing line or an artifact of routing the argument through a mathlib lemma named for distributive lattices. It is genuine, and the failure it guards against is total, not merely a failure of separation:

Definition 61 (the diamond[Lean])**.** M_3 is the five-element lattice with a bottom z, a top o, and three pairwise-incomparable atoms (elements directly above the bottom) a, b, c, with $x \sqcup y = o$ and $x \sqcap y = z$ for every two distinct atoms x, y.

M_3 is the smallest lattice that is not distributive: $a \sqcap (b \sqcup c) = a \sqcap o = a$, while $(a \sqcap b) \sqcup (a \sqcap c) = z \sqcup z = z$. It is modular ($x \le z$ implies $x \sqcup (y \sqcap z) = (x \sqcup y) \sqcap z$), so this failure is specific to distributivity rather than some coarser defect of the order.

Theorem 62 (no sharp valuation on the diamond[Lean])**.** *No lattice valuation on M_3 is sharp.*

Modularity applied to each of the three pairs of distinct atoms forces $v(x) + v(y) = 1$, since $x \sqcup y = \top$ and $x \sqcap y = \bot$. If v were sharp, $v(a), v(b), v(c) \in \{0, 1\}$ would have to satisfy all three pairwise sum-to-one equations at once, and a two-element codomain cannot support that: fixing any one value propagates through the other two equations to a contradiction with the third. The collapse on M_3 is therefore not a failure of separation, points that happen to coincide, but a failure of existence: there is no all-or-nothing valuation on M_3 to separate anything with, and the first qualifying condition of Section 16 is unsatisfiable outright.

Where this leaves the boundary. Frames are complete distributive lattices. Theorem 60 shows completeness is not needed for the sharp/point correspondence's separation half, and Theorem 62 shows distributivity cannot be given up in exchange. In a classical metatheory, the event algebras on which sharp valuations reconstruct the order, the order-theoretic core of criterion (1) of Section 1, are exactly the distributive bounded lattices, no more and no less. Criterion (2) and the negation-dependent machinery of Sections 3–7, slack, the complement bound, the De Morgan gap, are untouched by this section and remain tied to frames: nothing here shows, or claims, that $\neg$ or $\Rightarrow$ survives dropping completeness. This bears on the orthomodular event lattices of quantum probability discussed in Section 15: giving up distributivity, as those lattices do, does not merely change which valuations exist, it can remove sharp valuations altogether, so the global construction has nothing to say about that setting. Section 12.1 replaces global certain states by certainty within each context.

A second boundary: grading a non-idempotent tensor. The event algebras of linear logic are quantales (complete lattices with an associative multiplication $\otimes$ that distributes over arbitrary joins), and Section 15 notes that their tensor $\otimes$ is not the lattice meet $\sqcap$ this section's valuations are built from, so Definition 53's modularity axiom does not even parse for $\otimes$ without first deciding what to write in $\sqcap$'s place. Granting the substitution provisionally, to see what it costs, shows the obstruction runs deeper than a notational mismatch:

Definition 63 (tensor valuation[Lean])**.** Fix a join-semilattice Ω (a partial order with binary joins) and a binary operation $\otimes$ on Ω. A *tensor valuation* for $\otimes$ is a function $v : \Omega \to [0, \infty]$ such that, for all $a, b \in \Omega$, $v(a) + v(b) = v(a \sqcup b) + v(a \otimes b)$.

Theorem 64 (the transplant collapses[Lean])**.** *For every join-semilattice Ω, binary operation $\otimes$ on Ω, and tensor valuation v for $\otimes$, if $v(a) < \infty$ then $v(a \otimes a) = v(a)$.*

Setting $b := a$ in the defining identity and using only $a \sqcup a = a$, join-idempotence, already gives $v(a) + v(a) = v(a) + v(a \otimes a)$, hence the claim by cancellation. No property of $\otimes$ is used: not associativity, not a unit, not distributivity over $\sqcup$. This is a sharper failure than Theorem 62's: M_3 shows meet-based modularity can fail to admit a sharp valuation on one uncooperative lattice, while Theorem 64 shows the $\otimes$-based transplant admits no valuation separating a from $a \otimes a$ on *any* Ω and for *any* $\otimes$, idempotent or not. A non-idempotent $\otimes$, linear logic's whole point, is exactly what such a valuation could never distinguish from $\sqcap$'s behavior.

Example 65 (a concrete witness[Lean])**.** Take $\Omega = \mathbb{N}$ with $\sqcup = \max$ and $\otimes = +$: addition is associative and commutative but not idempotent ($1 + 1 = 2 \neq 1$), so the hypothesis of Theorem 64 is not vacuous. Substituting $a = 0$, where the join's unit and the tensor's unit coincide, forces $v(n) = v(0)$ for every n: every tensor valuation for $+$ is constant.

This is not a toy chosen merely by analogy with linear logic. Every quantale, the event algebra Section 15 attaches to linear logic, is by definition a complete lattice equipped with an associative $\otimes$ that need not be idempotent, exactly the data $(\Omega, \sqcup, \otimes)$ Theorem 64 quantifies over, with the quantale's own join playing the role of $\sqcup$. Example 65 is a quantale itself ($+$ distributes over max), kept only because it is small enough to check by hand: it fails idempotence for the same reason $\otimes$ does, no algebraic law identifies two copies of a resource with one.

Corollary 66 (grading and discrimination are exclusive[Lean])**.** *For every join-semilattice Ω, binary operation $\otimes$ on Ω, and $a \in \Omega$ with $v(a) < \infty$: no v is both a tensor valuation for $\otimes$ and satisfies $v(a \otimes a) \neq v(a)$.*

This restates Theorem 64 as the dichotomy it really is. A candidate probabilistic semantics for a non-idempotent $\otimes$ must choose between being graded by Definition 53's

compositional law and discriminating the one thing a non-idempotent $\otimes$ exists to discriminate, a from $a \otimes a$. It cannot keep both. The probabilistic semantics that does exist for linear logic, probabilistic coherence spaces [Danos and Ehrhard 2011], keeps discrimination: two independent copies of a resource are not scored as interchangeable with one. Corollary 66 shows the price of that choice is forced, not chosen: probabilistic coherence spaces cannot also be graded by this paper's modularity law, on pain of losing exactly the discrimination they exist to keep, so they instead evaluate proofs with a classical, Kolmogorov-style structure over $\otimes$'s type-level behavior, never a valuation of propositions in Definition 53's sense. Nothing here is a missed opportunity on either side. The dichotomy forecloses the combination before any further axiom of Definition 53 is even reached. Section 10 replaces these axioms by a construction that applies to linear logic.

The lattice of logics. Ordered by inclusion of their theorems, the logics of this paper lie below classical logic. Classical logic is the unique maximal consistent intermediate logic (a logic between intuitionistic and classical propositional logic), and the unique maximal consistent extension of quantum logic, because $\{\bot, \top\}$ is a subalgebra of every nontrivial Heyting algebra and of every nontrivial orthomodular lattice (a lattice with an orthocomplement $a \mapsto a^\perp$ in which $a \le b$ implies $b = a \sqcup (a^\perp \sqcap b)$), so the Boolean algebras form the least nontrivial variety (a class of algebras defined by equations) in both cases. Among substructural logics (logics without some of the structural rules, such as contraction and weakening) it is one of continuum many maximal consistent logics [Galatos 2004]. The three logics leave classical logic in different directions, and the criteria fail in correspondingly different ways.

- Intuitionistic logic drops excluded middle and keeps distributivity. Certain states exist and reconstruct the order (Theorem 59), and the complement rule fails (Theorem 11), so slack appears. Its two parts detect the steps from intuitionistic logic to the logic of weak excluded middle and from there to classical logic (Theorem 6).
- Quantum logic drops distributivity and keeps $a \sqcup a^\perp = \top$. The complement rule holds, and certain states can vanish (Theorems 62 and 95), so criterion (1) fails for a global calculus.
- Linear logic drops the idempotence of $\otimes$. Grading its event lattice identifies $a \otimes a$ with a (Theorem 64), so the calculus lives on the webs of formulas (the sets of points on which their probabilistic coherence spaces are defined, Section 10.3), where criterion (3) first fails at !Bool (reusable Booleans, Theorem 78).

Classical probability is the one case where certain states reconstruct the order, the complement rule holds, and the grading respects the tensor.

10 Probability Calculi from Certain States

Corollary 66 shows that linear logic admits no grading by Definition 53's axioms that distinguishes a from $a \otimes a$. The obstruction is in the modularity axiom, which uses the lattice operations of the event algebra. This section uses instead a construction that needs no lattice operations: the calculus of a logic is the set of mixtures of its certain states, its $\{0,1\}$-valued assignments. For classical logic this is de Finetti's characterization of coherent probability assignments [de Finetti 1974], and Paris [Paris 2001] and Williams [Williams 2012] use it for non-classical logics, where it is known as generalized probabilism. On finite frames, and in particular on finite Boolean algebras, it returns exactly the valuations of Definition 2, an instance of Paris's theorem for logics whose certain states respect $\wedge$ and $\vee$ [Paris 2001]. Linear logic is not such a logic. For every formula of multiplicative–additive linear logic over finite data types (Definition 79), its output is contained in the probabilistic coherence space (Definition 76) of the formula, and the certain elements of that space are cliques (sets of pairwise coherent points) of Girard's coherence space (Section 10.3). At implications between data types it returns the probabilistic coherence space. At the exponential !Bool (the type of Booleans that may be used any number of times) it returns a strict subset.

10.1 The Construction

Definition 67 (presentation by certain states, mixtures^{Lean})**.** A *presentation* is a set E of events together with a set S of $\{0,1\}$-valued functions $E \to [0,\infty]$, the *certain states*. Its *calculus* $\mathrm{Mix}(S)$ is the set of finite convex combinations $\sum_i w_i\, s_i$ with $s_i \in S$, $w_i \in [0,\infty]$, and $\sum_i w_i = 1$.

For classical logic the certain states are the two-valued models, for intuitionistic logic on a frame they are the sharp valuations (Theorem 13), and for a type of linear logic they are the deterministic programs of that type (Section 10.3). Two properties hold for every presentation.

Theorem 68 (certain elements and mixing$^{\text{Lean Lean}}$)**.** *For every presentation (E, S): (i) a $\{0,1\}$-valued x lies in $\mathrm{Mix}(S)$ iff $x \in S$, and (ii) $\mathrm{Mix}(\mathrm{Mix}(S)) = \mathrm{Mix}(S)$.*

Part (i) is the first condition of Section 1, that the $\{0,1\}$-valued elements of the calculus reconstruct the logic. In a presentation the logic is given by S, so the condition says that mixing creates no new $\{0,1\}$-valued elements. The proof: if $x = \sum_i w_i s_i$ is $\{0,1\}$-valued and $w_j \neq 0$, then at each e with $x_e = 0$ every s_i with $w_i \neq 0$ has $s_i(e) = 0$, and at each e with $x_e = 1$ every such s_i has $s_i(e) = 1$, since $\sum_i w_i = 1$ and $s_i(e) \le 1$. So $s_j = x$, and $x \in S$. Part (ii) states that a convex combination of elements of $\mathrm{Mix}(S)$ is again in $\mathrm{Mix}(S)$. For valuations on a frame this is the closure of valuations under mixing of Section 4^{Lean}. The second condition of Section 1, that the classical case returns the classical theory, is Theorem 71 below.

The three axioms of Definition 2 are instances of one form:

Definition 69 (affine law[Lean]). For finite E, an *affine law* is given by $a, b : E \to [0,\infty]$ and $k, k' \in [0,\infty]$. A function $x : E \to [0,\infty]$ *satisfies* it if $\sum_e a_e x_e + k \le \sum_e b_e x_e + k'$.

Monotonicity $x_a \le x_b$, normalization ($x_\top \le 1$ and $1 \le x_\top$, $x_\bot \le 0$), and modularity (two opposite inequalities) are affine laws.

Theorem 70 (completeness[Lean]). *For finite E and every set S of $\{0,1\}$-valued functions on E, $\mathrm{Mix}(S)$ is the set of functions satisfying every affine law that every $s \in S$ satisfies.*

The inclusion from left to right holds because affine laws are preserved by convex combinations[Lean]. The converse separates a point outside the convex hull of S by a hyperplane, using mathlib's geometric Hahn–Banach theorem, and reads the hyperplane off as an affine law. For rational points and integer certain states, Section 3 gives a proof without choice by Fourier–Motzkin elimination (elimination of variables from a system of linear inequalities)[Lean]. Theorem 70 is the generalization of the three axioms: the axioms of the calculus of a presentation are the affine laws valid on its certain states, and nothing else. On frames these are the three axioms of Definition 2:

Theorem 71 (frames[Lean]). *For a finite poset P, the valuations on the frame $\mathrm{Low}(P)$ of lower sets of P are exactly Mix of its sharp valuations.*

The proof combines Theorem 18 with the fact that a sharp valuation on $\mathrm{Low}(P)$ is a point valuation δ_p[Lean]. Every finite frame is of the form $\mathrm{Low}(P)$ (Section 4), and finite Boolean algebras are the case of a discrete P (no two distinct elements comparable), where the valuations are the probability mass functions on P. So the construction reduces to the valuation calculus on finite intuitionistic event algebras and to Kolmogorov's calculus on finite classical ones.

For infinite E the laws are taken with finite support. A *window* is a finite set $F \subseteq E$ of events, and a *finite law* is an affine law on some window, applied to the restrictions of functions to F.

Theorem 72 (barycenters[Lean Lean]). *Let S be a set of $\{0,1\}$-valued functions on an arbitrary set E. For $x : E \to [0,\infty]$ the following are equivalent: (i) x satisfies every finite law valid on S. (ii) For every window F, the restriction of x to F lies in Mix of the restrictions of S to F. (iii) There is a Borel probability measure μ on $\{0,1\}^E$, with the product topology (the topology of convergence at each event), such that $x_e = \mu\{\sigma \mid \sigma_e = 1\}$ for every $e \in E$, and $\mu(\mathrm{bad}_F) = 0$ for every window F, where bad_F is the set of σ whose restriction to F is the restriction of no $s \in S$.*

The equivalence of (i) and (ii), which Paris states for infinite languages [Paris 2001], applies Theorem 70 to each window. For (ii) ⇒ (iii), the mixture on a window F gives a probability measure μ_F on $\{0,1\}^E$, a finite convex combination of point masses at certain states. The windows are directed by inclusion. The probability measures on the compact space $\{0,1\}^E$ form a compact space in the weak topology (the coarsest topology in which $\nu \mapsto \int f\,d\nu$ is continuous for every bounded continuous f)[2], so the family $(\mu_F)_F$ has a cluster point μ. A *cylinder set* is a set of σ determined by the values of σ on a window. Cylinder sets are clopen (closed and open), so $\nu \mapsto \nu(C)$ is continuous in the weak topology for every cylinder set C. The sets $\{\sigma \mid \sigma_e = 1\}$ and bad_F are cylinder sets, and μ_F has the required values on them once $e \in F$, so μ has them. For (iii) ⇒ (i), a finite law valid on S holds at μ-almost every σ, and integrating it against μ gives the law at x.

Relation to generalized probabilism. Theorem 70 is de Finetti's theorem in the form used for non-classical logics [Paris 2001; Williams 2012]. An affine law valid on every certain state and violated at x is a finite system of bets priced by x with a net loss in every certain state, a Dutch book, so the theorem says that x admits no Dutch book iff it is a mixture of certain states. Paris proves it with a separating hyperplane, as the Lean proof does, and Williams derives from the same fact that these are the assignments not dominated in accuracy [Williams 2012]. Paris also shows that when every certain state respects $\wedge$ and $\vee$ the affine laws reduce to normalization, monotonicity, and modularity [Paris 2001], which gives Theorem 71. Theorem 72(iii) is the counterpart, for $\{0,1\}$-valued certain states, of the representation of the states of an MV-algebra (an algebra of Łukasiewicz many-valued logic) as integrals over its valuations [Kroupa 2006; Kühr and Mundici 2007; Panti 2008]. Gil Sanchez, Gyenis, and Wroński axiomatize the convex hull for a class of non-classical evaluations and question the use of Dutch books outside classical logic [Gil Sanchez et al. 2022], Bradley studies these convex hulls for non-classical probability [Bradley 2016], and Gyenis proves that for every logic given by a finite algebra of truth values, each with a numerical value, the convex hull of the evaluations has an effective axiomatization in an equational calculus, finite for finitely many variables and not finite in general [Gyenis 2025]. Theorem 70 is the elementary counterpart without a formula language: it applies to an arbitrary set of $\{0,1\}$-valued certain states on a finite set of events, and its laws are individual affine inequalities, not schemata uniform in the variables. This is what lets it apply to the web of a linear formula, whose certain states are not evaluations into a fixed algebra of truth values. This section adds the mechanization of these results, a proof of Theorem 70 without choice for rational data, and their use for logics that Paris's axiomatization does not cover: for linear logic, where modularity is not available (Corollary 66), in Sections 10.3 and 11, and for quantum event logic in Section 12.

[2] In mathlib this is an instance proved from Prokhorov's theorem.

10.2 Uniqueness

$\mathrm{Mix}(S)$ is the least convex set containing S. Criteria (1) and (2) of Section 1 do not single it out: the probabilistic coherence space !Bool satisfies (1) and strictly contains the calculus (Theorem 78). Criterion (3) does: every affine law that holds on all certain states holds on the whole calculus. These laws are, relative to the logic, Boole's conditions of possible experience [Pitowsky 1994].

Theorem 73 (uniqueness^{Lean})**.** *For finite E and a set S of $\{0,1\}$-valued functions on E, let C contain S, be closed under finite mixtures, and satisfy every affine law that every $s \in S$ satisfies. Then $C = \mathrm{Mix}(S)$.*

Closure under mixtures gives $\mathrm{Mix}(S) \subseteq C$, and Theorem 70 gives $C \subseteq \mathrm{Mix}(S)$. The sets that contain S and are closed under mixtures have a least element, $\mathrm{Mix}(S)$. The sets that satisfy the laws of S have a greatest element, the set of solutions of those laws. The theorem holds because the two coincide, which is Theorem 70. The mathematics is de Finetti's and Paris's [de Finetti 1974; Paris 2001]. What the theorem adds is the use of the three conditions as criteria for a calculus.

For infinitely many events one more condition is needed, since $\mathrm{Mix}(S)$ is in general not closed and so is not the solution set of any family of laws. A state is checked against finitely many events at a time. We therefore require the calculus to be closed in the product topology on $[0,\infty]^E$, the topology of convergence at each event: a function that is approximated by states of the calculus on every finite window belongs to the calculus.

Theorem 74 (uniqueness for arbitrary events^{Lean})**.** *For an arbitrary set E and a set S of $\{0,1\}$-valued functions on E, let C contain S, be closed under finite mixtures and in the product topology, and satisfy every finite law that every $s \in S$ satisfies. Then C is the closure of* $\mathrm{Mix}(S)$*, and it is the set of functions satisfying every finite law valid on S.*

A function satisfying every finite law valid on S lies, on each window F, in Mix of the restrictions of S to F (Theorem 72). So every basic neighbourhood of it, which constrains finitely many events, meets $\mathrm{Mix}(S)^{\mathsf{Lean}}$. The rest of the proof is that of Theorem 73.

Corollary 75 (valuations on frames)**.** *In a classical metatheory, for every frame Ω the valuations on Ω are the closure of* Mix *of its sharp valuations. So they are the only set of functions on Ω that contains the sharp valuations, is closed under mixtures and in the product topology, and satisfies every finite law valid on the sharp valuations.*

A finite set of events generates a finite distributive sublattice L of Ω. The restriction of a valuation to L is a valuation on L, hence a mixture of sharp valuations on L (Theorem 71, with $L \cong \mathrm{Low}(P)$ by Birkhoff's theorem, which presents a finite distributive lattice as the lower sets of its join-irreducible elements). Each sharp valuation on L is the restriction of a sharp valuation on Ω, by the prime ideal theorem. So every valuation satisfies the condition of Theorem 72(ii), and the valuations satisfy every finite law valid on the sharp valuations. Conversely, normalization, monotonicity, and modularity are such laws. Theorem 74 then applies. This corollary is not formalized.

Why the third criterion. The third criterion has four readings, and on finite E they agree.

- *Coherence.* A law valid on S and violated at x is a Dutch book against x, so the criterion excludes every state against which the logic guarantees a loss [Paris 2001].
- *Accuracy.* Williams shows that, for the Brier score (the sum over events of the squared distance to the truth value), the states outside $\mathrm{Mix}(S)$ are exactly those dominated in accuracy by a state inside it [Williams 2012]. The criterion excludes dominated states.
- *Ignorance.* For finite E, a compact convex set of real-valued functions on E is the convex hull of its extreme points (the points that are not mixtures of other points of the set), by Minkowski's theorem. So a compact convex set containing S satisfies the criterion iff its extreme points are certain states. Uncertainty is then uncertainty about which certain state obtains. This equivalence is not formalized.
- *No new behaviours.* Every known failure of the criterion is a state that no mixture of certain states produces: the point of !Bool that violates the law $x_{(1,1)} \leq 0$, valid on its certain states, the points of the probabilistic coherence space at second order outside the calculus (Section 11), and the quantum states on Cabello's set, which violate the bound of Theorem 97.

The criterion is therefore a choice and not a consequence of (1) and (2). A calculus that drops it is either a semantics of programs with more behaviours than its certain states, as for probabilistic coherence spaces, or a contextual theory (one whose states are certain only within each context, Section 12.1), as for quantum states. How the criteria should treat logics without enough certain states is the subject of Section 12.1.

Each condition of Theorem 73 is needed. Without (3), !Bool is a counterexample. Without closure under mixtures, S itself is one whenever S has two elements. Without $S \subseteq C$, every proper convex subset of $\mathrm{Mix}(S)$ is one. Criterion (2) is a consequence of the others: at a finite Boolean algebra the certain states are the point valuations of its atoms, and their mixtures are the probability mass functions on the atoms (Theorem 71). On infinite Boolean algebras they give the finitely additive probabilities (Corollary 75), so countable additivity would be an additional requirement. Under (3) the criterion is redundant, but it still constrains calculi that drop (3). The local calculus of Section 12.1 and the probabilistic coherence spaces violate (3) and satisfy (2): with a single

context the local calculus is classical probability on the outcomes, and at first order the probabilistic coherence space is the calculus (Theorem 77).

Not conflating events. Criteria (1) and (3) are the two halves of one condition. A set that contains every certain state satisfies no affine law that some certain state violates. So under (1) the calculus imposes no law that the logic does not have. Criterion (3) is the converse. Together they say that the calculus satisfies exactly the affine laws of the certain states[Lean]. On a frame this excludes two kinds of conflation, in a classical metatheory.

- *Of events.* If $a \neq b$, a sharp valuation separates them (Theorem 59), so some state of the calculus gives a and b different values.
- *Of negation with complement.* If every sharp valuation satisfies $v(a) + v(\neg a) = 1$, then $a \sqcup \neg a = \top$[Lean]. So a calculus that contains the certain states obeys the complement rule at a only where excluded middle holds at a.

The classical alternative has the same numbers and different events. A valuation on a frame extends to a finitely additive probability on the Boolean algebra the frame generates [Pettis 1951], and reading $\neg a$ in that algebra as the Boolean complement of a assigns it $1 - v(a)$. This grading satisfies the complement rule and gives the event "a is not verified" the credence of the event "a is refuted". The second item says that no calculus satisfying (1) on the frame does this. The difference is the slack, which is the boundary mass in the measure model (Theorem 33). For linear logic the analogous conflation is of a with $a \otimes a$, which every grading of the event lattice by Definition 53 makes (Corollary 66) and the calculus of Section 10.3 does not.

10.3 Linear Logic

Definition 76 (probabilistic coherence space [Danos and Ehrhard 2011][Lean])**.** For a set W and a set Q of functions $W \to [0,\infty]$, let the *orthogonal* of Q be $Q^\perp = \{y : W \to [0,\infty] \mid \sum_w x_w y_w \leq 1 \text{ for all } x \in Q\}$. A *probabilistic coherence space* (PCS) with *web* W is a set P of functions $W \to [0,\infty]$ such that $P = P^{\perp\perp}$, every $w \in W$ has some $x \in P$ with $x_w \neq 0$ (totality), and $\sup_{x \in P} x_w < \infty$ for every $w \in W$ (boundedness).

On a finite web, $P = P^{\perp\perp}$ holds iff P contains 0 and is closed, convex, and downward closed, by Girard's bipolar theorem [Girard 2004]. The map $Q \mapsto Q^\perp$ is the antiblocking duality of polyhedral combinatorics [Fulkerson 1971], which Section 11.4 uses.

We use finite webs. For a finite set X, the *data type* X is the PCS of $x : X \to [0,\infty]$ with $\sum_a x_a \leq 1$. The PCS $X \multimap Y$ on $X \times Y$ is the set of t with $\sum_b \sum_a x_a\, t(a,b) \leq 1$ for every x in X, equivalently $\sum_b t(a,b) \leq 1$ for every a[Lean]. Both satisfy the three conditions[Lean]. A *coherence space* [Girard 1987] is a set with a reflexive symmetric relation, its coherence, and a *clique* is a set of pairwise coherent elements. In the coherence space $X \multimap Y$ of two data types, two points (a,b) and (a',b') cohere iff they are equal or $a \neq a'$.

Probabilistic coherence spaces interpret every formula of linear logic, including the exponentials (the modalities ! and ?, which let an argument be used any number of times), and model probabilistic PCF (a typed functional language with recursion and probabilistic choice) [Danos and Ehrhard 2011]. The construction of Definition 67 applies to every such type, with the $\{0,1\}$-valued elements of its probabilistic coherence space as certain states. At first order, the implications between data types, the two agree (Theorem 77), and the exponential !Bool separates them (Theorem 78). Section 10.4 proves the facts that hold at every multiplicative–additive formula, and Section 11.2 shows that the two differ at every formula with a negative occurrence (an occurrence under an odd number of negations) of a second-order core (the formula $(\mathbf{n} \multimap \mathbf{m}) \otimes (\mathbf{n}' \multimap \mathbf{m}')$ of Section 11.1). Whether they agree at the remaining types is open.

Theorem 77 (first order[Lean Lean])**.** *$X \multimap Y = \mathrm{Mix}\{\det f \mid f : X \to Y_\perp\}$, where $\det f(a,b) = 1$ if $f(a) = b$ and 0 otherwise, and $Y_\perp$ adds a divergent value. A $\{0,1\}$-valued t lies in $X \multimap Y$ iff its support is a clique of the coherence space $X \multimap Y$.*

At first order the certain states are the deterministic partial programs, they are the cliques of the coherence-space model, and the calculus is the PCS.

The PCS !Bool has web the finite multisets over $\{t,f\}$, and is the biorthogonal closure $(\cdot)^{\perp\perp}$ of the promotions $x^!(i,j) = x_t^i x_f^j$ of the elements x of Bool, where (i,j) is the multiset with i copies of t and j copies of f.

Theorem 78 (exponential[Lean Lean])**.** *Every $\{0,1\}$-valued element of* !Bool *vanishes at every (i,j) with $i,j \geq 1$. The promotion of $x = (\frac{1}{2},\frac{1}{2})$ takes the value $\frac{1}{4}$ at $(1,1)$, so it lies in* !Bool *and not in* Mix *of the $\{0,1\}$-valued elements.*

Danos and Ehrhard compute the largest value of an element of !Bool at (i,j), which is $i^i j^j/(i+j)^{i+j}$, below 1 when $i,j \geq 1$, and note that these multisets are not points of the web of !Bool in Girard's coherence spaces [Danos and Ehrhard 2011]. The first statement of Theorem 78 follows, and the second is a direct consequence of the first. The Lean proof of the first uses the dual element $4\,\delta_{(i,j)}$, which is admissible because $x_t + x_f \leq 1$ implies $4x_t x_f \leq 1$.

10.4 All Multiplicative–Additive Formulas

Definition 79 (formulas and their semantics[Lean])**.** The formulas are generated from the data types $\mathbf{n}$, for $n \in \mathbb{N}$, by linear negation $A^\perp$, $A \otimes B$, and $A \,\&\, B$. Write $A \parr B = (A^\perp \otimes B^\perp)^\perp$, $A \oplus B = (A^\perp \,\&\, B^\perp)^\perp$, and $A \multimap B = (A \otimes B^\perp)^\perp$. The web $|A|$ is $\{1,\dots,n\}$ for $\mathbf{n}$, $|A|$ for $A^\perp$, $|A| \times |B|$ for $A \otimes B$, and $|A| \sqcup |B|$ for $A \,\&\, B$. The PCS $P(A)$ is the data type for $\mathbf{n}$, $P(A)^\perp$ for $A^\perp$, $\{x \otimes y \mid x \in P(A),\ y \in P(B)\}^{\perp\perp}$ for $A \otimes B$,

where $(x \otimes y)(a,b) = x_a y_b$, and the set of z whose restrictions to $|A|$ and $|B|$ lie in $P(A)$ and $P(B)$ for $A \,\&\, B$ [Danos and Ehrhard 2011]. The coherence $\asymp_A$ is equality on $\mathbf{n}$, and $s \asymp_{A^\perp} t$ iff $s = t$ or not $s \asymp_A t$. On $A \otimes B$ it is componentwise. On $A \,\&\, B$ it is $\asymp_A$ and $\asymp_B$ on the two components, and any two points from different components cohere [Girard 1987]. The *coherence graph* G_A of A has the web $|A|$ as vertices and an edge between any two distinct coherent points.

Theorem 80 (every formula$^{\text{Lean Lean Lean Lean}}$)**.** *For every formula A: (i) $P(A)$ is a PCS. (ii) The indicator of every point and of every pair of distinct coherent points lies in $P(A)$, and $x_s + x_t \le 1$ for every $x \in P(A)$ and every pair of distinct incoherent points s, t. (iii) Every $\{0,1\}$-valued element of $P(A)$ is the indicator of a clique of the coherence space A. (iv)* Mix *of the $\{0,1\}$-valued elements of $P(A)$ is contained in $P(A)$, and its $\{0,1\}$-valued elements are exactly those of $P(A)$.*

Part (ii) is proved by simultaneous induction on A. Negation exchanges the statement about coherent pairs with the statement about incoherent pairs. For $A \otimes B$, the indicator of a coherent pair is bounded by the tensor of indicators of its projections, and a pair incoherent in $A \otimes B$ is incoherent in one component, where the bound of A or B applies. Part (iii) follows from (ii): two incoherent points in the support of a $\{0,1\}$-valued element would give it total 2 on a pair bounded by 1. Part (iv) is Theorem 68 together with the closure of an orthogonal $Q^\perp$ under mixing^{Lean}. Girard introduces probabilistic coherence spaces with the intuition that two incoherent points of the web have weights summing to at most 1, and then sets coherence aside [Girard 2004]. Part (ii) shows that the intuition holds at every multiplicative–additive formula over data types. The converse of (iii) fails (Section 11.3).

The semantics of Definition 79 computes the spaces of Section 10.3: $P(\mathbf{n} \multimap \mathbf{m})$ is the PCS $X \multimap Y$ of that section for $X = \{1,\dots,n\}$ and $Y = \{1,\dots,m\}$^{Lean}. So Theorem 77 is a statement about these formulas^{Lean}.

The exponentials, whose webs are infinite, are not covered by Definition 79. Theorem 78 treats !Bool separately.

11 Second-Order Linear Types

This section studies the construction of Section 10 at second-order types (types of programs whose arguments are functions), where it returns a strict subset of the probabilistic coherence space. A formula has a *gap* if its calculus is strictly smaller than its probabilistic coherence space. Section 11.1 treats the type T of a program with two functional arguments, Section 11.2 extends the strict inclusion to every formula with a negative occurrence of the core of T, the formula whose negation is T, and Sections 11.3 and 11.4 relate the gap to coherence spaces and to graph polytopes and characterize it at second order. By computation, not formalized, the construction and the probabilistic coherence space agree on (Bool $\multimap$ Bool) $\multimap$ Bool and ((Bool $\multimap$ Bool) $\multimap$ Bool) $\multimap$ Bool, the two iterated implications into Bool we checked.

11.1 Randomized Sequential Programs

At second order, let $T = ((X \multimap Y) \otimes (X' \multimap Y'))^\perp$, the type of a program that receives $f : X \multimap Y$ and $g : X' \multimap Y'$, uses each once, and halts or diverges. Its negation $(X \multimap Y) \otimes (X' \multimap Y')$ is the *core* of T. We take T to be the set of $t : (X \times Y) \times (X' \times Y') \to [0,\infty]$ with $\sum_{p,q} y_p\, t(p,q)\, z_q \le 1$ for all y in $X \multimap Y$ and z in $X' \multimap Y'$, the orthogonal of the products $y \otimes z$. For $X = Y = X' = Y' =$ Bool, Lean checks that T satisfies the three PCS conditions^{Lean}. A *randomized sequential program* queries one argument first. Querying f first, it chooses the input a with probability $\mu(a)$, after the answer b chooses the input c to g with probability $\kappa(a,b,c)$, and after the answer d halts with probability $\rho(a,b,c,d)$. Its value at $((a,b),(c,d))$ is $\mu(a)\kappa(a,b,c)\rho(a,b,c,d)$ plus the analogous term for programs that query g first, with $\sum \mu \le 1$ over both orders, $\sum_c \kappa(a,b,c) \le 1$, and $\rho \le 1$. It is *deterministic* if its values are in $\{0,1\}$.

Theorem 81 (second order$^{\text{Lean Lean}}$)**.** *For nonempty finite X, X' and finite Y, Y': (i) A $\{0,1\}$-valued z lies in T iff its support lies in a tree $\{((a,b),(c(b),d)) \mid b \in Y, d \in Y'\}$ for some $a \in X$ and $c : Y \to X'$, or in a tree $\{((a(d),b),(c,d))\}$ that queries g first, and these are exactly the deterministic sequential programs^{Lean}. (ii)* Mix *of the $\{0,1\}$-valued elements of T is exactly the set of randomized sequential programs.*

Part (ii) contains both directions of Kuhn's theorem for this type [Kuhn 1953]: every randomized sequential program is a mixture of deterministic ones^{Lean}, and every mixture of randomized sequential programs is a randomized sequential program^{Lean}. The second direction conditions the choices at each node on the history, and nodes reached with probability 0 are given weight 0. For $X = Y = X' = Y' =$ Bool the same statements are also proved directly^{Lean}.

A language for T. Randomized sequential programs are the denotations of a small language^{Lean}. It has four syntactic stages: no function called, f called, g called, and both called. Each stage has divergence and a finite probabilistic choice $\text{choose}(w_1 P_1, \dots, w_k P_k)$ with $\sum_i w_i \le 1$, the missing mass diverging. The first stage has $\text{call}_f\, a\, k$ and $\text{call}_g\, c\, k$ with a continuation k for each answer, the middle stages call the remaining function, and the last stage halts. The stages make each function called exactly once. A choice denotes the weighted sum of the denotations of its branches, and $\text{call}_f\, a\, k$ denotes $((a',b),(c,d)) \mapsto [a' = a]\, [\![k\, b]\!](c,d)$.

Theorem 82 (soundness and definability$^{\text{Lean Lean}}$)**.** *For nonempty finite X, X' and finite Y, Y', every program denotes a randomized sequential program, hence an element of* Mix *of the $\{0,1\}$-valued elements of T. Every $\{0,1\}$-valued element of T is the denotation of a program without choice. So* Mix *of*

the $\{0,1\}$-valued elements of T is exactly the set of denotations of programs[Lean].

The language is not a full programming language: it has no recursion and no higher-order arguments beyond f and g. Relating it to probabilistic PCF at this type is not done here.

The calculus is strictly smaller than the PCS. Call two distinct web points of T *exclusive* if a single pair of deterministic partial programs (f, g) produces both, that is, if their first components cohere in $X \multimap Y$ and their second components cohere in $X' \multimap Y'$. The *exclusivity graph* of T has the web points as vertices and the exclusive pairs as edges.

Theorem 83 (strict inclusion[Lean Lean])**.** *Let X, Y, X', Y' have at least two elements each. There are five web points of T such that the function with value $\frac{1}{2}$ on them and 0 elsewhere lies in T and is not in* Mix *of the $\{0,1\}$-valued elements of T. For $X = Y = X' = Y' =$ Bool, the total weight on the five points is at most 2 for every randomized sequential program, 2 is attained, and it is at most $\frac{5}{2}$ on T, attained by the function above.*

Over Bool, writing (a, b) for the observation $f(a) = b$, the five points are $((t,t),(t,t))$, $((t,f),(f,f))$, $((f,t),(t,f))$, $((f,t),(f,t))$, and $((f,f),(f,f))$. They induce a 5-cycle in the exclusivity graph. Lean checks the two facts the proof uses, that the induced subgraph has no triangle and no three pairwise non-adjacent vertices[Lean], and embeds the five points into larger types by injections from Bool.

The witness is partial: for some pairs of total functions it halts with probability less than 1. At T this is necessary. An element of T that halts with probability 1 for every pair of total functions is a two-party classical process without predefined causal order (a process in which the order of the two calls may depend on the inputs), and every such process is a mixture of processes in a fixed order [Baumeler and Wolf 2016]. For $X = Y = X' = Y' =$ Bool we checked by vertex enumeration, not formalized, that these elements form a polytope whose 12 vertices are $\{0,1\}$-valued. With three functional arguments the gap remains among the elements that halt with probability 1.

Proposition 84 (a gap among total programs[Lean])**.** *Let $T_3 = ((\mathbf{2} \multimap \mathbf{2}) \otimes (\mathbf{2} \multimap \mathbf{2}) \otimes (\mathbf{2} \multimap \mathbf{2}))^{\perp}$. Some element of $P(T_3)$ halts with probability 1 on every triple of total functions and is not in the calculus of T_3.*

The witness has value 1 at one point of the web and $\frac{1}{2}$ at fourteen others. Every $\{0,1\}$-valued element of $P(\mathbf{2} \multimap \mathbf{2})$ and of $P((\mathbf{2} \multimap \mathbf{2}) \otimes (\mathbf{2} \multimap \mathbf{2}))$ lies below the graph of a total function or a product of two, so by the description of $P((X \otimes Y)^{\perp})$ in Section 11.4 membership reduces to the 64 triples of total functions, on each of which the witness has mass exactly 1. Five of its points of value $\frac{1}{2}$ are pairwise exclusive along a 5-cycle, so, as for Theorem 83, every mixture of $\{0,1\}$-valued elements has weight at most 2 on them, against $\frac{5}{2}$. The witness is an instance of the three-party processes of Baumeler and Wolf that are not mixtures of deterministic ones [Baumeler and Wolf 2016].

The semantics of Definition 79 computes T: $P(((\mathbf{n} \multimap \mathbf{m}) \otimes (\mathbf{n}' \multimap \mathbf{m}'))^{\perp})$ is T for $X = \{1,\dots,n\}$, $Y = \{1,\dots,m\}$, $X' = \{1,\dots,n'\}$, $Y' = \{1,\dots,m'\}$[Lean]. So Theorem 81(ii) and the strict inclusion of Theorem 83 are statements about these formulas[Lean].

Related work. Full abstraction (the model identifies exactly the programs that no context distinguishes) of probabilistic coherence spaces for probabilistic PCF [Ehrhard et al. 2018] is proved from the analyticity of the morphisms (their power-series form), not from definability of every point, and in the concurrent game model of Castellan, Clairambault, Paquet and Winskel [Castellan et al. 2018] definability fails until strategies are restricted to sequential probabilistic innocent ones (strategies that call one argument at a time and depend only on the history they observe). Kuhn's theorem [Kuhn 1953] is the classical form of Theorem 81(ii) for games with perfect recall (each player remembers its own past moves and observations). The tensor of probabilistic coherence spaces has been related to no-signalling composition (composition in which neither component's outputs depend on the other's inputs) [Simmons and Kissinger 2022]. What is added here is an explicit point outside the calculus at a type without exponentials, its transfer to every formula with a negative occurrence of the core (Theorem 87), the description of the calculus and of the space as STAB and QSTAB of one graph (Theorem 90), the exact condition at second order (Theorem 92), and the mechanization.

11.2 Other Formulas

The strict inclusion transfers along a simple kind of embedding of webs.

Definition 85 (coordinate retract[Lean])**.** For formulas D and A of Definition 79, a *coordinate retract* of D into A is an injective map $j : |D| \to |A|$ such that the pushforward j_*, with $(j_*x)(j(d)) = x_d$ and value 0 outside the image of j, maps $P(D)$ into $P(A)$, and the pullback $z \mapsto z \circ j$ maps $P(A)$ into $P(D)$.

Theorem 86 (transfer[Lean Lean])**.** *(i) If D is a coordinate retract of A and the calculus of D is strictly smaller than $P(D)$, then the calculus of A is strictly smaller than $P(A)$. (ii) Coordinate retracts compose. If D is a coordinate retract of A, then $D^{\perp}$ is a coordinate retract of $A^{\perp}$. The formula D is a coordinate retract of $D \,\&\, B$, $B \,\&\, D$, and, if $|B|$ is nonempty, of $D \otimes B$ and $B \otimes D$.*

For (i), take $x \in P(D)$ outside the calculus. If j_*x were a mixture of $\{0,1\}$-valued elements of $P(A)$, pulling the mixture back along j would write $x = (j_*x) \circ j$ as a mixture of $\{0,1\}$-valued elements of $P(D)$. For negation, the pairing satisfies $\sum_a (j_*u)_a z_a = \sum_d u_d\, z_{j(d)}$, which exchanges the two conditions. For $D \otimes B$, the map is $j(d) = (d, b)$ for a point b

of B, so $j_*x = x \otimes \delta_b$, and the pullback condition uses that every element of $P(B)$ is at most 1 at b (Theorem 80(ii)).

An occurrence of a formula D in A is *positive* if it lies under an even number of negations and *negative* otherwise, the derived connectives $\parr$, $\oplus$, and $\multimap$ being unfolded. So the left argument of $\multimap$ is a negative position.

Theorem 87 (the gap at other formulas[Lean]). *Let $n, m, n', m' \geq 2$, and let A be a formula all of whose data types are nonempty. If $(\mathbf{n} \multimap \mathbf{m}) \otimes (\mathbf{n}' \multimap \mathbf{m}')$ occurs negatively in A, the calculus of A is strictly smaller than $P(A)$.*

A positive occurrence of D gives a coordinate retract of D into A, and a negative one a coordinate retract of $D^\perp$, by Theorem 86(ii) and induction on A, using that D is a coordinate retract of $D^{\perp\perp}$. Theorem 86(i) and Theorem 83 then give the claim. Examples are T itself, the type $((\mathbf{n} \multimap \mathbf{m}) \otimes (\mathbf{n}' \multimap \mathbf{m}')) \multimap B$ of a program that receives two functions and returns a value of type B, for every B with nonempty data types (Lean checks the instance $B = \mathbf{2}$), and every formula containing one of these positively. A positive occurrence of the core does not suffice: Section 11.4 shows that at the core itself the calculus equals the probabilistic coherence space.

The occurrence condition of Theorem 87 is syntactic. The following version depends only on the coherence graphs of Definition 79. An *induced 4-cycle* of a formula X is a quadruple of points a, b, c, d with $a \asymp b \asymp c \asymp d \asymp a$ and with a, c and b, d incoherent.

Theorem 88 (four-cycles[Lean Lean Lean]). *If X and Y both have an induced 4-cycle, the calculus of $(X \otimes Y)^\perp$ is strictly smaller than its probabilistic coherence space, and so is the calculus of every formula over nonempty data types in which $X \otimes Y$ occurs negatively.*

The four points form a coordinate retract of $\mathbf{2} \multimap \mathbf{2}$, with input 0 sent to a, c and input 1 to b, d. The pushforward lands in $P(X)$ because the coherent pairs of the cycle lie in $P(X)$, and the pullback lands in $P(\mathbf{2} \multimap \mathbf{2})$ because the incoherent pairs sum to at most 1 (Theorem 80(ii)). Coordinate retracts pass through $\otimes$, so Theorems 83 and 86 give the claim. The coherence graph of $\mathbf{n} \multimap \mathbf{m}$ has an induced 4-cycle iff $n, m \geq 2$, so Theorem 87 is the case of a literal occurrence of the core. The curried type $(\mathbf{2} \multimap \mathbf{2}) \multimap ((\mathbf{2} \multimap \mathbf{2}) \multimap \mathbf{2})$, which Theorem 87 does not cover, is an instance[Lean].

11.3 A Clique Outside the PCS

The converse of Theorem 80(iii) fails. Let $T = ((\mathbf{2} \multimap \mathbf{2}) \otimes (\mathbf{2} \multimap \mathbf{2}))^\perp$. On the five points of Theorem 83, two distinct points cohere in T iff they are not exclusive. Order them as $c_0, \ldots, c_4$ along the 5-cycle of the exclusivity graph. The coherence graph on them is the complement of this cycle, again a 5-cycle. Two distinct points of $T \otimes T$ are incoherent iff they are incoherent in one component. A clique of $(T \otimes T)^\perp$ contained in $\{c_0, \ldots, c_4\}^2$ is therefore a stable set of the strong product of the two coherence graphs (the graph on pairs in which two distinct pairs are adjacent iff each coordinate is equal or adjacent), a copy of $C_5 \boxtimes C_5$. The set $\{(c_i, c_{2i}) \mid i \in \mathbb{Z}/5\}$ is such a stable set, of size 5. It is Shannon's stable set, which shows that the Shannon capacity of C_5 (the growth rate of the largest stable sets of its strong powers) is at least $\sqrt{5}$ [Lovász 1979].

Theorem 89 (a clique outside the PCS[Lean Lean]). *Shannon's set is a clique of the coherence space $(T \otimes T)^\perp$, and its indicator is not an element of $P((T \otimes T)^\perp)$.*

The function z of Theorem 83, with value $\frac{1}{2}$ on the five points, lies in $P(T)$, so $z \otimes z$ lies in $P(T \otimes T)$. Its pairing with the indicator of Shannon's set is $5 \cdot \frac{1}{4} = \frac{5}{4} > 1$. The theorem is also proved for the web of T given directly by Bool[Lean]. From $(T \otimes T)^\perp$ on, the two natural choices of certain states therefore differ. With the $\{0,1\}$-valued elements of the PCS, the calculus is contained in the PCS at every formula (Theorem 80(iv)). With the cliques of the coherence space, it is not.

11.4 When the Calculus Equals the PCS

For a finite graph G, a *clique* is a set of pairwise adjacent vertices and a *stable set* a set of pairwise non-adjacent vertices. Let $\mathrm{STAB}(G)$ be Mix of the indicator functions of stable sets, and $\mathrm{QSTAB}(G)$ the set of $x : V \to [0, \infty]$ with $\sum_{v \in C} x_v \leq 1$ for every clique C [Grötschel et al. 1988].

Theorem 90 (graph polytopes[Lean Lean]). *$T = \mathrm{QSTAB}(G)$ and* Mix *of the $\{0,1\}$-valued elements of T is $\mathrm{STAB}(G)$, for G the exclusivity graph of T.*

Theorem 91 (odd holes and antiholes[Lean Lean]). *If $\mathrm{STAB}(G) = \mathrm{QSTAB}(G)$, then G contains no induced cycle of odd length $2k + 1 \geq 5$ and no induced complement of one. In particular, if* Mix *of the $\{0,1\}$-valued elements of T equals T, the exclusivity graph of T has neither.*

A graph with neither is *Berge*. A graph is *perfect* if every induced subgraph has chromatic number equal to its largest clique size. The converse of Theorem 91 holds: $\mathrm{STAB}(G) = \mathrm{QSTAB}(G)$ iff G is perfect [Chvátal 1975; Lovász 1972], and G is perfect iff it is Berge, by the strong perfect graph theorem [Chudnovsky et al. 2006]. The converse is not formalized.

The exact condition at second order. A formula X has the *clique property* if $P(X)$ is the set of mixtures of the indicators of the cliques of its coherence graph G_X. Data types, their negations, and $\mathbf{n} \multimap \mathbf{m}$ (Theorem 77) have it, and so does every formula built from data types with & and $\oplus$, since $P(A \,\&\, B)$ is the product of $P(A)$ and $P(B)$ and $P(A \oplus B)$ is their convex hull, embedded in the two summands [Danos and Ehrhard 2011]. Lean checks this for formulas built from $\mathbf{1}$ with & and $\oplus$[Lean]. The coherence graphs of the latter are the *cographs*, the graphs with no induced path on four vertices,

with & as join (all edges between the two parts added) and ⊕ as disjoint union.

Theorem 92 (second order$^{\text{Lean Lean Lean}}$). *Let X and Y have the clique property. (i) The calculus of $X \otimes Y$ equals its probabilistic coherence space, and the calculus of $(X \otimes Y)^{\perp}$ equals its probabilistic coherence space iff the strong product $G_X \boxtimes G_Y$ is perfect. (ii) If G_X and G_Y are cographs, this holds iff G_X or G_Y has no induced 4-cycle.*

If the calculus equals the probabilistic coherence space at X and at Y, it does so at $X \otimes Y$, $X \,\&\, Y$, and $X \oplus Y$, and $P((X \otimes Y)^{\perp})$ is cut out by the products of $\{0,1\}$-valued elements of $P(X)$ and $P(Y)$. The Lean proof derives a bipolar theorem for downward-closed sets of $\{0,1\}$-vectors from Theorem 70. With the clique property these products are the indicators of the products $K \times L$ of cliques, which are the maximal cliques of $F = G_X \boxtimes G_Y$. So $P((X \otimes Y)^{\perp}) = \mathrm{QSTAB}(F)$, its $\{0,1\}$-valued elements are the stable sets of F, and the calculus is $\mathrm{STAB}(F)$, which equals $\mathrm{QSTAB}(F)$ iff F is perfect [Chvátal 1975; Lovász 1972]. In (ii), if both graphs have an induced 4-cycle the gap is Theorem 88. The closure statements, the description of $P((X \otimes Y)^{\perp})$, and (ii) are formalized, and the equivalence with perfection in (i) is cited.

A direct proof of (ii). The converse in (ii) has a direct proof, by an explicit decomposition, that does not use perfect graph theory. Its Lean proof uses classical case distinctions on real numbers, through the library. If neither side of $A \,\&\, B$ is a clique, an incoherent pair in A and one in B form an induced 4-cycle. So a formula built from $\mathbf{1}$ with & and ⊕ that has no induced 4-cycle is isomorphic to one built from $\mathbf{1}$ with ⊕ and $A \mapsto A \,\&\, \mathbf{1}$, by a bijection of the webs that preserves coherence in both directions^{Lean}. Such bijections, and the exchange of the factors of ⊗, are coordinate retracts, so we may assume that X is built in this way. Its points are the nodes of a rooted forest, the point of $\mathbf{1}$ in $A \,\&\, \mathbf{1}$ being the root above the forest of A, and two points cohere iff one lies on the path from a root to the other. So G_X is the comparability graph of the forest (two nodes adjacent iff one is an ancestor of the other) [Golumbic 1978; Wolk 1962]. For $v : |Y| \to [0,\infty)$ let $\omega(v)$ be the largest weight of a clique of G_Y, a sum at & and a maximum at ⊕. With $z_x = z(x,\cdot)$, a nonnegative z on $|X| \times |Y|$ lies in $P((X \otimes Y)^{\perp})$ iff $\omega(\sum_{x \in \pi} z_x) \le 1$ for every path π from a root^{Lean}.

Every such z is a mixture of $\{0,1\}$-valued functions whose sums along the paths are indicators of stable sets of G_Y, and these are elements of the calculus^{Lean}. The proof is an induction on the forest. At a root r above a forest X', the pair of $m = z_r$ and the restriction y of z to X' is a mixture of pairs $(1_I, y')$ with I stable and $\omega(1_I + \sum_{x \in \pi} y'_x) \le 1$ for every path π of X', and the induction continues at y'. This root step is an induction on Y. At $Y_1 \oplus Y_2$ the two components are decomposed separately and combined with product weights. At $Y_1 \,\&\, Y_2$ a stable set lies in one component. With m_j the restriction of m to Y_j, the set I is drawn in Y_j with probability $\omega(m_j)$ and is empty with probability $1 - \omega(m_1) - \omega(m_2)$. Before the draw, the restriction y_j of y to Y_j is split as $p_j + q_j$ with $\omega(m_j + \sum_{x \in \pi} p_{j,x}) = \omega(m_j)$ and $\omega(\sum_{x \in \pi} q_{j,x}) \le \omega(m_j + \sum_{x \in \pi} y_{j,x}) - \omega(m_j)$ along every path π. The part p_j is assigned to the draws in Y_j and $q_1 + q_2$ to the empty set. The split is built by induction on Y_j and a pass down the forest that keeps the value at a node while ω stays within the level and, at the first node where it does not, keeps the part that raises ω exactly to the level^{Lean}.

Lean checks (ii) in both directions for X and Y built from $\mathbf{1}$ with & and ⊕^{Lean}. For the other formulas in (ii) with nonempty webs, the clique property makes a coherence-preserving bijection onto such a formula a coordinate retract in both directions, and this transfer is not formalized. With (i), the proof shows that the strong product of a forest comparability graph and a cograph is perfect. Ravindra proved that the strong product of two graphs with no induced path on four vertices and no induced 4-cycle, which are exactly the forest comparability graphs, is perfect, and characterized the perfect strong products of two bipartite graphs [Ravindra 1978]. The case of a forest comparability graph and an arbitrary cograph extends the first result, and we did not find it in the literature. For a single coherence space, an exercise of Ehrhard's asks for the self-duality of its clique-constrained probabilistic coherence space when the space is generated from $\mathbf{1}$ by $\perp$ and &, so that its graph is a cograph, and for its failure at the five-cycle [Ehrhard 2021]. A proof through perfection is also possible, and is not formalized: the product has no induced cycle of length at least 5 and no complement of one of length at least 6, so it is perfect by Hayward's theorem [Hayward 1985].

For $X = \mathbf{n} \multimap \mathbf{m}$ the coherence graph is complete multipartite (vertices in classes, adjacent iff in different classes) with n classes of size m, which has an induced 4-cycle iff $n, m \ge 2$. So the calculus of T equals T iff one of X, Y, X', Y' has at most one element, and at the core $(X \multimap Y) \otimes (X' \multimap Y')$ itself the calculus equals the probabilistic coherence space.

Beyond second order. The condition of Theorem 88 is not necessary in general.

Proposition 93 (a gap without four-cycles^{Lean}). *The calculus of $(\mathbf{2} \,\&\, \mathbf{1}) \multimap ((\mathbf{2} \multimap \mathbf{2}) \otimes (\mathbf{2} \multimap \mathbf{2}))$ is strictly smaller than its probabilistic coherence space. Every tensor occurring negatively in it has a factor with at most three points, so none has an induced 4-cycle.*

The formula is $((\mathbf{2} \,\&\, \mathbf{1}) \otimes T)^{\perp}$. The witness has value $\frac{1}{2}$ at five points of the web. Its membership uses four incoherent pairs of T. The five points form a 5-cycle of the coherence graph, so every $\{0,1\}$-valued element is 1 at no more than two of them, and every mixture has total weight at most 2 there, against $\frac{5}{2}$. The coherence graph of T contains an induced house, a 5-cycle with one chord, and on fifteen points of the web the

space is QSTAB of the strong product of a path on three vertices with the house, which is not perfect. Beyond second order the coherence graphs are no longer cographs, and an exact condition would have to account for the imperfect strong products of the graphs that arise.

The *independence number* $\alpha(G)$ is the largest size of a stable set, and the *fractional packing number* $\alpha^*(G)$ is the largest value of $\sum_v x_v$ over QSTAB(G). The bounds 2 and $\frac{5}{2}$ of Theorem 83 are $\alpha(C_5)$ and $\alpha^*(C_5)$. Cabello, Severini and Winter [Cabello et al. 2014] show that in a contextuality scenario (a set of outcomes with a family of jointly measurable subsets) with exclusivity graph G the noncontextual bound is $\alpha(G)$, the quantum bound is the Lovász number $\vartheta(G)$ (the largest total weight on the theta body of G, a convex set defined by orthonormal representations of the graph) [Lovász 1979], and the bound for theories satisfying only exclusivity is $\alpha^*(G)$. For C_5 this is the KCBS inequality (the five-outcome noncontextuality inequality of Klyachko, Can, Binicioğlu, and Shumovsky) [Klyachko et al. 2008], with $\vartheta(C_5) = \sqrt{5}$. A calculus for T corresponding to the theta body of the exclusivity graph [Grötschel et al. 1988], which lies between STAB and QSTAB, is not constructed here. Section 12 makes the same comparison for a Kochen–Specker set.

12 Quantum Event Logic

In quantum logic the events are the closed subspaces of a Hilbert space [Birkhoff and von Neumann 1936]. We restrict to rays, the one-dimensional subspaces, of $\mathbb{R}^d$. A *context* is an orthogonal basis, a maximal set of pairwise orthogonal rays, whose events can be observed jointly. A *state* assigns to each ray a number in $[0, \infty]$ such that the numbers on every context sum to 1. For a unit vector ψ, the *Born rule* $x_v = (\psi \cdot v)^2/(v \cdot v)$ gives a state, and mixtures of such states, the density matrices (positive semidefinite operators of trace 1), are the *quantum states*. For $d \geq 3$, Gleason's theorem [Gleason 1957] states that every state on all rays of $\mathbb{R}^d$ is a quantum state.

This section tests the construction of Section 10 on this event logic. The mathematics is known: the Kochen–Specker theorem [Kochen and Specker 1967], its 18-ray instance [Cabello et al. 1996], and the bounds of the graph-theoretic approach to contextuality (the impossibility of assigning outcomes independently of the context in which they are measured) [Cabello et al. 2014]. Cox's method has been applied to non-distributive lattices of this kind, giving non-Kolmogorovian probability measures for quantum mechanics [Holik et al. 2014]. What is added is its reading in terms of the construction, with the same graph polytopes as Section 11.4, and its mechanization.

The presentation of Definition 67 takes the rays as events and, as certain states, the $\{0,1\}$-valued states, which give exactly one ray of every context the value 1. By the Kochen–Specker theorem [Kochen and Specker 1967], for $d \geq 3$ there are none. We formalize a finite instance.

Definition 94 (Cabello's set^{Lean})**.** Cabello, Estebaranz, and García-Alcaine [Cabello et al. 1996] give 18 vectors of $\mathbb{R}^4$ with coordinates in $\{0, \pm 1\}$, and 9 sets of 4 of them:

$\{0001, 0010, 1100, 1\bar{1}00\}$, $\{0001, 0100, 1010, 10\bar{1}0\}$, $\{1\bar{1}\bar{1}\bar{1}, 1\bar{1}\bar{1}1, 1100, 0011\}$,
$\{1\bar{1}\bar{1}\bar{1}, 1111, 10\bar{1}0, 010\bar{1}\}$, $\{0010, 0100, 1001, 100\bar{1}\}$, $\{1\bar{1}\bar{1}1, 1111, 1001, 01\bar{1}0\}$,
$\{11\bar{1}1, 111\bar{1}, 1\bar{1}00, 0011\}$, $\{11\bar{1}1, \bar{1}111, 1010, 010\bar{1}\}$, $\{111\bar{1}, \bar{1}111, 1001, 01\bar{1}0\}$

where $\bar{1}$ stands for -1.

Theorem 95 (Kochen–Specker for Cabello's set$^{\text{Lean Lean}}$)**.** *Each of the* 9 *sets is an orthogonal basis of* $\mathbb{R}^4$*, and each of the* 18 *vectors lies in exactly two of them. No function from the* 18 *vectors to* $\{0,1\}$ *takes the value* 1 *at exactly one vector of every basis.*

The first two statements are checked by evaluation. The third is a parity count. Summing such a function over the 9 bases gives 9, and counts every vector twice, so the sum is even.

Theorem 96 (no certain states$^{\text{Lean Lean}}$)**.** *For the presentation whose events are the* 18 *vectors and whose certain states are the* $\{0,1\}$*-valued functions summing to* 1 *on every basis, the set of certain states and* Mix *of it are empty. For every* $\psi \in \mathbb{R}^4$ *with* $\psi \cdot \psi = 1$*, the Born-rule assignment sums to* 1 *on every basis, so it is a state that is not in* Mix *of the certain states.*

The Born-rule statement is Parseval's identity $\sum_{v \in B} (\psi \cdot v)^2/(v \cdot v) = \psi \cdot \psi$ for each orthogonal basis B, checked basis by basis^{Lean}.

Allowing divergence. In a probabilistic coherence space an element may have total mass below 1, the missing mass being the probability of divergence [Danos and Ehrhard 2011]. Allowing the same here, a state sums to at most 1 on every context. The certain states are then the $\{0,1\}$-valued functions with at most one ray of value 1 in each basis, the indicators of the stable sets of the graph that joins two rays when they lie in a common basis. These exist, and the construction is no longer empty.

Theorem 97 (a stable-set bound^{Lean})**.** *Every mixture of these certain states has total weight at most* 4 *on the* 18 *rays, and* 4 *is attained. Every Born-rule assignment of a unit vector has total weight* $\frac{9}{2}$*. In particular no Born-rule assignment is a mixture of these certain states.*

Summing over the nine bases counts every ray twice. A certain state has weight at most 1 on each basis, so its total weight is at most $\frac{9}{2}$, hence at most 4 as it is an integer. A Born-rule assignment has weight 1 on each basis. Summed over the bases, every mixture of certain states has weight at most 8, so it diverges with probability at least $\frac{1}{9}$ on some basis, while a quantum state diverges on none. The bound 4 is the independence number of the graph, and the quantum

value $\frac{9}{2}$ does not depend on the state. This is the comparison of Section 11.4 for a different graph, and the gap between 4 and $\frac{9}{2}$ is the Kochen–Specker contradiction in quantitative form [Cabello 2008; Cabello et al. 2014].

Quantum probability therefore fails criterion (1) of Section 1: no state is $\{0,1\}$-valued, so the $\{0,1\}$-valued states cannot reconstruct the event logic. Theorem 96 proves this for Cabello's set. For all rays of $\mathbb{R}^d$ with $d \geq 3$ it follows from the Kochen–Specker theorem, which is not formalized. With normalized certain states, the construction of Section 10 produces nothing at this event logic: with no certain states, every affine law is valid on the certain states, including $1 \leq 0$, and Theorem 70 returns the empty set. The mixtures of certain states are the noncontextual hidden-variable models of the event logic, and Theorem 95 is the statement that this event logic has none [Kochen and Specker 1967]. For lattice valuations, Theorem 62 shows the corresponding absence on the non-distributive lattice M_3, which has no sharp valuation.

12.1 Local Certainty

The construction of Section 10 mixes states that are certain on every context at once, and the Kochen–Specker theorem rules these out. A weaker requirement asks for certainty only within each context.

Definition 98 (local and global calculi[Lean])**.** Let E be a finite set of outcomes and C a family of subsets of E, the contexts. The *local calculus* is the set of $x : E \to [0,\infty]$ with $\sum_{e\in C} x_e = 1$ for every context C, and the *global calculus* is Mix of its $\{0,1\}$-valued elements. With divergence allowed, the sums are at most 1.

An element of the local calculus restricts on every context to an element of that context's classical calculus. Global states are local, and with a single context the two calculi agree and are classical probability on the outcomes[Lean], as criterion (2) of Section 1 requires. In the sheaf-theoretic approach to contextuality [Abramsky and Brandenburger 2011], the local calculus corresponds to the compatible families of local distributions (distributions on the contexts that agree on their overlaps) and the global calculus to the global sections (distributions on assignments of outcomes to all events), the noncontextual hidden-variable models. For hypergraphs of contexts (families of contexts on a common set of outcomes), Acín, Fritz, Leverrier, and Sainz study the same two sets, the general probabilistic models and the classical models [Acín et al. 2015]. Contexts of this kind are the tests (sets of outcomes of one experiment) of Foulis and Randall [Foulis and Randall 1972].

Theorem 99 (local certainty in linear logic[Lean])**.** *For a formula A of Definition 79, take as contexts the supports of the $\{0,1\}$-valued elements of $P(A^\perp)$, with divergence allowed. The calculus of A is contained in $P(A)$, which is contained in the local calculus, and $P(A)$ equals the local calculus iff the calculus of $A^\perp$ equals $P(A^\perp)$. At T the first inclusion is strict and the second is an equality. At $(T \otimes T)^\perp$ the second inclusion is strict.*

The local calculus is the orthogonal of the calculus of $A^\perp$, and $P(A)$ is the orthogonal of $P(A^\perp)$. Both the calculus and P are biorthogonally closed (Section 11.4), which gives the equivalence. At T the dual is the core, which has no gap (Theorem 92), and the first inclusion is strict by Theorem 83. At $(T \otimes T)^\perp$ the indicator of Shannon's set (Theorem 89) is a $\{0,1\}$-valued element of the local calculus outside P. So the probabilistic coherence space is in general neither the global nor the local calculus of its formula. It is determined by the duality between a formula and its negation. We did not find this placement in the literature.

Theorem 100 (local certainty in quantum logic[Lean])**.** *On Cabello's set with the nine bases of Definition 94 as contexts, the local calculus is the set of states and the global calculus is empty. Born-rule assignments are local. The function h with value $\frac{1}{2}$ on the nine rays $\bar{1}111$, 0001, 0010, 0100, $1\bar{1}\bar{1}1$, $1\bar{1}1\bar{1}$, $11\bar{1}1$, $111\bar{1}$, 1111 and 0 elsewhere is local and is not a mixture of Born-rule assignments.*

Every basis contains exactly two of the nine rays, so h is local. The ray 0001 lies in the span of the orthogonal rays 1001 and $100\bar{1}$, and every Born-rule assignment satisfies $x_{0001} \leq x_{1001} + x_{100\bar{1}}$, since the difference of the two sides is ψ_1^2. The function h gives these rays the values $\frac{1}{2}$, 0, and 0. The local calculus sees orthogonality within contexts but not the order of the subspaces across them, the monotonicity law of Definition 2. On a finite set of rays local certainty therefore admits states that are not mixtures of Born-rule assignments of those rays. This does not decide whether h is quantum for some other assignment of projections to the rays, in a Hilbert space of any dimension, with the same contexts. Acín, Fritz, Leverrier, and Sainz state that they do not know whether every local state on Cabello's set is quantum in that sense [Acín et al. 2015]. On all rays of $\mathbb{R}^d$ with $d \geq 3$ it does not: there every local state is a quantum state by Gleason's theorem [Gleason 1957], which is not formalized. Holik, Plastino, and Sáenz reach the quantum states from Cox's approach on non-distributive lattices [Holik et al. 2014].

Criterion (1) must be weakened as well, since no state is $\{0,1\}$-valued. A weaker form asks that the events each state is certain of, together with the events of probability 0, determine the event logic, a condition in the spirit of Mackey's order-determining sets of states (sets of states whose values determine the order of the event lattice) [Mackey 1963].

Proposition 101 (Born states determine the event logic[Lean])**.** *For rays u, v of Cabello's set, let x^u be the Born-rule assignment of the unit vector of u. Then $x^u_v = 0$ iff u and v are orthogonal, and $x^u_v = 1$ iff $u = v$.*

So the extension of quantum event logic to a probability calculus needs two changes to the construction. Certainty is

required within each context and not on all of them at once, and criterion (1) is read through the certainty and impossibility of events instead of through $\{0,1\}$-valued states. On all rays of $\mathbb{R}^d$, $d \geq 3$, the resulting calculus is the set of quantum states. On a finite fragment it is larger, and excluding the extra states needs the order of the subspace lattice across contexts.

13 Four Categorical Readings

The construction $S \mapsto \mathrm{Mix}(S)$ and criteria (1)–(3) can be read in four categorical settings: free algebras of a probability monad, orthogonality, effect algebras, and limits. The mathematics of each reading is standard, and we credit it below. This section records what each reading says about the calculi of this paper, with the new statements mechanized.

13.1 Free Convex Algebras and Simplices

Convex sets are the algebras of the finite distribution monad D [Fritz 2009; Jacobs 2011; Świrszcz 1974]. The monad sends a set to the finitely supported probability distributions on it, and an algebra of it is a set with an operation that evaluates such a distribution as a mixture. For a presentation (E, S), the inclusion $S \subseteq [0,\infty]^E$ extends to a unique affine map (a map preserving mixtures) $D(S) \to [0,\infty]^E$, and its image is $\mathrm{Mix}(S)$. The certain states are the image of the unit of D (the map sending an element to the point mass at it). Abramsky and Brandenburger describe the noncontextual models of a measurement scenario in the same way, as the image of the distributions on global assignments [Abramsky and Brandenburger 2011], and Pitowsky's correlation polytopes are the same sets [Pitowsky 1991]. In these terms, criterion (1) with closure under mixtures says that the calculus contains the image of the free algebra on the certain states, and criterion (3) says that it lies in the solution set of their laws. Theorem 73 says the two coincide.

The calculus is a *simplex* when the map $D(S) \to \mathrm{Mix}(S)$ is injective, that is, when every state is a mixture of certain states in only one way. For finite S this holds iff S is affinely independent. In the theory of generalized probabilistic theories, which describes a system by a convex set of states and a set of effects (the affine maps from states to $[0,1]$ that give the probabilities of outcomes), a system is called classical when its state space is a simplex [Barrett 2007; Plávala 2023].

Theorem 102 (simplices$^{\text{Lean Lean Lean}}$)**.** *(i) For a finite poset P, two mixtures of point valuations on $\mathrm{Low}(P)$ that agree on every lower set have the same weights. (ii) At $\mathbf{2} \multimap \mathbf{2}$ the even mixture of the identity and negation equals the even mixture of the two constant programs. (iii) Let (E, S) be a presentation with an event certain in every $s \in S$, and for all events a, b an event certain in $s \in S$ iff a and b are. Two probability measures on $\{0,1\}^E$ that vanish on bad_F for every window F and give each event the same mass agree on the σ-algebra generated by the events. The sharp valuations of every frame satisfy the hypothesis, with $\top$ and $a \sqcap b$.*

Part (i) evaluates the mixtures at $\downarrow p$ and at $\downarrow p \setminus \{p\}$, whose difference is the weight of p. It is the Möbius inversion (the inversion of summation over a partial order) of a valuation on a finite distributive lattice [Geissinger 1973; Rota 1964]. Part (ii) is the centre of the square of 2×2 stochastic matrices, Barrett's gbit [Barrett 2007]. Part (iii) applies the uniqueness of measures on a π-system (a family of sets closed under finite intersections): the finite intersections of events form one, and off the bad set of a window an intersection of events is the event of their meet.

So the valuation calculus of a finite frame is a simplex. The calculus of linear logic is not one already at first order. Part (iii) with Theorem 72 and Corollary 75 gives, in a classical metatheory, that every valuation on a frame Ω is the barycenter of a probability measure on $\{0,1\}^\Omega$ that vanishes on every bad set, unique on the σ-algebra of the events. The certain states here are all sharp valuations, the prime filters, and not only the points: on the powerset of $\mathbb{N}$ the non-principal ultrafilters can carry mass. With the extension theorem of Smiley, Horn and Tarski, and Pettis [Horn and Tarski 1948; Pettis 1951; Smiley 1944] and the correspondence between additive functionals on a Boolean algebra and Radon (inner regular) measures on its Stone space [Fremlin 2003], this makes the valuations a Bauer simplex (a compact convex set whose extreme points form a closed set and in which every point is the barycenter of a unique probability measure on them). For Boolean algebras, and more generally for MV-algebras, this is Kroupa's theorem [Kroupa 2006], and for continuous valuations on stably compact spaces (the spaces of domain theory whose patch topology is compact Hausdorff) it is the theorem of Lawson and of Alvarez-Manilla, Jung, and Keimel [Alvarez-Manilla et al. 2004]. For valuations on frames without continuity we found it only as a combination of these parts. It is not formalized.

This gives a reading of intuitionistic probability as a generalized probabilistic theory, which we did not find in the literature. On a finite frame its state space is a simplex, so it is classical in that sense. What is not classical is the set of effects, the convex hull of the indicators of opens, which is closed under pointwise $\wedge$ and $\vee$ and not under complement. Restricted theories in the sense of Janotta and Lal assume closure under complement [Janotta and Lal 2013]. Filippov, Gudder, Heinosaari, and Leppäjärvi derive it from the requirement that every effect belong to a measurement, a family of effects summing to the unit effect [Filippov et al. 2020]. Here the indicator of an open U belongs to a measurement iff U is clopen, since the other effects of the measurement sum to the indicator of the complement of U, and a $\{0,1\}$-valued nonnegative combination of indicators of opens is the indicator of their union. The effects do satisfy the conditions that Plávala lists: they are convex, contain 0 and the

unit, and separate the states [Plávala 2023]. With the states taken as the measures on certain states, part (iii) says that the opens separate them. The states cannot be recovered from the effects by the duality of generalized probabilistic theories. On the Sierpiński space, with points $\bot \le \top$ and the open $\{\top\}$, the functional $(1+t)\,\delta_\top - t\,\delta_\bot$ is nonnegative on every effect and 1 at the unit for every $t \ge 0$, and gives $\{\top\}$ the value $1+t$. The calculus fixes its states by mixing certain states instead. The simplex reading depends on enough certain states. Heunen, Landsman, and Spitters represent the quasi-states (functionals linear on each commuting family of observables) of a quantum system, which are its states when Gleason's theorem applies, as valuations on a locale internal to a topos (a category that behaves as a universe of sets with an intuitionistic internal logic), and this locale has no points when the Hilbert space has dimension greater than 2 [Heunen et al. 2009]. So quantum states are intuitionistic valuations without certain states, and the separation of intuitionistic from quantum probability above holds in a classical metatheory, where the prime ideal theorem supplies the certain states.

13.2 Orthogonality

States and affine laws form a Galois connection (a pair of order-reversing maps, sending a set of states to the laws they all satisfy and a set of laws to the states satisfying all of them, whose composite is a closure operator), and Theorem 70 says that $\mathrm{Mix}(S)$ is the closure of S. A probabilistic coherence space is closed for a smaller class of tests, the nonnegative y with $\sum_w x_w y_w \le 1$, by Girard's bipolar theorem [Girard 2004], and the category of probabilistic coherence spaces is an orthogonality category in the sense of Hyland and Schalk [Fiore et al. 2023; Hyland and Schalk 2003; Tsukada and Asada 2022]. An object of such a category is a tight pair, a set of states and a set of tests each of which is the orthogonal of the other. We ask when the calculi of a formula and of its negation form a tight pair.

Theorem 103 (duality[Lean Lean]). *For every formula A of Definition 79, the orthogonal of the calculus of A is the local calculus of $A^\perp$, and the orthogonal of the local calculus of A is the calculus of $A^\perp$. The calculi of A and of $A^\perp$ form a tight pair iff the calculus of A equals its local calculus, iff neither A nor $A^\perp$ has a gap. At the core $T^\perp$ they do not form a tight pair.*

The first two statements combine the biorthogonal closure of the calculus with the fact that orthogonals do not see mixing. The third follows with Theorem 99, and the last with Theorem 92 and the gap at T (Theorem 83). Linear negation therefore exchanges the global and the local calculus, and the probabilistic coherence space $P(A)$, with $P(A^\perp) = P(A)^\perp$, is the self-dual set between them. The calculus of certain states is not a model of linear negation, since it is not tight at $T^\perp$. The failure arises at a tensor of tight types.

Corollary 104 (tightness and the tensor[Lean]). *For all n and m, the local calculus of $\mathbf{n} \multimap \mathbf{m}$ is $P(\mathbf{n} \multimap \mathbf{m})$, so $\mathbf{n} \multimap \mathbf{m}$ is tight. For $n, m, n', m' \ge 2$, the tensor $(\mathbf{n} \multimap \mathbf{m}) \otimes (\mathbf{n}' \multimap \mathbf{m}')$ of two tight types is not tight.*

The indicator of a row $\{a\} \times \{1, \dots, m\}$ is a certain state of the dual of $\mathbf{n} \multimap \mathbf{m}$, and these indicators cut out the row-substochastic matrices, which form $P(\mathbf{n} \multimap \mathbf{m})$. With Theorem 77 the calculus is the local calculus. The second statement is Theorem 103 with the gap of Theorem 83. So the assignment of the certain-state calculus to formulas is not compositional: tightness at A and at B does not imply tightness at $A \otimes B$. The probabilistic coherence space is compositional by construction.

The convex geometry is the antiblocking duality of Fulkerson and Lovász [Fulkerson 1971; Lovász 1972]. When the certain states of A and of $A^\perp$ are the cliques of their coherence graphs, the calculus and the local calculus are the stable set polytope and the clique-constrained polytope of the complementary graph, the duality is Knuth's $\mathrm{STAB}(\overline{G})^\perp = \mathrm{QSTAB}(G)$, and tightness is STAB = QSTAB, which holds iff the graph is perfect [Chvátal 1975; Knuth 1994]. Lovász's theta body lies between the two polytopes and is also self-dual [Knuth 1994], and the chain of the three is Cabello, Severini, and Winter's classical, quantum, and exclusivity bounds [Cabello et al. 2014]. The set $P(A)$ is not the theta body: at T it is the local calculus (Theorem 99). Two sources state parts of this. In an exercise sheet, Ehrhard assigns to a coherence space E the probabilistic coherence space $p(E)$ of the $x \ge 0$ with total weight at most 1 on every clique of $E^\perp$, and asks for proofs that $p(E^\perp) = p(E)^\perp$ for the coherence spaces generated from $\mathbf{1}$ by $\bot$ and &, and that this fails whenever the five-cycle embeds in E [Ehrhard 2021]. When the certain states of the dual are all its cliques, $p(E)$ is the local calculus. Borodulin-Nadzieja, Farkas, and Pelczar-Barwacz show that a family of finite sets and its orthogonal family are geometrically dual (each family's norm ball is the dual of the other's) iff they are the cliques and the anticliques (stable sets) of a perfect graph [Borodulin-Nadzieja et al. 2026]. We did not find the tight-pair formulation for the calculus of the certain states of a formula, whose certain states can be fewer than the cliques (Theorem 89), its equivalence with the absence of a gap at the formula and at its negation, or the failure at the core.

At the core the failure of tightness is Bell's theorem. A point $((x,a),(y,b))$ of the web of $T^\perp = (\mathbf{2} \multimap \mathbf{2}) \otimes (\mathbf{2} \multimap \mathbf{2})$ is the event that two parties with inputs x and y give outputs a and b. Two events are *locally orthogonal* if some party has equal inputs and different outputs. This is the opposite of the exclusivity of points of T in Section 11.1, since the web of T is the web of $T^\perp$ and its points are read as tests instead of outcomes.

Theorem 105 (Bell at the core[Lean]). *The $\{0,1\}$-valued elements of $P(T)$ are indicators of sets of pairwise locally orthogonal events. The Popescu–Rohrlich box, with value $\frac{1}{2}$ on the events with $a+b \equiv xy \pmod 2$ and 0 elsewhere, lies in the local calculus of $T^\perp$ and not in $P(T^\perp)$.*

Two events that are not locally orthogonal are both hit by one deterministic product strategy (a pair of deterministic programs, one for each party), which lies in $P(T^\perp)$ and pairs to 2 with their indicator. So the local calculus of $T^\perp$ is the set of functions with total weight at most 1 on every set of pairwise locally orthogonal events, the principle of local orthogonality, and no three events in the support of the box are pairwise locally orthogonal. The box is outside $P(T^\perp)$ because every product of two substochastic matrices satisfies the CHSH inequality, total weight at most 3 on the eight winning events, which the box violates with 4. By Theorem 92, $P(T^\perp)$ is the calculus, the mixtures of the cliques of the core, which are the subsets of the supports of deterministic product strategies. Its normalized elements, with total weight 1 for each pair of inputs, are therefore the Bell-local behaviours. For two parties local orthogonality is equivalent to no-signalling on normalized behaviours [Fritz et al. 2013], so on normalized behaviours the local calculus of $T^\perp$ is the no-signalling polytope (the behaviours in which each party's output distribution does not depend on the other party's input), which is also Kissinger and Uijlen's tensor of the two first-order types in their category of causal processes (processes compatible with a causal order) [Kissinger and Uijlen 2019]. The identification of the normalized faces is not formalized. By Theorem 99, the gap between P and the local calculus at $T^\perp$ and the gap between the calculus and P at T are equivalent, so the pentagon point at T and the box at $T^\perp$ witness one phenomenon from its two sides.

13.3 Effect Algebras and Extension of Scalars

An effect algebra is a set with a partial commutative associative addition $\oplus$, a unit 1, and for each a a unique orthosupplement $a^\perp$ with $a \oplus a^\perp = 1$, in which $a \oplus 1$ is defined only for $a = 0$ [Foulis and Bennett 1994]. Its states are the maps to $[0,1]$ that preserve $\oplus$ and 1, so they satisfy the complement rule. Jacobs and Mandemaker extend a logic to probability by tensoring its effect algebra with the unit interval, and they state Gleason's theorem as the equation of the effects on a Hilbert space with the free effect module (the effect algebra with scalars from $[0,1]$ freely generated by a given one) on its projections [Jacobs 2015; Jacobs and Mandemaker 2012]. This reading does not reach intuitionistic logic, for three reasons.

Theorem 106 (effect-algebra states on frames[Lean Lean]). *(i) If every element of a frame has a complement, the frame is Boolean. (ii) Call $q : \Omega \to [0,\infty]$ a partial-sum state if it is normalized, monotone, and additive on joins of disjoint elements. Every valuation is a partial-sum state. On the five-element frame of Example 30, some partial-sum state violates modularity, an affine law valid on every valuation.*

Part (i) is known in substance: in a Heyting effect algebra (an effect algebra that is also a Heyting algebra in a compatible way) the Heyting negation lies below the orthosupplement, with equality exactly at the central elements (those that split the algebra as a product) [Foulis 2000]. Part (ii) uses Darst's example [Darst 1970]. Reading a frame as an effect algebra, with disjoint join as partial sum and with an orthosupplement, forces it to be Boolean by (i). Keeping the partial sum and dropping the orthosupplement gives the partial-sum states, which can violate criterion (3), by (ii) and Theorem 73. The extension of scalars that does satisfy the criteria on frames replaces partial addition by modularity. It is the valuation monoid of Definition 55, the monoid of Tarski and of Horn and Tarski [Horn and Tarski 1948; Tarski 1938], whose normalized monotone homomorphisms to $[0,\infty]$ are the valuations (Theorem 56). State notions on residuated lattices (lattices with a monoid operation and implications adjoint to it) that keep the complement rule fail criterion (1) instead. Bosbach and Riečan states (two notions of state for residuated lattices, defined by additivity conditions on their operations) on a Heyting algebra satisfy $s(\neg x) = 1 - s(x)$ and factor through its regular elements [Dvurečenskij and Rachůnek 2006]. By the second item of the paragraph on conflation in Section 10.2, on a frame that is not Boolean some sharp valuation violates this equation, so these states do not contain the certain states. For quantum logic, the local calculus of Section 12.1 is the set of states of a test space [Acín et al. 2015; Foulis and Randall 1972], and on all rays of $\mathbb{R}^d$ with $d \geq 3$ Gleason's theorem identifies it with the quantum states, the case that Jacobs and Mandemaker treat.

13.4 Limits of Finite Calculi

For arbitrarily many events, the calculi on finite windows form a functor (an assignment of sets to windows and of maps to inclusions that preserves identities and composition) from the opposite of the poset of finite windows to sets, sending a window F to Mix of the restrictions of S to F and an inclusion of windows to restriction. Its limit is the set of families of window states that agree under restriction.

Theorem 107 (limit[Lean]). *For every presentation (E,S), the set of functions satisfying every finite law valid on S, which is the calculus of Theorem 74, is in bijection with the limit of this functor, by restriction to windows.*

The proof identifies the limit with the compatible families of window states and reads a family off its values on singleton windows. Theorem 72 gives the rest. So the closure condition of Theorem 74 says that the calculus on all events is determined by its finite windows. For frames the same holds with finite sublattices in place of windows: the valuations on

a distributive lattice L are the limit, over its finite sublattices L_0, of the simplices $D(\mathrm{pt}\, L_0)$ of distributions on their prime filters. Distributive lattices are the filtered colimits (directed unions) of their finite sublattices, and valuations turn these colimits into limits, so the valuation functor is the right Kan extension of $D \circ \mathrm{pt}$ (its extension from finite lattices to all lattices by taking limits) along the inclusion of finite distributive lattices [Adámek and Rosický 1994]. This is folklore. We did not find it stated, but Gehrke, Jakl, and Reggio prove the same continuity for lattice-valued measures on distributive lattices [Gehrke et al. 2022]. It is not formalized.

The closest results are codensity theorems. The codensity monad of a functor G is the right Kan extension of G along itself. It generalizes the monad $G \circ F$ that G induces when it has a left adjoint F. Leinster shows that the ultrafilter monad (sending a set to its ultrafilters) is the codensity monad of the inclusion of finite sets into sets [Leinster 2013]. Avery shows that the finitely additive Giry monad is a codensity monad [Avery 2016], and Van Belle that it is the codensity monad of the functor sending a finite set to its simplex of distributions [Van Belle 2022]. The ultrafilters on a set are the certain states of its powerset frame, and the finitely additive probabilities are its calculus (Corollary 75). So Leinster's theorem concerns the certain states of the powerset frame and Van Belle's its calculus. The valuation functor on distributive lattices goes from lattices to convex sets and is not an endofunctor (a functor from a category to itself), which is why the statement above is a Kan extension and not a codensity monad. We do not know whether the valuations on frames arise as a codensity monad in a suitable category of spaces.

14 The Axioms of a Probability Calculus

Theorem 73 singles out $\mathrm{Mix}(S)$ by all affine laws valid on the certain states. This section asks which of these laws are needed. We call a finite set of laws *axioms* of the calculus when exactly one extension of the logic satisfies it and no proper subset has this property. We prove that an extension satisfying a set of laws exists iff the laws are valid on the certain states, and that it is unique iff the functions satisfying the laws are exactly $\mathrm{Mix}(S)$. For finitely many events, known facts about polytopes then identify the axioms with the laws that define the facets of $\mathrm{Mix}(S)$. So for finitely many events the axioms of a probability calculus are determined by the logic, up to positive factors and the choice of laws implying the equations of the affine hull of $\mathrm{Mix}(S)$. We determine them in three cases. For a Boolean algebra they are Kolmogorov's axioms. For a finite frame they are normalization, modularity, and one instance of monotonicity at each point that is not minimal. At the second-order type T over Bool they are the clique laws of T and eight laws of the form of the CHSH inequality, so what separates T from its calculus is a set of Bell inequalities.

In this section functions take finite values. Every element of $\mathrm{Mix}(S)$ does, and some affine laws hold trivially at infinite values, for instance $x_a \le x_b$ whenever $x_b = \infty$.

14.1 Existence and Uniqueness

Definition 108 (extension, axioms)**.** Let (E, S) be a presentation with E finite, and let Λ be a set of affine laws on E. An *extension* of S satisfying Λ is a set of functions $E \to [0, \infty)$ that contains S, is closed under mixtures, and satisfies every law in Λ. The set Λ *determines the calculus* if exactly one extension of S satisfies Λ. A finite set of *axioms* of the calculus is a finite set of laws that determines the calculus and has no proper subset that does.

We prove the following.

Theorem 109 (existence and uniqueness[Lean Lean Lean])**.** *Let (E, S) be a presentation with E finite, and let Λ be a set of affine laws on E. (i) An extension of S satisfying Λ exists iff every law in Λ is valid on S. (ii) Λ determines the calculus iff every law in Λ is valid on S and $\mathrm{Mix}(S)$ is the set of finite-valued functions satisfying Λ. The extension is then $\mathrm{Mix}(S)$. (iii) The laws valid on S determine the calculus, and so does every set of laws valid on S that contains a set determining it.*

If the laws are valid on S, they hold on $\mathrm{Mix}(S)$, which is then an extension. The finite-valued functions satisfying Λ form an extension too, and every extension lies between $\mathrm{Mix}(S)$ and this set. The first statement of (iii) is Theorem 73. So the sets of laws that determine the calculus are the sets of valid laws whose finite-valued solutions are $\mathrm{Mix}(S)$. They are closed under adding valid laws, the largest of them is the set of all valid laws, and the finite sets of axioms are the minimal finite ones.

14.2 Axioms and Facets

For finite E the set $\mathrm{Mix}(S)$ is a polytope, the convex hull of finitely many points. Its *affine hull* is the set of solutions of the linear equations that hold on all of $\mathrm{Mix}(S)$, and a *facet* is a face whose dimension is one less than that of $\mathrm{Mix}(S)$. Every linear inequality with real coefficients is an affine law after its negative terms are moved to the other side. The following is known [Ziegler 1995], and we do not formalize it.

Theorem 110 (axioms are facets [Ziegler 1995])**.** *Let E be finite and Λ a finite set of laws valid on S. Then Λ determines the calculus iff the laws in Λ imply the equations of the affine hull of $\mathrm{Mix}(S)$ and, for every facet F of $\mathrm{Mix}(S)$ on which no coordinate x_e vanishes without vanishing on all of $\mathrm{Mix}(S)$, Λ contains a law that holds with equality on F and not on all of $\mathrm{Mix}(S)$. So a finite set of axioms consists of one such law for each such facet, together with a minimal set of laws implying the equations of the affine hull.*

A linear inequality that follows from finitely many linear inequalities is a nonnegative combination of them, up to a constant (Farkas's lemma). Every inequality with positive weight in such a combination for a facet law is tight on the facet. So the facet law is obtained only from a law that defines the same facet, or from the constraints $x_e \geq 0$ when a coordinate vanishes on the facet and not everywhere. Conversely, the affine hull and one defining inequality for each facet describe the polytope [Ziegler 1995]. The facet laws are unique up to positive factors and the equations of the affine hull. The laws implying those equations are not unique, since any minimal generating set of them will do. For infinite sets of laws the theorem fails. On the square $[0,1]^2$, the laws $x_1 + \varepsilon x_2 \leq 1 + \varepsilon$ for all $\varepsilon > 0$, together with the other three facet laws, determine the square without the law $x_1 \leq 1$. This is why Definition 108 asks for finite sets.

On Cabello's set no state is certain (Theorem 95), so $\mathrm{Mix}(S)$ is empty, and the single law $1 \leq 0$ is a set of axioms. In this sense quantum event logic has an empty probability calculus, and its states lie in the local relaxation of Section 12.1.

14.3 Finite Frames

The axioms of a valuation (Definition 2) are normalization, monotonicity and modularity. On a finite frame most instances of monotonicity follow from the others. For a finite poset P and $p \in P$ we write $\Downarrow p = \downarrow p \setminus \{p\}$, as in Section 4. We prove the following.

Theorem 111 (axioms on finite frames$^{\text{Lean Lean Lean}}$)**.** *Let P be finite and $x : \mathrm{Low}(P) \to [0,\infty)$. (i) x is a valuation iff $x(\emptyset) = 0$, $x(P) = 1$, x is modular, and $x(\Downarrow p) \leq x(\downarrow p)$ for every $p \in P$. (ii) For every $p \in P$ with some $q < p$, the law $x(\Downarrow p) \leq x(\downarrow p)$ is needed: some x satisfies all the other laws of (i) and is not a valuation.*

Over the real numbers, $x(\emptyset) = 0$ and modularity alone give $x(L) = \sum_{p \in L} \mathrm{mass}(p)$ by the induction of Theorem 17, which uses monotonicity only to subtract in $[0,\infty]$. This is the Möbius inversion of x over P [Rota 1964]. The law at p says $\mathrm{mass}(p) \geq 0$, so the laws give monotonicity. For (ii), give p mass -1, q mass 2, and every other point mass 0. Every lower set that contains p contains q, so $x \geq 0$, and $x(\downarrow p) < x(\Downarrow p)$.

By Theorem 71 the valuations on $\mathrm{Low}(P)$ form Mix of the sharp valuations. This is a simplex whose barycentric coordinates are the masses (Theorem 102), so its facets are the sets on which one mass vanishes. At a minimal point p the mass is $x(\{p\})$, a coordinate that vanishes on the facet and not on all of $\mathrm{Mix}(S)$. At a point that is not minimal, the only coordinate that vanishes on the facet is $x(\emptyset)$, which vanishes everywhere. With Theorem 110, the axioms of the calculus of a finite frame are therefore normalization, a minimal set of instances of modularity, and the law $x(\Downarrow p) \leq x(\downarrow p)$ at each point p that is not minimal. Monotonicity is an axiom only at those points. In a Boolean algebra every point is minimal, so its axioms are normalization and instances of additivity, Kolmogorov's axioms for a finite algebra with nonnegativity built into the values.

14.4 The Second-Order Type

Let $X = Y = X' = Y' = \mathrm{Bool}$, and let G be the exclusivity graph of T (Section 11.4). For total functions $f, g : \mathrm{Bool} \to \mathrm{Bool}$, the four points produced by the pair (f, g) form a maximal clique $K_{f,g}$ of G. For $\alpha, \beta, \gamma \in \{0,1\}$, let $S_{\alpha\beta\gamma}$ be the set of points $((x,a),(y,b))$ with $a \oplus b = (x \oplus \alpha)(y \oplus \beta) \oplus \gamma$, the support of a relabelled Popescu–Rohrlich box (Theorem 105). We prove the following.

Theorem 112 (axioms at second order$^{\text{Lean Lean Lean}}$)**.** *Let $X = Y = X' = Y' = \mathrm{Bool}$. (i) Every element x of Mix of the $\{0,1\}$-valued elements of T satisfies $\sum_{e \in K_{f,g}} x_e \leq 1$ for all total f, g, and $\sum_{e \in S_{\alpha\beta\gamma}} x_e \leq 2$ for all α, β, γ. (ii) Each of these 24 laws is independent of the other 23: some function satisfies the other 23 laws and violates it. For each of the eight laws of the second kind such a function lies in T. In particular T violates each of these eight laws.*

The laws of the first kind hold on T. No three points of a set $S_{\alpha\beta\gamma}$ are pairwise non-exclusive, which Lean checks by evaluation, so a stable set of G meets $S_{\alpha\beta\gamma}$ in at most two points, and the law holds on $\mathrm{STAB}(G)$, the calculus (Theorem 90). For the law of $K_{f,g}$, the function with value $\frac{1}{3}$ on $K_{f,g}$ satisfies the other laws: another clique meets $K_{f,g}$ in at most three points, since $K_{f,g}$ is maximal, and $S_{\alpha\beta\gamma}$ meets it in at most three, since a deterministic pair wins at most three of the four rounds of a CHSH game. For the law of $S_{\alpha\beta\gamma}$, take value $\frac{1}{2}$ on five points of $S_{\alpha\beta\gamma}$ with no triangle in G. This function lies in T by Theorem 90, and every other set $S_{\alpha'\beta'\gamma'}$ contains at most four of the five points.

By an exact computation with the double description method in cddlib, not formalized, Mix of the $\{0,1\}$-valued elements of T has exactly 40 facets. On 16 of them a coordinate vanishes, and the others are the facets of the 16 clique laws and of the 8 laws of the second kind in Theorem 112. The polytope has dimension 16, so its affine hull imposes no equations. With Theorem 110, the 24 laws are the axioms of the calculus of T over Bool, and as a set of inequalities the only finite set of axioms up to positive factors. The pentagon law of Theorem 83, total weight at most 2 on its five points, is not among them: the five points lie in one set $S_{\alpha\beta\gamma}$, and the law of that set implies it.

So T satisfies the 16 clique laws, which cut it out (Theorem 90), and violates the other 8 axioms of the calculus. On the normalized behaviours of the core, the eight CHSH inequalities and positivity are necessary and sufficient for a local model, by Fine's theorem [Fine 1982]. The eight laws of the second kind are the CHSH inequalities in the form that local orthogonality gives them on T (Theorem 105). At T over Bool, the axioms of the calculus are therefore the

axioms of the probabilistic coherence space together with eight Bell inequalities.

15 Related Work

Informal precedents. Pairing probability with intuitionistic logic is not a new idea, though the literature on it is scattered. Weatherson develops intuitionistic probability axiomatically and defends it with Dutch-book arguments [Weatherson 2003]. His axioms match our valuation axioms exactly, with negation derived rather than assumed, so the object itself and the bound $v(a) + v(\neg a) \leq 1$ are his. What is new here is the decomposition of that bound's slack into dnGap and deMorganGap and its determinacy from v alone (Theorem 7). Paris shows that the assignments avoiding a Dutch book are the mixtures of a logic's two-valued valuations, and that when the valuations respect $\wedge$ and $\vee$, as in intuitionistic and modal logics, they are exactly the normalized, monotone, modular assignments [Paris 2001]. On finite frames this is Theorem 18. Williams obtains the same set from accuracy domination, under the name generalized probabilism [Williams 2012]. Paris also shows, from Shafer's results, that the mixtures of the "known true" valuations, which make a sentence true when it follows from what is known, are exactly the Dempster–Shafer belief functions [Paris 1994, 2001; Shafer 1976]. These valuations do not respect $\vee$. We mechanize an analogous fact for our intuitionistic setting: the 2-monotone case follows directly from modularity[Lean], and the full tower is Theorem 24. The identification $\mathrm{Bel} = P(\Box\,\cdot)$ connecting belief functions [Dempster 1967; Shafer 1976] to modal logic is established [Ruspini 1986]. Section 7 shows this identification is not a separate axiom for us but a special case of our frame-level slack, recovered automatically once the frame is a topology and $\neg$ is read as the interior of the complement [Lean].

On the localic side, valuations on frames are standard objects of constructive measure theory, with Vickers's work and the Coquand–Spitters development as canonical references [Coquand and Spitters 2009; Vickers 2008]. These normally build in Scott continuity and target additive, Riesz-style goals (representations of valuations as integrals). Section 4 shows Scott continuity to be a substantive assumption (it decides point-representability on the frames we examine), so our base structure omits it. Cox, Jaynes, Halpern's counterexample, and the Van Horn and Arnborg–Sjödin repairs [Arnborg and Sjödin 2000; Cox 1946; Halpern 1999; Horn 2003; Jaynes 2003] are classical throughout, and Cox's and Van Horn's derivations both lean on double-negation elimination. Section 6 proves the sum-rule half without that move, at the price of positing modularity as an axiom rather than deriving it (Example 30). Van Horn observes that dropping axiom R3 yields an interval-valued theory, exactly the belief/plausibility pair our slack separates. Weatherson derives the complement rule from excluded middle [Weatherson 2003], which is the forward direction of Theorem 11, and Gaines states the same for valuations on lattices of propositions [Gaines 1978]. The converse reduces, through sharp valuations, to Nachbin's theorem that a distributive lattice whose prime filters are all maximal is Boolean [Nachbin 1947]. We did not find the equivalence of Dempster's rule with Bayesian conditioning and zero slack at the evidence (Theorem 28) stated in the literature.

Prior formalizations. Three bodies of mechanized work are adjacent, and none overlaps. ALEA [Audebaud and Paulin-Mohring 2009] formalizes a discrete distribution monad in Coq for randomized programs: additive probability, no event logic in our sense. Closest is the HoTT/Coq line of Faissole and Spitters on *valuations* over lower reals (reals given by their rational lower bounds) [Bidlingmaier et al. 2021; Faissole and Spitters 2017], done constructively with Scott continuity and aimed at integration (Riesz) and a Giry-style valuation *monad*. Their valuations differ from ours in carrying Scott continuity, and their monad is the Scott-continuous instance of the question examined in Section 8. Our development drops the continuity their base structure builds in, and the two mechanizations do not otherwise overlap. Belief functions on finite Boolean algebras are formalized in Coq, for games of incomplete information [Pomeret-Coquot et al. 2023]. On the domain-theoretic side, commutativity of the full valuations monad on the category of directed-complete partial orders (orders in which every directed set has a supremum) is open. A hierarchy of commutative submonads, generated from simple valuations (finite mixtures of point valuations), has been constructed instead [Jia et al. 2021]. Proposition 52, on mixtures with countable weights of arbitrary valuations, is a first step in a similar direction for frames.

Non-Boolean probability elsewhere. Quantum probability on orthomodular lattices, with Gleason's theorem [Birkhoff and von Neumann 1936; Gleason 1957], is the long-standing precedent for re-basing probability on a non-Boolean event logic. Orthomodular lattices give up distributivity itself, while our frames stay distributive by definition (Definition 1), locating all the non-classical behavior in the failure of excluded middle alone. Section 12 applies the construction from certain states to a quantum event logic.

Linear logic marks a different kind of departure. Its event algebras are quantales, complete lattices equipped with an associative tensor $\otimes$ that need not be idempotent [Girard 1987]. The lattice reduct (the underlying lattice) of a quantale is already an instance of Section 9's generalization, but $\otimes$ itself is not $\sqcap$, and a valuation grading it cannot be phrased in Definition 53's vocabulary without first deciding what

modularity even means for a non-idempotent operation. Section 10 replaces modularity by the affine laws valid on the certain states. The probabilistic semantics that does exist for linear logic, Girard's probabilistic coherence spaces as developed by Danos and Ehrhard [Danos and Ehrhard 2011], grades the multiplicative structure directly rather than valuing a lattice. Section 10 shows that it contains the calculus of certain states at every multiplicative–additive formula and equals it at implications between data types, and Section 11 that the inclusion is strict at second order.

Free probability (probability for noncommuting random variables), fuzzy measures (monotone set functions without additivity), and imprecise probability [Walley 1991] vary other parameters (independence, additivity, single-valuedness). Where Walley's coherent lower previsions (lower expectations that avoid a sure loss) posit the belief/-plausibility gap directly as a coherence primitive, our slack (Section 2) is derived, forced by modularity and negation rather than assumed. Fritz's synthetic Markov-category framework (symmetric monoidal categories axiomatizing stochastic maps) [Fritz 2020] varies the ambient structure categorically rather than order-theoretically, axiomatizing conditional independence and sufficient statistics abstractly across discrete, measure-theoretic, and Gaussian probability alike. It remains unmechanized and is orthogonal to our variation in the underlying logic. Wolpert and Kinney model mathematics itself as a stochastic process, replacing binary theoremhood with a probability distribution [Wolpert and Kinney 2024]. Theoremhood is a semidecidable event, so the computability guard of Section 7 bears directly on that program: sharp credences about such events are uncomputable. Our variation is specifically the *logic*, holding the Cox-style methodology fixed. Within DS theory, the two conditionings and their inequivalence are classical [Fagin and Halpern 1991; Shafer 1976]. What is new in Theorem 28 is the exact logical characterization of their agreement, and its mechanization.

16 Conclusion

In this work, we take seriously the Cox–Jaynes reading of probability as *extended logic*, and make the extension a construction with the logic of events as a parameter. The construction fixes an event algebra (the chosen logic in algebraic form) and takes as its assignments of degrees of certainty the valuations: the maps from events to numbers satisfying monotonicity, normalization, and modularity. We specify the two conditions that qualify the result as a probability calculus: when events are certain (0 or 1 certainty), the valuations must reconstruct the underlying event logic, and the value of the construction at classical logic must be the classical theory.

The motivation for developing this extension comes from the study of program behaviour in terms of outcome probabilities. Many properties of programs are only semidecidable. However, the Boolean event algebra underlying Kolmogorov's axioms imposes the law of excluded middle (and therefore decidability) on the structure of logical connectives applied to events. This makes it not a good fit for reasoning about events that are not decidable, such as whether an arbitrary program halts: insisting on a sharp classical readout for such an event does not decide it, but instead produces an object we show to be uncomputable, and a non-sharp classical credence assigns its remainder to non-termination, an event no finite observation confirms. The extension process we propose, when fixing its logic parameter to be intuitionistic, does not conflict with the underlying event logic of computation by construction, and is capable of reasoning about the possibility of non-termination.

Substituting the Heyting event algebra (and therefore intuitionistic logic) for the Boolean algebra representing the classical logic case changes only the consequences that excluded middle supplied. Both qualifying conditions hold at this parameter value (Sections 2 and 3), and classicality enters only through the complement rule (Theorem 11) and its dynamic counterpart, the agreement of Dempster and Bayesian conditioning (Theorem 28). The complement rule degrades to the inequality $v(a) + v(\neg a) \leq 1$. The deficit in this inequality, for which we re-purpose the term slack, is a new quantity: the value of the undecided region, given by the boundary mass in the measure model. Slack is submodular yet does not determine the valuation. The weaker logic also adds the two-gap decomposition of slack, the atomic/diffuse divide, and a belief-function reading through $\neg\neg$.

Everything else holds verbatim on frames: conditioning, the product rule, Bayes symmetry, disjoint additivity, total probability over genuine partitions, and inclusion–exclusion itself. On finite frames, every valuation is a classical mass function on the frame's points. The product-rule half of Cox's theorem is independent of the logic while the sum-rule half must be posited, so the direct transcription of the classical Cox statement is unsatisfiable. Every result is machine-checked in Lean 4 over mathlib.

A recurring shape. Three of this paper's negative results share a form worth naming: two individually reasonable demands on a valuation, jointly unsatisfiable, with the proof identifying which demand is forced to give rather than merely asserting that some construction fails. On M_3 (Theorem 62), sharpness and modularity together are unsatisfiable: a 2-valued readout cannot satisfy all three pairwise sum-to-one equations at once, and distributivity is the hypothesis that has to give. Grading a non-idempotent tensor (Corollary 66) is the same shape one level up, at the axiom itself: being graded by modularity's compositional law and discriminating a from $a \otimes a$ cannot both hold, for any Ω and any $\otimes$,

and probabilistic coherence spaces [Danos and Ehrhard 2011] are the witness that keeps discrimination and gives up gradability. Demanding a sharp, uniform, *computable* readout of a semidecidable event (Theorem 47) is the same trade-off again, one level further out: bivalence and computability cannot both hold, and the classical route keeps bivalence by fiat and gives up an algorithm. None of the three results says only that a construction fails; each names the exact pair of demands responsible and shows the failure is forced, not an artifact of how the construction happened to be set up.

Toward a monad. Section 8 lays the groundwork for a categorical exploration of the construction: Val is a mechanized functor on locales, and its countably-presented fragment composes as kernels, though whether that composition is canonical on valuations rather than their presentations remains open. A commutative monad structure, where it exists, would be the intuitionistic counterpart of the Giry monad and a step toward compositional semantics for probabilistic programs with semidecidable events. One candidate formulation exists: the space of Scott-continuous valuations is the classifying locale of a geometric theory [Coquand and Spitters 2009; Vickers 2008]. Deleting its continuity axiom still leaves a geometric theory, whose points, read classically, are exactly our valuations. Its multiplication, however, must integrate against a merely modular valuation, and the diffuse part of Example 19 is exactly what such integration fails to capture. We offer the formulation as a target, not a result. Beyond intuitionistic logic, no ambient category sending a logic to its probability calculus functorially is evident from our construction, and this question remains open.

Other open problems and future work. A natural extension of our mechanization effort involves completing the Aczél development, and mechanizing the Dutch-book theorem [Weatherson 2003]. Both require substantial additional effort and were deemed out of scope for this work. To extend our mechanization effort with preliminary monad results, we would quotient the countable presentations (Proposition 52) into a Kleisli category of countably-presented kernels, and mechanize the classifying-locale formulation (which needs classifying locales of geometric theories, infrastructure mathlib currently lacks).

Some related mathematical problems remain open as well. The first is developing a representation of the diffuse part on non-spatial frames. The second is deriving modularity from a criterion more primitive than the mixture characterization (Theorem 18), knowing it cannot come from disjunction data alone (Example 30).

The construction method applies beyond intuitionistic logic: state the classical desiderata, delete or substitute the axioms the target logic no longer licenses, and check what survives, what splits, and what becomes unsatisfiable. Linear logic was the first case tried, and Section 9 closes it in the negative rather than leaving it open: substituting the quantale tensor $\otimes$ for $\sqcap$ in modularity is not merely difficult but unsatisfiable (Corollary 66), so no valuation in this paper's sense can grade $\otimes$'s consumption of evidence without losing the very discrimination $\otimes$ exists to carry [Danos and Ehrhard 2011]. What remains open is the next question this raises, not this instance of it: whether some compositional law other than modularity transplanted verbatim could play our construction's role for logics whose primitive connectives are not idempotent.

References

Samson Abramsky and Adam Brandenburger. 2011. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics* 13 (2011), 113036. https://doi.org/10.1088/1367-2630/13/11/113036

Antonio Acín, Tobias Fritz, Anthony Leverrier, and Ana Belén Sainz. 2015. A Combinatorial Approach to Nonlocality and Contextuality. *Communications in Mathematical Physics* 334, 2 (2015), 533–628. https://doi.org/10.1007/s00220-014-2260-1

János Aczél. 1966. *Lectures on Functional Equations and Their Applications*. Academic Press, New York, NY, USA.

Jiří Adámek and Jiří Rosický. 1994. *Locally Presentable and Accessible Categories*. London Mathematical Society Lecture Note Series, Vol. 189. Cambridge University Press.

Reynald Affeldt, Cyril Cohen, and Damien Rouhling. 2018. Formalization Techniques for Asymptotic Reasoning in Classical Analysis. *Journal of Formalized Reasoning* 11, 1 (2018), 43–76.

Xavier Allamigeon, Quentin Canu, and Pierre-Yves Strub. 2023. A Formal Disproof of the Hirsch Conjecture. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2023)*. ACM. https://doi.org/10.1145/3573105.3575678

Xavier Allamigeon and Ricardo D. Katz. 2019. A Formalization of Convex Polyhedra Based on the Simplex Method. *Journal of Automated Reasoning* 63, 2 (2019), 323–345.

Xavier Allamigeon, Ricardo D. Katz, and Pierre-Yves Strub. 2022. Formalizing the Face Lattice of Polyhedra. *Logical Methods in Computer Science* 18, 2 (2022). https://doi.org/10.46298/lmcs-18(2:10)2022

Mauricio Alvarez-Manilla, Achim Jung, and Klaus Keimel. 2004. The Probabilistic Powerdomain for Stably Compact Spaces. *Theoretical Computer Science* 328, 3 (2004), 221–244. https://doi.org/10.1016/j.tcs.2004.06.021

Stefan Arnborg and Gunnar Sjödin. 2000. Bayes Rules in Finite Models. In *European Conference on Artificial Intelligence (ECAI)*. IOS Press, Amsterdam, Netherlands, 571–575.

Philippe Audebaud and Christine Paulin-Mohring. 2009. Proofs of Randomized Algorithms in Coq. *Science of Computer Programming* 74, 8 (2009), 568–589.

Tom Avery. 2016. Codensity and the Giry Monad. *Journal of Pure and Applied Algebra* 220, 3 (2016), 1229–1251. https://doi.org/10.1016/j.jpaa.2015.08.017

Jonathan Barrett. 2007. Information Processing in Generalized Probabilistic Theories. *Physical Review A* 75, 3 (2007), 032304. https://doi.org/10.1103/PhysRevA.75.032304

Ämin Baumeler and Stefan Wolf. 2016. The space of logically consistent classical processes without causal order. *New Journal of Physics* 18 (2016), 013036. https://doi.org/10.1088/1367-2630/18/1/013036

Martin E. Bidlingmaier, Florian Faissole, and Bas Spitters. 2021. Synthetic Topology in Homotopy Type Theory for Probabilistic Programming. *Mathematical Structures in Computer Science* 31, 10 (2021), 1301–1329. https://doi.org/10.1017/S0960129521000165

Garrett Birkhoff. 1937. Rings of Sets. *Duke Mathematical Journal* 3, 3 (1937), 443–454.

Garrett Birkhoff and John von Neumann. 1936. The Logic of Quantum Mechanics. *Annals of Mathematics* 37, 4 (1936), 823–843.

Piotr Borodulin-Nadzieja, Barnabás Farkas, and Anna Pelczar-Barwacz. 2026. Geometric Duality, Perfect Graphs, and the Sierpiński Space. arXiv:2605.14072

Seamus Bradley. 2016. Nonclassical Probability and Convex Hulls. *Erkenntnis* 82, 1 (2016), 87–101. https://doi.org/10.1007/s10670-016-9807-9

Adán Cabello. 2008. Experimentally Testable State-Independent Quantum Contextuality. *Physical Review Letters* 101, 21 (2008), 210401.

Adán Cabello, José M. Estebaranz, and Guillermo García-Alcaine. 1996. Bell–Kochen–Specker Theorem: A Proof with 18 Vectors. *Physics Letters A* 212, 4 (1996), 183–187.

Adán Cabello, Simone Severini, and Andreas Winter. 2014. Graph-Theoretic Approach to Quantum Correlations. *Physical Review Letters* 112, 4 (2014), 040401.

Simon Castellan, Pierre Clairambault, Hugo Paquet, and Glynn Winskel. 2018. The Concurrent Game Semantics of Probabilistic PCF. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. 215–224.

Gregory J. Chaitin. 1975. A Theory of Program Size Formally Identical to Information Theory. *J. ACM* 22, 3 (1975), 329–340.

Maria Chudnovsky, Neil Robertson, Paul Seymour, and Robin Thomas. 2006. The Strong Perfect Graph Theorem. *Annals of Mathematics* 164, 1 (2006), 51–229.

Václav Chvátal. 1975. On Certain Polytopes Associated with Graphs. *Journal of Combinatorial Theory, Series B* 18, 2 (1975), 138–154.

Thierry Coquand and Bas Spitters. 2009. Integrals and Valuations. *Journal of Logic and Analysis* 1, 3 (2009), 1–22.

Richard T. Cox. 1946. Probability, Frequency and Reasonable Expectation. *American Journal of Physics* 14, 1 (1946), 1–13.

Vincent Danos and Thomas Ehrhard. 2011. Probabilistic Coherence Spaces as a Model of Higher-Order Probabilistic Computation. *Information and Computation* 209, 6 (2011), 966–991.

Richard B. Darst. 1970. The Lebesgue Decomposition, Radon–Nikodym Derivative, Conditional Expectation, and Martingale Convergence for Lattices of Sets. *Pacific J. Math.* 35, 3 (1970), 581–600. https://doi.org/10.2140/pjm.1970.35.581

Bruno de Finetti. 1974. *Theory of Probability*. Vol. 1. Wiley.

Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Conference on Automated Deduction (CADE)*. Springer, Cham, Switzerland, 625–635.

Arthur P. Dempster. 1967. Upper and Lower Probabilities Induced by a Multivalued Mapping. *Annals of Mathematical Statistics* 38, 2 (1967), 325–339.

Dieter Denneberg. 1994. *Non-Additive Measure and Integral*. Kluwer, Dordrecht, Netherlands.

Anatolij Dvurečenskij and Jiří Rachůnek. 2006. On Riečan and Bosbach States for Bounded Non-Commutative Rℓ-Monoids. *Mathematica Slovaca* 56, 5 (2006), 487–500.

Thomas Ehrhard. 2021. MPRI 2-02, TD 2. Exercise sheet, IRIF, https://www.irif.fr/~ehrhard/pub/td2021-2.pdf.

Thomas Ehrhard, Michele Pagani, and Christine Tasson. 2018. Full Abstraction for Probabilistic PCF. *J. ACM* 65, 4 (2018), 23:1–23:44.

Martín H. Escardó. 2004. Synthetic Topology of Data Types and Classical Spaces. *Electronic Notes in Theoretical Computer Science* 87 (2004), 21–156.

Ronald Fagin and Joseph Y. Halpern. 1991. A New Approach to Updating Beliefs. *Uncertainty in Artificial Intelligence* 6 (1991), 347–374.

Florian Faissole and Bas Spitters. 2017. Synthetic Topology in Homotopy Type Theory for Probabilistic Programming. In *Programming Languages meets Program Verification (PPS)*. Full version with M. E. Bidlingmaier: Faissole–Bidlingmaier–Spitters, "Synthetic topology in Homotopy Type Theory for probabilistic programming," Math. Struct. Comput. Sci. 31(4), 2021..

Sergey N. Filippov, Stan Gudder, Teiko Heinosaari, and Leevi Leppäjärvi. 2020. Operational Restrictions in General Probabilistic Theories. *Foundations of Physics* 50, 8 (2020), 850–876. https://doi.org/10.1007/s10701-020-00352-6

Arthur Fine. 1982. Hidden Variables, Joint Probability, and the Bell Inequalities. *Physical Review Letters* 48, 5 (1982), 291–295.

Marcelo Fiore, Zeinab Galal, and Farzad Jafarrahmani. 2023. Fixpoint Constructions in Focused Orthogonality Models of Linear Logic. *Electronic Notes in Theoretical Informatics and Computer Science* 3 (2023). https://doi.org/10.46298/entics.12302 MFPS 2023.

Branden Fitelson and James Hawthorne. 2010. How Bayesian Confirmation Theory Handles the Paradox of the Ravens. In *The Place of Probability in Science*, Ellery Eells and James H. Fetzer (Eds.). Boston Studies in the Philosophy of Science, Vol. 284. Springer, 247–275.

Yannick Forster, Dominik Kirst, and Gert Smolka. 2019. On Synthetic Undecidability in Coq, with an Application to the Entscheidungsproblem. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*. 38–51.

David J. Foulis. 2000. MV and Heyting Effect Algebras. *Foundations of Physics* 30, 10 (2000), 1687–1706. https://doi.org/10.1023/A:1026454318245

David J. Foulis and M. K. Bennett. 1994. Effect Algebras and Unsharp Quantum Logics. *Foundations of Physics* 24, 10 (1994), 1331–1352. https://doi.org/10.1007/BF02283036

David J. Foulis and Charles H. Randall. 1972. Operational statistics. I. Basic concepts. *J. Math. Phys.* 13 (1972), 1667–1675. https://doi.org/10.1063/1.1665890

D. H. Fremlin. 2003. *Measure Theory, Volume 4: Topological Measure Spaces*. Torres Fremlin.

Tobias Fritz. 2009. Convex Spaces I: Definition and Examples. arXiv:0903.5522

Tobias Fritz. 2020. A Synthetic Approach to Markov Kernels, Conditional Independence and Theorems on Sufficient Statistics. *Advances in Mathematics* 370 (2020), 107239.

Tobias Fritz, Ana Belén Sainz, Remigiusz Augusiak, Jonatan Bohr Brask, Rafael Chaves, Anthony Leverrier, and Antonio Acín. 2013. Local Orthogonality as a Multipartite Principle for Quantum Correlations. *Nature Communications* 4 (2013), 2263. https://doi.org/10.1038/ncomms3263

Delbert R. Fulkerson. 1971. Blocking and Anti-Blocking Pairs of Polyhedra. *Mathematical Programming* 1 (1971), 168–194.

Brian R. Gaines. 1978. Fuzzy and Probability Uncertainty Logics. *Information and Control* 38, 2 (1978), 154–169. https://doi.org/10.1016/S0019-9958(78)90165-1

Nikolaos Galatos. 2004. Minimal Varieties of Residuated Lattices. *Algebra Universalis* 52, 2–3 (2004), 215–239.

Mai Gehrke, Tomáš Jakl, and Luca Reggio. 2022. A Duality Theoretic View on Limits of Finite Structures. *Logical Methods in Computer Science* 18, 1 (2022), 16:1–16:38.

Ladnor Geissinger. 1973. Valuations on distributive lattices I. *Archiv der Mathematik* 24 (1973), 230–239. https://doi.org/10.1007/BF01228204

Paolo Ghirardato. 1997. On Independence for Non-Additive Measures, with a Fubini Theorem. *Journal of Economic Theory* 73, 2 (1997), 261–291.

Michał Gil Sanchez, Zalán Gyenis, and Leszek Wroński. 2022. Nonclassical Probability, Convex Hulls, and Dutch Books. *Episteme* 21, 2 (2022), 498–518. https://doi.org/10.1017/epi.2022.13

Jean-Yves Girard. 1987. Linear Logic. *Theoretical Computer Science* 50, 1 (1987), 1–101.

Jean-Yves Girard. 2004. Between Logic and Quantic: A Tract. In *Linear Logic in Computer Science*, Thomas Ehrhard, Jean-Yves Girard, Paul Ruet, and Philip Scott (Eds.). London Mathematical Society Lecture Note Series, Vol. 316. Cambridge University Press, 346–381.

Michèle Giry. 1982. A Categorical Approach to Probability Theory. In *Categorical Aspects of Topology and Analysis*. Lecture Notes in Mathematics, Vol. 915. Springer, Berlin, Germany, 68–85.

Andrew M. Gleason. 1957. Measures on the Closed Subspaces of a Hilbert Space. *Journal of Mathematics and Mechanics* 6, 6 (1957), 885–893.

Kurt Gödel. 1933. Eine Interpretation des intuitionistischen Aussagenkalküls. *Ergebnisse eines mathematischen Kolloquiums* 4 (1933), 39–40.

Ben Goertzel. 2017. Cost-Based Intuitionist Probabilities on Spaces of Graphs, Hypergraphs and Theorems. arXiv:1703.04382.

Martin Charles Golumbic. 1978. Trivially perfect graphs. *Discrete Mathematics* 24 (1978), 105–107.

I. J. Good. 1967. The White Shoe Is a Red Herring. *British Journal for the Philosophy of Science* 17, 4 (1967), 322.

Martin Grötschel, László Lovász, and Alexander Schrijver. 1988. *Geometric Algorithms and Combinatorial Optimization.* Springer.

Zalán Gyenis. 2025. Probabilities are Always Axiomatizable. arXiv:2511.00228.

Joseph Y. Halpern. 1999. A Counterexample to Theorems of Cox and Fine. *Journal of Artificial Intelligence Research* 10 (1999), 67–85.

Ryan B. Hayward. 1985. Weakly triangulated graphs. *Journal of Combinatorial Theory, Series B* 39, 3 (1985), 200–208.

Carl G. Hempel. 1945. Studies in the Logic of Confirmation (I.). *Mind* 54, 213 (1945), 1–26.

Chris Heunen, Nicolaas P. Landsman, and Bas Spitters. 2009. A Topos for Algebraic Quantum Theory. *Communications in Mathematical Physics* 291, 1 (2009), 63–110. https://doi.org/10.1007/s00220-009-0865-6

Otto Hölder. 1901. Die Axiome der Quantität und die Lehre vom Mass. *Berichte über die Verhandlungen der Sächsischen Gesellschaft der Wissenschaften zu Leipzig* 53 (1901), 1–64.

Federico Holik, Angel Plastino, and Manuel Sáenz. 2014. A discussion on the origin of quantum probabilities. *Annals of Physics* 340, 1 (2014), 293–310.

Alfred Horn and Alfred Tarski. 1948. Measures in Boolean Algebras. *Trans. Amer. Math. Soc.* 64, 3 (1948), 467–497. https://doi.org/10.1090/S0002-9947-1948-0028922-8

Kevin S. Van Horn. 2003. Constructing a Logic of Plausible Inference: A Guide to Cox's Theorem. *International Journal of Approximate Reasoning* 34, 1 (2003), 3–24.

Janina Hosiasson-Lindenbaum. 1940. On Confirmation. *Journal of Symbolic Logic* 5, 4 (1940), 133–148.

Martin Hyland and Andrea Schalk. 2003. Glueing and Orthogonality for Models of Linear Logic. *Theoretical Computer Science* 294, 1–2 (2003), 183–231. https://doi.org/10.1016/S0304-3975(01)00241-9

Bart Jacobs. 2011. Probabilities, Distribution Monads, and Convex Categories. *Theoretical Computer Science* 412, 28 (2011), 3323–3336. https://doi.org/10.1016/j.tcs.2011.04.005

Bart Jacobs. 2015. New Directions in Categorical Logic, for Classical, Probabilistic and Quantum Logic. *Logical Methods in Computer Science* 11, 3:24 (2015), 1–76. https://doi.org/10.2168/LMCS-11(3:24)2015

Bart Jacobs and Jorik Mandemaker. 2012. The Expectation Monad in Quantum Foundations. In *Proceedings 8th International Workshop on Quantum Physics and Logic (QPL 2011) (EPTCS, Vol. 95).* 143–182. https://doi.org/10.4204/EPTCS.95.12

Peter Janotta and Raymond Lal. 2013. Generalized Probabilistic Theories without the No-Restriction Hypothesis. *Physical Review A* 87, 5 (2013), 052131. https://doi.org/10.1103/PhysRevA.87.052131

Edwin T. Jaynes. 2003. *Probability Theory: The Logic of Science.* Cambridge University Press, Cambridge, UK.

Xiaodong Jia, Bert Lindenhovius, Michael Mislove, and Vladimir Zamdzhiev. 2021. Commutative Monads for Probabilistic Programming Languages. In *Logic in Computer Science (LICS).* IEEE.

Peter T. Johnstone. 1980. Conditions related to De Morgan's law. In *Applications of Sheaves.* Lecture Notes in Mathematics, Vol. 753. Springer, Berlin, 479–491.

Peter T. Johnstone. 1982. *Stone Spaces.* Cambridge University Press, Cambridge, UK.

Claire Jones. 1990. *Probabilistic Non-Determinism.* Ph. D. Dissertation. University of Edinburgh.

Claire Jones and Gordon D. Plotkin. 1989. A Probabilistic Powerdomain of Evaluations. In *Logic in Computer Science (LICS).* IEEE Computer Society Press, 186–195.

Aleks Kissinger and Sander Uijlen. 2019. A Categorical Semantics for Causal Structure. *Logical Methods in Computer Science* 15, 3 (2019), 15:1–15:48. https://doi.org/10.23638/LMCS-15(3:15)2019

Alexander A. Klyachko, M. Ali Can, Sinem Binicioğlu, and Alexander S. Shumovsky. 2008. Simple Test for Hidden Variables in Spin-1 Systems. *Physical Review Letters* 101, 2 (2008), 020403.

Donald E. Knuth. 1994. The Sandwich Theorem. *Electronic Journal of Combinatorics* 1 (1994), A1. https://doi.org/10.37236/1193

Kevin H. Knuth and John Skilling. 2012. Foundations of Inference. *Axioms* 1, 1 (2012), 38–73. https://doi.org/10.3390/axioms1010038

Simon Kochen and Ernst P. Specker. 1967. The Problem of Hidden Variables in Quantum Mechanics. *Journal of Mathematics and Mechanics* 17, 1 (1967), 59–87.

Anders Kock. 2012. Commutative Monads as a Theory of Distributions. *Theory and Applications of Categories* 26 (2012), 97–131.

Andrey N. Kolmogorov. 1933. *Grundbegriffe der Wahrscheinlichkeitsrechnung.* Springer, Berlin, Germany.

Tomáš Kroupa. 2006. Every State on Semisimple MV-Algebra Is Integral. *Fuzzy Sets and Systems* 157, 20 (2006), 2771–2782.

Harold W. Kuhn. 1953. Extensive Games and the Problem of Information. In *Contributions to the Theory of Games II.* Annals of Mathematics Studies, Vol. 28. Princeton University Press, 193–216.

Jan Kühr and Daniele Mundici. 2007. De Finetti Theorem and Borel States in [0, 1]-Valued Algebraic Logic. *International Journal of Approximate Reasoning* 46, 3 (2007), 605–616.

Tom Leinster. 2013. Codensity and the Ultrafilter Monad. *Theory and Applications of Categories* 28, 13 (2013), 332–370.

László Lovász. 1972. Normal Hypergraphs and the Perfect Graph Conjecture. *Discrete Mathematics* 2, 3 (1972), 253–267.

László Lovász. 1979. On the Shannon Capacity of a Graph. *IEEE Transactions on Information Theory* 25, 1 (1979), 1–7.

George W. Mackey. 1963. *The Mathematical Foundations of Quantum Mechanics.* W. A. Benjamin, New York.

Assia Mahboubi and Enrico Tassi. 2021. *Mathematical Components.* Zenodo. https://doi.org/10.5281/zenodo.4457887

The mathlib Community. 2020. The Lean Mathematical Library. In *Certified Programs and Proofs (CPP).* ACM, New York, NY, USA, 367–381.

J. C. C. McKinsey and Alfred Tarski. 1944. The Algebra of Topology. *Annals of Mathematics* 45, 1 (1944), 141–191.

Leopoldo Nachbin. 1947. Une propriété caractéristique des algèbres booléiennes. *Portugaliae Mathematica* 6 (1947), 115–118.

Giovanni Panti. 2008. Invariant Measures in Free MV-Algebras. *Communications in Algebra* 36, 8 (2008), 2849–2861.

Jeff B. Paris. 1994. *The Uncertain Reasoner's Companion: A Mathematical Perspective.* Cambridge University Press, Cambridge, UK.

Jeff B. Paris. 2001. A Note on the Dutch Book Method. In *Proceedings of the Second International Symposium on Imprecise Probabilities and Their Applications (ISIPTA).* 301–306.

B. J. Pettis. 1951. On the Extension of Measures. *Annals of Mathematics* 54, 1 (1951), 186–197. https://doi.org/10.2307/1969320

Itamar Pitowsky. 1991. Correlation Polytopes: Their Geometry and Complexity. *Mathematical Programming* 50 (1991), 395–414. https://doi.org/10.1007/BF01594946

Itamar Pitowsky. 1994. George Boole's 'Conditions of Possible Experience' and the Quantum Puzzle. *The British Journal for the Philosophy of Science* 45, 1 (1994), 95–125. https://doi.org/10.1093/bjps/45.1.95

Martin Plávala. 2023. General Probabilistic Theories: An Introduction. *Physics Reports* 1033 (2023), 1–64. https://doi.org/10.1016/j.physrep.2023.09.001

Pierre Pomeret-Coquot, Hélène Fargier, and Érik Martin-Dorel. 2023. BelGames: A Formal Theory of Games of Incomplete Information Based

on Belief Functions in the Coq Proof Assistant. In *Interactive Theorem Proving (ITP 2023) (LIPIcs, Vol. 268)*. 25:1–25:19. https://doi.org/10.4230/LIPIcs.ITP.2023.25

K. P. S. Bhaskara Rao and M. Bhaskara Rao. 1983. *Theory of Charges: A Study of Finitely Additive Measures*. Pure and Applied Mathematics, Vol. 109. Academic Press, New York, NY, USA.

G. Ravindra. 1978. Perfectness of normal products of graphs. *Discrete Mathematics* 24, 3 (1978), 291–298. https://doi.org/10.1016/0012-365X(78)90100-0

Giuseppe Rosolini. 1986. *Continuity and Effectiveness in Topoi*. Ph. D. Dissertation. University of Oxford.

Gian-Carlo Rota. 1964. On the Foundations of Combinatorial Theory I: Theory of Möbius Functions. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete* 2 (1964), 340–368. https://doi.org/10.1007/BF00531932

Enrique H. Ruspini. 1986. *The Logical Foundations of Evidential Reasoning*. Technical Report 408. SRI International, Menlo Park, CA, USA.

Glenn Shafer. 1976. *A Mathematical Theory of Evidence*. Princeton University Press, Princeton, NJ, USA.

Will Simmons and Aleks Kissinger. 2022. Higher-Order Causal Theories Are Models of BV-Logic. In *47th International Symposium on Mathematical Foundations of Computer Science (MFCS) (LIPIcs, Vol. 241)*. 80:1–80:14.

M. F. Smiley. 1944. An Extension of Metric Distributive Lattices with an Application in General Analysis. *Trans. Amer. Math. Soc.* 56 (1944), 435–447. https://doi.org/10.1090/S0002-9947-1944-0011531-0

Tadeusz Świrszcz. 1974. Monadic Functors and Convexity. *Bulletin de l'Académie Polonaise des Sciences, Série des Sciences Mathématiques, Astronomiques et Physiques* 22 (1974), 39–42.

Alfred Tarski. 1938. Algebraische Fassung des Maßproblems. *Fundamenta Mathematicae* 31 (1938), 47–66.

Takeshi Tsukada and Kazuyuki Asada. 2022. Linear-Algebraic Models of Linear Logic as Categories of Modules over Σ-Semirings. In *Proceedings of the 37th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2022)*. https://doi.org/10.1145/3531130.3533373

Ruben Van Belle. 2022. Probability Monads as Codensity Monads. *Theory and Applications of Categories* 38, 21 (2022), 811–842.

Steven Vickers. 2008. A Localic Theory of Lower and Upper Integrals. *Mathematical Logic Quarterly* 54, 1 (2008), 109–123.

Peter Walley. 1991. *Statistical Reasoning with Imprecise Probabilities*. Chapman and Hall, London, UK.

Brian Weatherson. 2003. From Classical to Intuitionistic Probability. *Notre Dame Journal of Formal Logic* 44, 2 (2003), 111–123.

J. Robert G. Williams. 2012. Generalized Probabilism: Dutch Books and Accuracy Domination. *Journal of Philosophical Logic* 41, 5 (2012), 811–840.

E. S. Wolk. 1962. The comparability graph of a tree. *Proc. Amer. Math. Soc.* 13 (1962), 789–795.

David H. Wolpert and David B. Kinney. 2024. A Stochastic Model of Mathematics and Science. *Foundations of Physics* 54 (2024), 21.

Adam Yedidia and Scott Aaronson. 2016. A Relatively Small Turing Machine Whose Behavior Is Independent of Set Theory. *Complex Systems* 25, 4 (2016), 297–327.

Chunlai Zhou. 2013. Belief functions on distributive lattices. *Artificial Intelligence* 201 (2013), 1–31. https://doi.org/10.1016/j.artint.2013.05.003

Günter M. Ziegler. 1995. *Lectures on Polytopes*. Graduate Texts in Mathematics, Vol. 152. Springer.

A Values in the Extended Reals

Valuations take values in the extended non-negative reals $[0, \infty]$, following the measure-theoretic convention of mathlib. As a result, sums indexed by an arbitrary family are unconditionally defined, with no summability hypothesis, and they commute over dependent sums: for a function f on a sigma type $\Sigma\, a,\ \beta(a)$, one has $\sum_p f(p) = \sum_a \sum_b f(a, b)$ with no side condition. This is what lets the countable-mixture results of Section 8 hold as stated. Additionally, subtraction is truncated at 0. This never affects the main text, since every subtraction there is applied to a pair already ordered by monotonicity, as with the mass of Section 4, so the truncation is never triggered.

B Lean Statements

B.1 Introduction

B.2 Valuations on Frames

```
structure Valuation (Ω : Type*) [Order.
    Frame Ω] where
  toFun : Ω → ℝ≥0∞
  map_bot' : toFun ⊥ = 0
  map_top' : toFun ⊤ = 1
  mono' : Monotone toFun
  modular' : ∀ a b, toFun a + toFun b =
      toFun (a ⊔ b) + toFun (a ⊓ b)

theorem Valuation.add_compl_le_one (v :
    Valuation Ω) (a : Ω) :
    v a + v aᶜ ≤ 1

theorem Valuation.add_compl_eq_sup (v :
    Valuation Ω) (a : Ω) :
    v a + v aᶜ = v (a ⊔ aᶜ)

theorem Valuation.le_one (v : Valuation Ω)
     (a : Ω) : v a ≤ 1

noncomputable def slack (v : Valuation Ω)
    (a : Ω) : ℝ≥0∞ := 1 - (v a + v aᶜ)

noncomputable def dnGap (v : Valuation Ω)
    (a : Ω) : ℝ≥0∞ := v aᶜᶜ - v a

noncomputable def deMorganGap (v :
    Valuation Ω) (a : Ω) : ℝ≥0∞ := 1 - (v a
    ᶜᶜ + v aᶜ)

theorem Valuation.
    slack_eq_dnGap_add_deMorganGap (v :
    Valuation Ω) (a : Ω) :
    v.slack a = v.dnGap a + v.deMorganGap
        a

theorem Valuation.slack_eq_sub_add_sub (v
    : Valuation Ω) (a : Ω) :
    v.slack a = (v aᶜᶜ - v a) + (1 - (v a
        ᶜᶜ + v aᶜ))
```

```
theorem Valuation.le_compl_compl (v :
    Valuation Ω) (a : Ω) : v a ≤ v a^cc

theorem Valuation.slack_submodular (v :
    Valuation Ω) (a b : Ω) :
    v.slack (a ⊔ b) + v.slack (a ⊓ b) ≤ v.
        slack a + v.slack b

def Order.Frame.IsDeMorgan (Ω : Type*) [
    Order.Frame Ω] : Prop :=
  ∀ a : Ω, a^cc ⊔ a^c = ⊤

theorem Valuation.
    slack_eq_dnGap_of_isDeMorgan
    (hΩ : Order.Frame.IsDeMorgan Ω) (v :
        Valuation Ω) (a : Ω) :
    v.slack a = v.dnGap a

noncomputable def Valuation.condVal (v :
    Valuation Ω) (b : Ω) (hb : v b ≠ 0) :
    Valuation Ω

theorem Valuation.condVal_mul (v :
    Valuation Ω) (b : Ω) (hb : v b ≠ 0) (a
    : Ω) :
    v.condVal b hb a * v b = v (a ⊓ b)

theorem Valuation.condVal_symm (v :
    Valuation Ω) {a b : Ω}
    (ha : v a ≠ 0) (hb : v b ≠ 0) :
    v.condVal b hb a * v b = v.condVal a
        ha b * v a

theorem Valuation.cond_add_compl_le (v :
    Valuation Ω) (a b : Ω) :
    v (a ⊓ b) + v (a^c ⊓ b) ≤ v b

theorem Valuation.
    add_compl_eq_one_of_complemented (v :
    Valuation Ω) {a : Ω}
    (ha : a ⊔ a^c = ⊤) : v a + v a^c = 1

variable [CompleteBooleanAlgebra Ω]

theorem classical_additivity (v :
    Valuation Ω) (a : Ω) :
    v a + v a^c = 1

variable [CompleteBooleanAlgebra Ω]

theorem classical_slack_zero (v :
    Valuation Ω) (a : Ω) :
    v.slack a = 0

noncomputable def Valuation.mix (w : ι → ℝ
    ≥0∞) (hw : ∑ i, w i = 1)
    (v : ι → Valuation Ω) : Valuation Ω
```

B.3 Where Classicality Lives

```
def Valuation.HasClassicalNegation (v :
    Valuation Ω) : Prop :=
  ∀ a, v a + v a^c = 1

theorem hasClassicalNegation_of_em (hem :
    ∀ a : Ω, a ⊔ a^c = ⊤)
    (v : Valuation Ω) : v.
        HasClassicalNegation

theorem em_of_forall_hasClassicalNegation
    (h : ∀ v : Valuation Ω, v.
        HasClassicalNegation) : ∀ a : Ω, a
        ⊔ a^c = ⊤

def Valuation.IsSharp (v : Valuation Ω) :
    Prop := ∀ a, v a = 0 ∨ v a = 1

noncomputable def Ideal.toValuation (J :
    Order.Ideal Ω) (hJ : J.IsPrime) (htop :
     ⊤ ∉ J) :
    Valuation Ω where
  toFun x := if x ∈ J then 0 else 1

theorem sharp_iff_point (v : Valuation Ω)
    :
    v.IsSharp ↔ ∃ (J : Order.Ideal Ω) (hJ
        : J.IsPrime) (htop : ⊤ ∉ J),
      v = Ideal.toValuation J hJ htop

def ScottContinuous (v : Valuation Ω) :
    Prop :=
  ∀ S : Set Ω, S.Nonempty → DirectedOn (·
      ≤ ·) S → v (sSup S) = ⨆ a ∈ S, v a

def Ideal.IsCompletelyPrime (J : Order.
    Ideal Ω) : Prop :=
  J.IsPrime ∧ ∀ S : Set Ω, (∀ x ∈ S, x ∈ J
      ) → sSup S ∈ J

theorem
    sharp_scott_iff_completelyPrimePoint (v
     : Valuation Ω) :
    v.IsSharp ∧ v.ScottContinuous ↔
      ∃ (J : Order.Ideal Ω) (hJ : Ideal.
          IsCompletelyPrime J) (htop : ⊤ ∉
          J),
        v = Ideal.toValuation J hJ.1 htop
```

B.4 Valuation Representation

```
noncomputable def Valuation.mass (v :
    Valuation (LowerSet P)) (p : P) : ℝ≥0∞

noncomputable def LowerSet.toFinset (U :
    LowerSet P) : Finset P :=
  (Set.toFinite (U : Set P)).toFinset

theorem Valuation.eq_sum_mass (v :
    Valuation (LowerSet P)) (U : LowerSet P
    ) :
    v U = ∑ p ∈ U.toFinset, v.mass p

theorem Valuation.sum_mass (v : Valuation
    (LowerSet P)) : ∑ p, v.mass p = 1

noncomputable def Valuation.toPMF (v :
    Valuation (LowerSet P)) : PMF P

def Valuation.mapEquiv (e : Ω ≃o Γ) (v :
    Valuation Ω) : Valuation Γ where
  toFun b := v (e.symm b)

theorem Valuation.exists_pmf_presentation
    {Θ : Type u} [Order.Frame Θ] [Finite Θ]
    (v : Valuation Θ) :
    ∃ (P : Type u) (_ : PartialOrder P) (_
        : Fintype P) (_ : DecidableEq P)
      (e : Θ ≃o LowerSet P),
      (∀ a, v a = ∑ p ∈ (e a).toFinset, (v
          .mapEquiv e).mass p) ∧
        ∑ p, (v.mapEquiv e).mass p = 1

noncomputable def deltaPoint (p : P) :
    Valuation (LowerSet P)

theorem Valuation.eq_mix_deltaPoint (v :
    Valuation (LowerSet P)) :
    v = Valuation.mix v.mass v.sum_mass (
        fun p => deltaPoint p)

noncomputable def topIndicator : Valuation
     (LowerSet ℕ) where
  toFun U := if U = ⊤ then 1 else 0

theorem
    exists_valuation_not_point_representable
     :
    ∃ v : Valuation (LowerSet ℕ), ∑' p, v.
        mass p ≠ v ⊤

theorem topIndicator_mass (n : ℕ) :
    topIndicator.mass n = 0

theorem topIndicator_not_scott :
    (⨆ n, topIndicator (LowerSet.Iic n)) ≠
        topIndicator ⊤

def IsPurelyAtomic (v : Valuation (
    LowerSet P)) : Prop := ∑' p, v.mass p =
     v ⊤

theorem eq_tsum_mass_of_scott (v :
    Valuation (LowerSet ℕ))
    (hsc : v ⊤ = ⨆ n, v (LowerSet.Iic n))
        : v ⊤ = ∑' n, v.mass n

variable [∀ p : P, Finite (Set.Iic p)]

theorem Valuation.isPurelyAtomic_of_scott
    (v : Valuation (LowerSet P))
    (hsc : v ⊤ = ⨆ (S : Finset P), v (S.
        sup LowerSet.Iic)) : IsPurelyAtomic
         v

theorem tsum_mass_le (v : Valuation (
    LowerSet P)) : ∑' p, v.mass p ≤ v ⊤
```

B.5 The Dempster–Shafer Reading

```
noncomputable def iesOdd (v : Valuation Ω)
     (s : Finset ι) (a : ι → Ω) : ℝ≥0∞ :=
  ∑ T ∈ s.powerset, if Odd T.card then v (
      T.inf a) else 0

noncomputable def iesEven (v : Valuation Ω
    ) (s : Finset ι) (a : ι → Ω) : ℝ≥0∞ :=
  ∑ T ∈ s.powerset, if T.Nonempty ∧ Even T
      .card then v (T.inf a) else 0

theorem Valuation.inclusion_exclusion (v :
     Valuation Ω) (s : Finset ι)
    (a : ι → Ω) : v (s.sup a) + v.iesEven
        s a = v.iesOdd s a

theorem Valuation.infty_monotone (v :
    Valuation Ω) (s : Finset ι) (a : ι → Ω)
     :
    v.iesOdd s a ≤ v ((s.sup a)ᶜᶜ) + v.
        iesEven s a

theorem Valuation.two_monotone (v :
    Valuation Ω) (a b : Ω) :
    v a + v b ≤ v (a ⊔ b)ᶜᶜ + v (a ⊓ b)

noncomputable def Valuation.plausibility (
    v : Valuation Ω) (a : Ω) : ℝ≥0∞ := 1 -
    v aᶜ
```

```
theorem Valuation.plausibility_sub_self (v
    : Valuation Ω) (a : Ω) :
  v.plausibility a - v a = v.slack a

noncomputable def emGap (v : Valuation Ω)
    (a b : Ω) : ℝ≥0∞ :=
  v a - v (a ⊓ (b ⊔ bᶜ))

theorem Valuation.sup_compl_decomp (v :
    Valuation Ω) (a b : Ω) :
  v (a ⊔ bᶜ) = v bᶜ + (v (a ⊓ b) + v.
      emGap a b)

noncomputable def Valuation.dempsterCond (
    v : Valuation Ω) (b : Ω)
    (hb : v b ≠ 0) : Valuation Ω

theorem Valuation.dempsterCond_apply (v :
    Valuation Ω) (b : Ω) (hb : v b ≠ 0)
    (a : Ω) : v.dempsterCond b hb a = (v (
      a ⊔ bᶜ) - v bᶜ) / (1 - v bᶜ)

theorem Valuation.
    dempsterCond_eq_condVal_iff_slack (v :
    Valuation Ω) {b : Ω}
    (hb : v b ≠ 0) :
    (∀ a, v.dempsterCond b hb a = v.
      condVal b hb a) ↔ v.slack b = 0
```

B.6 Cox's Theorem

```
structure CoxModel (Ω : Type*) [Order.
    Frame Ω] where
  pl : Ω → Ω → ℝ
  mono_left : ∀ c, Monotone fun a => pl a
      c
  pl_bot : ∀ c, pl ⊥ c = 0
  pl_top : ∀ c, pl ⊤ c = 1
  F : ℝ → ℝ → ℝ
  conj : ∀ a b c, pl (a ⊓ b) c = F (pl a (
      b ⊓ c)) (pl b c)
  F_assoc : ∀ x y z, F (F x y) z = F x (F
      y z)
  F_strictMono_left : ∀ y, 0 < y →
      StrictMono fun x => F x y

theorem isDeMorgan_lowerSetV : Order.Frame
    .IsDeMorgan (LowerSet V)

/-- `o` below the incomparable `a` and `b
    `. -/
inductive V : Type
  | o | a | b

noncomputable def qV (S : LowerSet V) : ℝ≥
    0∞ :=
  if V.o ∈ S then (if V.a ∈ S ∧ V.b ∈ S
      then 1 else 2⁻¹) else 0

theorem modularity_irreducible :
    ∃ (Ω : Type) (_ : Order.Frame Ω) (q :
      Ω → ℝ≥0∞),
      Monotone q ∧ q ⊥ = 0 ∧ q ⊤ = 1 ∧
      (∀ x y : Ω, x ⊓ y = ⊥ → q (x ⊔ y) =
          q x + q y) ∧
      ¬ (∀ x y : Ω, q x + q y = q (x ⊔ y)
          + q (x ⊓ y))

theorem no_disjunction_functional :
    ¬ ∃ S : ℝ≥0∞ → ℝ≥0∞ → ℝ≥0∞,
      ∀ x y : LowerSet V, qV (x ⊔ y) = S
          (qV x) (qV y)

theorem constructive_cox (M : CoxModel Ω)
    (hmod : ∀ a b : Ω, M.pl a ⊤ + M.pl b ⊤
      = M.pl (a ⊔ b) ⊤ + M.pl (a ⊓ b) ⊤)
      :
    ∃ (v : Valuation Ω) (g : ℝ → ℝ≥0∞),
      StrictMonoOn g (Set.Icc 0 1) ∧ g 0 =
          0 ∧ g 1 = 1 ∧
      ∀ a, v a = g (M.pl a ⊤)

noncomputable def coxModelENNReal :
    CoxModel ℝ≥0∞

theorem Scale.exists_mul_generator :
    ∃ G : ℝ → ℝ,
      (∀ x y, S.u ≤ x → S.u ≤ y → G (S.F x
          y) = G x * G y) ∧
      (∀ x y, S.u ≤ x → x < y → G x < G y)

theorem Scale.aczelStatement_cone :
    ∃ G : ℝ → ℝ,
      (∀ x y, S.u ≤ x → S.u ≤ y → G (S.F x
          y) = G x * G y) ∧
      StrictMonoOn G (Set.Ici S.u)

theorem exists_diag_eq {F : ℝ → ℝ → ℝ} {a
    b c : ℝ} (hab : a ≤ b)
    (hcont : ContinuousOn (fun x => F x x)
        (Set.Icc a b))
    (hc : c ∈ Set.Icc (F a a) (F b b)) : ∃
        x ∈ Set.Icc a b, F x x = c

structure Scale where
  F : ℝ → ℝ → ℝ
  hdiv : ∀ c, ∃ x, F x x = c
```

```
    hassoc : ∀ x y z, F (F x y) z = F x (F y
        z)
    hpos : ∀ c x, x < F x c
    hmono : ∀ x₁ x₂ y₁ y₂ : ℝ, x₁ ≤ x₂ → y₁
       ≤ y₂ → F x₁ y₁ ≤ F x₂ y₂
    hcont : Continuous fun p : ℝ × ℝ => F p
       .1 p.2
    hbot : ∀ x, Filter.Tendsto (fun z => F x
        z) Filter.atBot (nhds x)
    u : ℝ
```

B.7 Two Grounding Models

```
noncomputable def MeasureTheory.Measure.
    toValuationOpens
    (μ : Measure X) [IsProbabilityMeasure
        μ] : Valuation (Opens X)

theorem toValuationOpens_slack_eq_frontier
     (μ : Measure X)
    [IsProbabilityMeasure μ] (U : Opens X)
        :
    μ.toValuationOpens.slack U = μ (
        frontier (U : Set X))

theorem classical_posterior_compl_eq (μ :
    Measure X)
    [IsProbabilityMeasure μ] (a b : Opens
        X)
    (hb : μ.toValuationOpens b ≠ 0) :
    μ ((a : Set X)ᶜ ∩ b) / μ b
      = μ.toValuationOpens.condVal b hb aᶜ
        + μ (frontier (a : Set X) ∩ b) / μ
            b

noncomputable def MeasureTheory.Measure.
    interiorMeasure (μ : Measure X) (S :
    Set X) :
    ℝ≥0∞ := μ (interior S)

theorem interiorMeasure_add_compl_le (μ :
    Measure X) [IsProbabilityMeasure μ]
    (S : Set X) : μ.interiorMeasure S + μ.
        interiorMeasure Sᶜ ≤ 1

theorem
    isDeMorgan_opens_iff_extremallyDisconnected
     :
    Order.Frame.IsDeMorgan (Opens X) ↔
        ExtremallyDisconnected X

theorem isDeMorgan_opens_iff_stonean [
    CompactSpace X] [T2Space X] :
    Order.Frame.IsDeMorgan (Opens X) ↔
        ExtremallyDisconnected X

inductive ThreePt : Type
  | ptA : ThreePt
  | ptB : ThreePt
  | ptZ : ThreePt

theorem frontier_subset_ptZ {U : Set
    ThreePt} (hU : IsOpen U) :
    frontier U ⊆ {ptZ}

theorem exists_ne_valuation_same_slack :
    ∃ v v' : Valuation (Opens ThreePt), v.
        slack = v'.slack ∧ v ≠ v'

theorem haltsOpen_compl : haltsOpenᶜ = (⊥
    : Opens Prop)

def haltsOpen : Opens Prop := ⟨{q | q},
    isOpen_setOf_self⟩

noncomputable def coinMeasure (p : ℝ≥0∞) :
     Measure Prop :=
  p • Measure.dirac True + (1 - p) •
      Measure.dirac False

noncomputable def sierpinskiValuation {p :
     ℝ≥0∞} (hp1 : p ≤ 1) : Valuation (Opens
     Prop) :=
  (coinMeasure p).toValuationOpens

theorem slack_eq_boundary {p : ℝ≥0∞} (hp1
    : p ≤ 1) :
    (sierpinskiValuation hp1).slack
        haltsOpen
      = coinMeasure p (frontier {q : Prop
          | q})

noncomputable def haltingWeight (n : ℕ) (c
     : Code) : ℝ≥0∞ :=
  if (c.eval n).Dom then 1 else 0

noncomputable def sharpReadout (n : ℕ) (c
    : Code) : Valuation (Opens Prop) :=
  sierpinskiValuation (
      haltingWeight_le_one n c)

theorem sharpReadout_not_computable (n : ℕ
    ) :
    ¬ComputablePred fun c : Code =>
        sharpReadout n c haltsOpen = 1
```

```
/-- One fair coin flip, `Ber(true, false, 1/2)`. -/
noncomputable def fairCoin : Measure Bool := Ber(true, false, halfI)

noncomputable def cantorMeasure : Measure (ℕ → Bool) :=
  Measure.infinitePi (fun _ : ℕ => fairCoin)

noncomputable def wordRestriction (w : List Bool) : ℕ → Set Bool :=
  fun i => if h : i < w.length then {w[i]} else Set.univ

def wordCylinder (w : List Bool) : Set (ℕ → Bool) :=
  Set.pi (Finset.range w.length : Set ℕ) (wordRestriction w)

def PrefixFree (F : Finset (List Bool)) : Prop :=
  ∀ v ∈ F, ∀ w ∈ F, v ≠ w → ¬ v <+: w

theorem isOpen_wordCylinder (w : List Bool) : IsOpen (wordCylinder w)

theorem cantorMeasure_wordCylinder (w : List Bool) :
    cantorMeasure (wordCylinder w) = (1 / 2) ^ w.length

theorem disjoint_wordCylinder_of_not_prefix {v w : List Bool}
    (hv : ¬ v <+: w) (hw : ¬ w <+: v) :
    Disjoint (wordCylinder v) (wordCylinder w)

theorem cantorMeasure_biUnion_wordCylinder {F : Finset (List Bool)}
    (hF : PrefixFree F) :
    cantorMeasure (⋃ w ∈ F, wordCylinder w) = ∑ w ∈ F, (1 / 2 : ℝ≥0∞) ^ w.length
```

B.8 The Monad Question

```
def Valuation.comap (f : FrameHom Γ Ω) (v : Valuation Ω) : Valuation Γ

theorem Valuation.comap_id (v : Valuation Ω) : v.comap (FrameHom.id Ω) = v

theorem Valuation.comap_comp (f : FrameHom Γ Ω) (g : FrameHom Δ Γ)
    (v : Valuation Ω) : v.comap (f.comp g) = (v.comap f).comap g

theorem exists_rect_decomposition (W : LowerSet (ℕ × ℕ)) :
    ∃ (N : ℕ) (A B : ℕ → LowerSet ℕ),
      W = (Finset.range N).sup fun i => A i ×ˢ B i

theorem Valuation.eq_on_finsetSup_of_eq_on {G : Set Ω}
    (hG : ∀ g ∈ G, ∀ g' ∈ G, g ⊓ g' ∈ G) (u u' : Valuation Ω)
    (h : ∀ g ∈ G, u g = u' g) {ι : Type*} [DecidableEq ι]
    (s : Finset ι) (a : ι → Ω) (ha : ∀ i ∈ s, a i ∈ G) :
    u (s.sup a) = u' (s.sup a)

theorem Valuation.product_unique (v w : Valuation (LowerSet ℕ))
    {u u' : Valuation (LowerSet (ℕ × ℕ))}
    (hu : ∀ A B : LowerSet ℕ, u (A ×ˢ B) = v A * w B)
    (hu' : ∀ A B : LowerSet ℕ, u' (A ×ˢ B) = v A * w B) : u = u'

noncomputable def ultrafilterValuation (W : Ultrafilter ι) : Valuation (Set ι)

theorem rect_mem_iff {α β : Type*} {W : Ultrafilter (α × β)}
    {S : Set α} {T : Set β} :
    S ×ˢ T ∈ W ↔ S ∈ W.map Prod.fst ∧ T ∈ W.map Prod.snd

theorem product_valuation_not_unique :
    ∃ u u' : Valuation (Set (ℕ × ℕ)),
      (∀ S T : Set ℕ, u (S ×ˢ T) = u' (S ×ˢ T)) ∧ u ≠ u'

theorem fubini_fails_for_valuations :
    ∃ (v w : Valuation (Set ℕ)) (u u' : Valuation (Set (ℕ × ℕ))),
      (∀ S T : Set ℕ, u (S ×ˢ T) = v S * w T) ∧
      (∀ S T : Set ℕ, u' (S ×ˢ T) = v S * w T) ∧ u ≠ u'

noncomputable def Valuation.mixCountable {ι : Type*} (w : ι → ℝ≥0∞)
```

```
    (hw : ∑' i, w i = 1) (v : ι →
        Valuation Ω) : Valuation Ω

theorem Valuation.
    mixCountable_mixCountable {ι : Type*} {
    κ : ι → Type*}
    (w : ι → ℝ≥0∞) (hw : ∑' i, w i = 1)
    (u : ∀ i, κ i → ℝ≥0∞) (hu : ∀ i, ∑' j,
        u i j = 1)
    (v : ∀ i, κ i → Valuation Ω) :
    Valuation.mixCountable w hw
        (fun i => Valuation.mixCountable (
            u i) (hu i) (v i))
      = Valuation.mixCountable (fun p : Σ
          i, κ i => w p.1 * u p.1 p.2)
          (tsum_sigma_mul_eq_one w hw u hu
              ) (fun p => v p.1 p.2)

theorem Valuation.mixCountable_unique (v :
     Valuation Ω) :
    Valuation.mixCountable (fun _ : PUnit
        => 1) (by simp) (fun _ => v) = v

theorem Valuation.mixCountable_comm {ι κ :
     Type*}
    (w : ι → ℝ≥0∞) (hw : ∑' i, w i = 1) (u
        : κ → ℝ≥0∞) (hu : ∑' j, u j = 1)
    (v : ι → κ → Valuation Ω) :
    Valuation.mixCountable w hw (fun i =>
        Valuation.mixCountable u hu (v i))
      = Valuation.mixCountable u hu
          (fun j => Valuation.mixCountable
              w hw (fun i => v i j))
```

B.9 Valuations Beyond Frames

Theorem 59.

```
theorem exists_sharp_separating {a b : Ω}
    (hab : ¬ a ≤ b) :
    ∃ v : Valuation Ω, v.IsSharp ∧ v a = 1
        ∧ v b = 0
```

Definition 53.

```
structure LValuation (Ω : Type*) [Lattice
    Ω] [BoundedOrder Ω] where
  toFun : Ω → ℝ≥0∞
  map_bot' : toFun ⊥ = 0
  map_top' : toFun ⊤ = 1
  mono'    : Monotone toFun
  modular' : ∀ a b, toFun a + toFun b
             = toFun (a ⊔ b) + toFun (a ⊓
                 b)
```

Proposition 54.

```
def LValuation.comap (f :
    BoundedLatticeHom Γ Ω) (v : LValuation
    Ω) :
    LValuation Γ

theorem LValuation.comap_id (v :
    LValuation Ω) :
    v.comap (BoundedLatticeHom.id Ω) = v

theorem LValuation.comap_comp (f :
    BoundedLatticeHom Γ Ω)
    (g : BoundedLatticeHom Δ Γ) (v :
        LValuation Ω) :
    v.comap (f.comp g) = (v.comap f).comap
        g
```

Theorem 60.

```
def LValuation.IsSharp (v : LValuation Ω)
    : Prop := ∀ a, v a = 0 ∨ v a = 1

theorem LValuation.exists_sharp_separating
     [DistribLattice Ω]
    {a b : Ω} (hab : ¬ a ≤ b) :
    ∃ v : LValuation Ω, v.IsSharp ∧ v a =
        1 ∧ v b = 0
```

Definition 61.

```
inductive M3 : Type
  | z | a | b | c | o

instance : Lattice M3 where
  -- z bottom, o top, a b c pairwise-
      incomparable atoms;
  -- any two distinct atoms join to o and
      meet to z

instance : BoundedOrder M3 where
  top := o
  bot := z

theorem M3.not_distributive :
    ¬ ∀ x y w : M3, x ⊓ (y ⊔ w) = (x ⊓ y)
        ⊔ (x ⊓ w)
```

Theorem 62.

```
theorem M3.no_sharp_valuation (v :
    LValuation M3) : ¬ v.IsSharp
```

Definition 63.

```
def IsTensorValuation (v : Ω → ℝ≥0∞) (t :
    Ω → Ω → Ω) : Prop :=
  ∀ a b, v a + v b = v (a ⊔ b) + v (t a b)
```

Theorem 64.

```
theorem IsTensorValuation.collapse {v : Ω
    → ℝ≥0∞} {t : Ω → Ω → Ω}
    (h : IsTensorValuation v t) {a : Ω} (
        ha : v a ≠ ⊤) :
    v (t a a) = v a
```

Example 65.

```
theorem forced_constant_of_add_valuation {
    v : ℕ → ℝ≥0∞}
    (h : IsTensorValuation v (· + ·)) (
        hfin : ∀ n, v n ≠ ⊤) (n : ℕ) :
    v n = v 0
```

Corollary 66.

```
theorem
    not_isTensorValuation_of_discriminates
    {v : Ω → ℝ≥0∞}
    {t : Ω → Ω → Ω} {a : Ω} (ha : v a ≠ ⊤)
        (hne : v (t a a) ≠ v a) :
    ¬ IsTensorValuation v t
```

B.10 Probability Calculi from Certain States

```
def IsSharpVec (x : E → ℝ≥0∞) : Prop := ∀
    e, x e = 0 ∨ x e = 1

def Mix (S : Set (E → ℝ≥0∞)) : Set (E → ℝ≥
    0∞) :=
  {x | ∃ (n : ℕ) (w : Fin n → ℝ≥0∞) (s :
      Fin n → E → ℝ≥0∞),
    (∀ i, s i ∈ S) ∧ ∑ i, w i = 1 ∧ x = ∑
        i, w i • s i}

theorem sharp_mem_Mix_iff {S : Set (E → ℝ≥
    0∞)} (hS : ∀ s ∈ S, IsSharpVec s)
    {x : E → ℝ≥0∞} (hx : IsSharpVec x) : x
        ∈ Mix S ↔ x ∈ S

theorem Mix_Mix (S : Set (E → ℝ≥0∞)) : Mix
    (Mix S) = Mix S



structure AffineLaw (E : Type*) [Fintype E
    ] where
  a : E → ℝ≥0∞
  b : E → ℝ≥0∞
  k : ℝ≥0∞
  k' : ℝ≥0∞

def AffineLaw.Holds [Fintype E] (L :
    AffineLaw E) (x : E → ℝ≥0∞) : Prop :=
  ∑ e, L.a e * x e + L.k ≤ ∑ e, L.b e * x
      e + L.k'

theorem AffineLaw.holds_of_Mix [Fintype E]
    (L : AffineLaw E) {S : Set (E → ℝ≥0∞)}
    (hL : ∀ s ∈ S, L.Holds s) {x : E → ℝ≥0
        ∞} (hx : x ∈ Mix S) : L.Holds x

theorem Mix_eq_laws' {S : Set (E → ℝ≥0∞)}
    (hS : ∀ s ∈ S, IsSharpVec s) :
    Mix S = {x | ∀ L : AffineLaw E, (∀ s ∈
        S, L.Holds s) → L.Holds x}

theorem frame_recipe :
    Set.range (fun v : Valuation (LowerSet
        P) => v.toFun)
      = Mix {f | ∃ v : Valuation (LowerSet
          P), v.IsSharp ∧ f = v.toFun}

theorem Valuation.eq_deltaPoint_of_isSharp
    {v : Valuation (LowerSet P)} (hv : v.
    IsSharp) :
    ∃ p, v = deltaPoint p

structure FinLaw (E : Type*) where
  F : Finset E
  a : E → ℝ≥0∞
  b : E → ℝ≥0∞
  k : ℝ≥0∞
  k' : ℝ≥0∞

def FinLaw.Holds (L : FinLaw E) (x : E → ℝ
    ≥0∞) : Prop :=
  ∑ e ∈ L.F, L.a e * x e + L.k ≤ ∑ e ∈ L.F
      , L.b e * x e + L.k'

def restr (F : Finset E) (x : E → ℝ≥0∞) :
    F → ℝ≥0∞ := fun e => x e

theorem laws_iff_local_Mix {S : Set (E → ℝ
    ≥0∞)} (hS : ∀ s ∈ S, IsSharpVec s)
    (x : E → ℝ≥0∞) :
    (∀ L : FinLaw E, (∀ s ∈ S, L.Holds s)
        → L.Holds x) ↔
      ∀ F : Finset E, restr F x ∈ Mix (
          restr F '' S)

noncomputable def ofVec (s : Ev → ℝ≥0∞) :
    Cfg Ev := fun e => decide (s e = 1)

noncomputable def testFn {U : Set (Cfg Ev)
    } (hU : IsClopen U) : Cfg Ev →ᵇ ℝ≥0

def Cfg (Ev : Type) := Ev → Bool

def Good (S : Set (Ev → ℝ≥0∞)) (F : Finset
    Ev) (σ : Cfg Ev) : Prop :=
```

```
    ∃ s ∈ S, ∀ e : F, s e.1 = vec σ e.1

def atom (e : Ev) : Set (Cfg Ev) := {σ | σ
    e = true}

def bad (S : Set (Ev → ℝ≥0∞)) (F : Finset
    Ev) : Set (Cfg Ev) := {σ | ¬ Good S F σ
    }

theorem laws_iff_barycenter {S : Set (Ev →
    ℝ≥0∞)} (hS : ∀ s ∈ S, IsSharpVec s)
    (x : Ev → ℝ≥0∞) :
    (∀ L : FinLaw Ev, (∀ s ∈ S, L.Holds s)
        → L.Holds x) ↔
      ∃ μ : ProbabilityMeasure (Cfg Ev), (
          ∀ F, (μ : Measure (Cfg Ev)) (bad
          S F) = 0) ∧
        ∀ e, x e = (μ : Measure (Cfg Ev))
            (atom e)

def flat (X : Type*) [Fintype X] : Set (X
    → ℝ≥0∞) := {x | ∑ a, x a ≤ 1}

noncomputable def app (t : X × Y → ℝ≥0∞) (
    x : X → ℝ≥0∞) : Y → ℝ≥0∞ :=
  fun b => ∑ a, x a * t (a, b)

def lin (X Y : Type*) [Fintype X] [Fintype
    Y] : Set (X × Y → ℝ≥0∞) :=
  {t | ∀ x ∈ flat X, app t x ∈ flat Y}

theorem mem_lin_iff [DecidableEq X] (t : X
    × Y → ℝ≥0∞) :
    t ∈ lin X Y ↔ ∀ a, ∑ b, t (a, b) ≤ 1

noncomputable def pairing (x y : E → ℝ≥0∞)
    : ℝ≥0∞ := ∑ e, x e * y e

def row (a : X) : X × Y → ℝ≥0∞ := fun ab
    => if ab.1 = a then 1 else 0

def orth (S : Set (E → ℝ≥0∞)) : Set (E → ℝ
    ≥0∞) := {y | ∀ x ∈ S, pairing x y ≤ 1}

structure IsPCS (P : Set (E → ℝ≥0∞)) :
    Prop where
  closed : orth (orth P) = P
  total : ∀ e, ∃ x ∈ P, x e ≠ 0
  bounded : ∀ e, ∃ M ≠ ⊤, ∀ x ∈ P, x e ≤ M

theorem isPCS_lin : IsPCS (lin X Y)

theorem isPCS_flat : IsPCS (flat X)

theorem isPCS_tensDual : IsPCS tensDual

theorem pcs_recipe {X Y : Type} [Fintype X
    ] [Fintype Y] [DecidableEq X] :
    lin X Y = Mix (Set.range (det : (X →
        Option Y) → X × Y → ℝ≥0∞))

def flatC (X : Type*) : X → X → Prop :=
    fun x y => x = y

def dualC {X : Type*} (c : X → X → Prop) :
     X → X → Prop := fun x y => x = y ∨ ¬ c
     x y

def tensC {X Y : Type*} (c : X → X → Prop)
     (d : Y → Y → Prop) : X × Y → X × Y →
    Prop :=
  fun p q => c p.1 q.1 ∧ d p.2 q.2

def arrowC {X Y : Type*} (c : X → X → Prop
    ) (d : Y → Y → Prop) : X × Y → X × Y →
    Prop :=
  dualC (tensC c (dualC d))

def IsClique {X : Type*} (c : X → X → Prop
    ) (B : X → Prop) : Prop := ∀ x y, B x →
     B y → c x y

def secondC : W × W → W × W → Prop :=
  dualC (tensC (arrowC (flatC Bool) (flatC
      Bool)) (arrowC (flatC Bool) (flatC
      Bool)))

noncomputable def throughAll (C : Finset W
    ) : Bool → Option Bool := fun a =>
  if h : ∃ p ∈ C, p.1 = a then some (
      Classical.choose h).2 else none

theorem sharp_lin_iff_clique {X Y : Type*}
     [Fintype X] [Fintype Y] [DecidableEq X
    ] [DecidableEq Y]
    {t : X × Y → ℝ≥0∞} (h01 : ∀ p, t p = 0
         ∨ t p = 1) :
    t ∈ lin X Y ↔ IsClique (arrowC (flatC
        X) (flatC Y)) (t · = 1)

def cov (f : X → Option Y) (g : X' →
    Option Y') (pq : (X × Y) × (X' × Y')) :
     Prop :=
  f pq.1.1 = some pq.1.2 ∧ g pq.2.1 = some
      pq.2.2

def cp {X Y : Type} (p q : X × Y) : Prop
    := p = q ∨ p.1 ≠ q.1
```

```
def cp2 (s t : (X × Y) × (X' × Y')) : Prop
    := cp s.1 t.1 ∧ cp s.2 t.2

noncomputable def thru {X Y : Type} [
    DecidableEq X] (C : Finset (X × Y)) : X
    → Option Y :=
  fun a => if h : ∃ p ∈ C, p.1 = a then
      some (Classical.choose h).2 else none

def GTree (c : X') (af : Y' → X) (pq : (X
    × Y) × (X' × Y')) : Prop :=
  pq.2.1 = c ∧ pq.1.1 = af pq.2.2

def tD (X Y X' Y' : Type) [Fintype X] [
    Fintype Y] [Fintype X'] [Fintype Y'] :
    Set ((X × Y) × (X' × Y') → ℝ≥0∞) :=
  {t | ∀ y ∈ lin X Y, ∀ z ∈ lin X' Y', ∑ p
      , ∑ q, y p * t (p, q) * z q ≤ 1}

def FTree (a : X) (cf : Y → X') (pq : (X ×
    Y) × (X' × Y')) : Prop :=
  pq.1.1 = a ∧ pq.2.1 = cf pq.1.2

theorem sharp_tD_iff [Nonempty X] [
    Nonempty X'] {z : (X × Y) × (X' × Y') →
    ℝ≥0∞}
    (h01 : ∀ pq, z pq = 0 ∨ z pq = 1) :
    z ∈ tD X Y X' Y' ↔
      (∃ a cf, ∀ pq, z pq = 1 → FTree a cf
          pq) ∨ (∃ c af, ∀ pq, z pq = 1 →
          GTree c af pq)

noncomputable def Core.val (K : Core A B C
    D) (a : A) (b : B) (c : C) (d : D) : ℝ
    ≥0∞ :=
  K.μ a * K.κ a b c * K.ρ a b c d

noncomputable def Core.mass (K : Core A B
    C D) : ℝ≥0∞ := ∑ a, K.μ a

theorem sharp_mem_sequentialG [Nonempty X]
    [Nonempty X'] {z : (X × Y) × (X' × Y')
    → ℝ≥0∞}
    (hz : z ∈ tD X Y X' Y') (h01 : ∀ u, z
        u = 0 ∨ z u = 1) : z ∈ sequentialG
        X Y X' Y'

structure Core (A B C D : Type) [Fintype A
    ] [Fintype C] where
  μ : A → ℝ≥0∞
  κ : A → B → C → ℝ≥0∞
  ρ : A → B → C → D → ℝ≥0∞
  hκ : ∀ a b, ∑ c, κ a b c ≤ 1
  hρ : ∀ a b c d, ρ a b c d ≤ 1

def sequentialG (X Y X' Y' : Type) [
    Fintype X] [Fintype X'] : Set ((X × Y)
    × (X' × Y') → ℝ≥0∞) :=
  {t | ∃ (F : Core X Y X' Y') (G : Core X'
      Y' X Y), F.mass + G.mass ≤ 1 ∧
    t = fun pq => F.val pq.1.1 pq.1.2 pq
        .2.1 pq.2.2 + G.val pq.2.1 pq.2.2
        pq.1.1 pq.1.2}

theorem recipe_eq_sequentialG [Nonempty X]
    [Nonempty X'] :
    Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u =
        0 ∨ z u = 1} = sequentialG X Y X'
        Y'

def sharpSeqG (X Y X' Y' : Type) [Fintype
    X] [Fintype X'] : Set ((X × Y) × (X' ×
    Y') → ℝ≥0∞) :=
  {z | z ∈ sequentialG X Y X' Y' ∧ ∀ u, z
      u = 0 ∨ z u = 1}

theorem sequentialG_subset_Mix :
    sequentialG X Y X' Y' ⊆ Mix (sharpSeqG
    X Y X' Y')

noncomputable def compl' (m : A → ℝ≥0∞) :
    Option A → ℝ≥0∞
  | none => 1 - ∑ a, m a
  | some a => m a

abbrev ChoiceG (A B C D : Type) := (Unit →
    Option A) × (A × B → Option C) × (A ×
    B × C × D → Bool)

noncomputable def detCoreG [DecidableEq A]
    [DecidableEq C] (σ : ChoiceG A B C D)
    : Core A B C D

theorem Mix_sequentialG : Mix (sequentialG
    X Y X' Y') = sequentialG X Y X' Y'

def emb (s : W × W) : (X × Y) × (X' × Y')
    :=
  ((ex s.1.1, ey s.1.2), (ex' s.2.1, ey' s
      .2.2))

noncomputable def pentE (ex : Bool → X) (
    ey : Bool → Y) (ex' : Bool → X') (ey' :
    Bool → Y') :
    (X × Y) × (X' × Y') → ℝ≥0∞ :=
  fun u => if u ∈ pent.image (emb ex ey ex
      ' ey') then 2⁻¹ else 0
```

```
theorem pentE_not_mix (hx : Function.
    Injective ex) (hy : Function.Injective
    ey)
    (hx' : Function.Injective ex') (hy' :
        Function.Injective ey')
    {n : ℕ} (w : Fin n → ℝ≥0∞) (hw : ∑ i,
        w i = 1) (z : Fin n → (X × Y) × (X'
         × Y') → ℝ≥0∞)
    (hz : ∀ i, z i ∈ tD X Y X' Y') (h01 :
        ∀ i u, z i u = 0 ∨ z i u = 1) :
    pentE ex ey ex' ey' ≠ ∑ i, w i • z i

theorem pentE_mem (hx : Function.Injective
     ex) (hy : Function.Injective ey)
    (hx' : Function.Injective ex') (hy' :
        Function.Injective ey') :
    pentE ex ey ex' ey' ∈ tD X Y X' Y'

noncomputable def idBool : Bool × Bool → ℝ
    ≥0∞ := fun ab => if ab.1 = ab.2 then 1
    else 0

noncomputable def det (f : X → Option Y) :
     X × Y → ℝ≥0∞

noncomputable def rowDist (t : X × Y → ℝ≥0
    ∞) (a : X) : Option Y → ℝ≥0∞
  | none => 1 - ∑ b, t (a, b)
  | some b => t (a, b)

noncomputable def tens (x : X → ℝ≥0∞) (y :
     Y → ℝ≥0∞) : X × Y → ℝ≥0∞ := fun ab =>
    x ab.1 * y ab.2

abbrev W := Bool × Bool

def pent : Finset (W × W) :=
  {((true, true), (true, true)), ((true,
      false), (false, false)),
   ((false, true), (true, false)), ((false
       , true), (false, true)),
   ((false, false), (false, false))}

noncomputable def pentElt : W × W → ℝ≥0∞
    := fun pq => if pq ∈ pent then 2⁻¹ else
     0

def covers (f g : Bool → Option Bool) (pq
    : W × W) : Prop :=
  f pq.1.1 = some pq.1.2 ∧ g pq.2.1 = some
      pq.2.2

def compat (p q : W) : Prop := p = q ∨ p.1
     ≠ q.1

def compat2 (s t : W × W) : Prop := compat
     s.1 t.1 ∧ compat s.2 t.2

def through (p q : W) : Bool → Option Bool
     := fun b => if b = p.1 then some p.2
    else some q.2

theorem pent_clique_le_two : ∀ B ∈ pent.
    powerset,
    (∀ s ∈ B, ∀ t ∈ B, compat2 s t) → B.
        card ≤ 2

theorem pent_indep_le_two : ∀ B ∈ pent.
    powerset,
    (∀ s ∈ B, ∀ t ∈ B, compat2 s t → s = t
        ) → B.card ≤ 2

structure FFirst where
  μ : Bool → ℝ≥0∞
  κ : Bool → Bool → Bool → ℝ≥0∞
  ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
  hμ : μ true + μ false ≤ 1
  hκ : ∀ a b, κ a b true + κ a b false ≤ 1
  hρ : ∀ a b c d, ρ a b c d ≤ 1

structure GFirst where
  μ : Bool → ℝ≥0∞
  κ : Bool → Bool → Bool → ℝ≥0∞
  ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
  hμ : μ true + μ false ≤ 1
  hκ : ∀ c d, κ c d true + κ c d false ≤ 1
  hρ : ∀ c d a b, ρ c d a b ≤ 1

noncomputable def FFirst.den (S : FFirst)
    : W × W → ℝ≥0∞ := fun pq =>
  S.μ pq.1.1 * S.κ pq.1.1 pq.1.2 pq.2.1 *
      S.ρ pq.1.1 pq.1.2 pq.2.1 pq.2.2

noncomputable def GFirst.den (S : GFirst)
    : W × W → ℝ≥0∞ := fun pq =>
  S.μ pq.2.1 * S.κ pq.2.1 pq.2.2 pq.1.1 *
      S.ρ pq.2.1 pq.2.2 pq.1.1 pq.1.2

noncomputable def copyStrategy : FFirst
    where
  μ a

def idleG : GFirst where
  μ _

def idleF : FFirst where
```

```
  μ _

def tensDual : Set (W × W → ℝ≥0∞) :=
  {t | ∀ y ∈ lin Bool Bool, ∀ z ∈ lin Bool
       Bool, ∑ p, ∑ q, y p * t (p, q) * z q
       ≤ 1}

def sequential : Set (W × W → ℝ≥0∞) :=
  {t | ∃ (α β : ℝ≥0∞) (F : FFirst) (G :
      GFirst), α + β ≤ 1 ∧ t = α • F.den +
      β • G.den}

theorem seq_pent_le_two {t : W × W → ℝ≥0∞}
     (ht : t ∈ sequential) : ∑ pq ∈ pent, t
     pq ≤ 2

theorem seq_pent_tight : ∃ t ∈ sequential,
     ∑ pq ∈ pent, t pq = 2

theorem pentElt_pent_sum : ∑ pq ∈ pent,
    pentElt pq = 5 * 2⁻¹

theorem tensDual_pent_le {t : W × W → ℝ≥0∞
    } (ht : t ∈ tensDual) :
    2 * ∑ pq ∈ pent, t pq ≤ 5

theorem sharp_bang_mixed_zero {z : ℕ × ℕ →
     ℝ≥0∞} (hz : z ∈ bang) (h01 : ∀ m, z m
    = 0 ∨ z m = 1)
    {i j : ℕ} (hi : 1 ≤ i) (hj : 1 ≤ j) :
        z (i, j) = 0

def orthI {E : Type*} (S : Set (E → ℝ≥0∞))
     : Set (E → ℝ≥0∞) := {y | ∀ x ∈ S, ∑' e
    , x e * y e ≤ 1}

def gens : Set (ℕ × ℕ → ℝ≥0∞) := {u | ∃ x
    ∈ flat Bool, u = prom x}

noncomputable def half : Bool → ℝ≥0∞ :=
    fun _ => 2⁻¹

noncomputable def prom (x : Bool → ℝ≥0∞) :
     ℕ × ℕ → ℝ≥0∞ := fun m => x true ^ m.1
    * x false ^ m.2

def bang : Set (ℕ × ℕ → ℝ≥0∞) := orthI (
    orthI gens)

theorem prom_half_not_mix : prom half ∈
    bang ∧
    prom half ∉ Mix {z | z ∈ bang ∧ ∀ m, z
         m = 0 ∨ z m = 1}

def excl (s t : (X × Y) × (X' × Y')) :
    Prop := s ≠ t ∧ cp2 s t

theorem tD_eq_QSTAB : tD X Y X' Y' = QSTAB
     (excl (X := X) (Y := Y) (X' := X') (Y'
     := Y'))

theorem recipe_eq_STAB :
    Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u =
         0 ∨ z u = 1}
      = STAB (excl (X := X) (Y := Y) (X'
         := X') (Y' := Y'))

def cyc {n : ℕ} (i j : ZMod n) : Prop := j
     = i + 1 ∨ i = j + 1

def IsCliqueG (C : Finset V) : Prop := ∀ u
     ∈ C, ∀ v ∈ C, u ≠ v → adj u v

def IsStableG (I : Finset V) : Prop := ∀ u
     ∈ I, ∀ v ∈ I, ¬ adj u v

noncomputable def ind (I : Finset V) : V →
     ℝ≥0∞ := fun v => if v ∈ I then 1 else
    0

structure Antihole (adj : V → V → Prop) (n
     : ℕ) where
  h : ZMod n → V
  inj : Function.Injective h
  adj_iff : ∀ i j, i ≠ j → (adj (h i) (h j
      ) ↔ ¬ cyc i j)

def QSTAB : Set (V → ℝ≥0∞) := {x | ∀ C,
    IsCliqueG adj C → ∑ v ∈ C, x v ≤ 1}

def STAB : Set (V → ℝ≥0∞) := Mix {x | ∃ I,
     IsStableG adj I ∧ x = ind I}

structure Hole (adj : V → V → Prop) (n : ℕ
    ) where
  h : ZMod n → V
  inj : Function.Injective h
  adj_iff : ∀ i j, i ≠ j → (adj (h i) (h j
      ) ↔ cyc i j)

theorem berge_of_stab_eq_qstab (heq : STAB
     adj = QSTAB adj) (k : ℕ) (hk : 2 ≤ k)
    [NeZero (2 * k + 1)] : IsEmpty (Hole
       adj (2 * k + 1)) ∧ IsEmpty (
       Antihole adj (2 * k + 1))

theorem berge_of_recipe_eq_pcs
```

```
    (heq : Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u
        , z u = 0 ∨ z u = 1} = tD X Y X' Y
        ')
    (k : ℕ) (hk : 2 ≤ k) [NeZero (2 * k +
        1)] :
    IsEmpty (Hole (excl (X := X) (Y := Y)
        (X' := X') (Y' := Y')) (2 * k + 1))
         ∧
      IsEmpty (Antihole (excl (X := X) (Y
          := Y) (X' := X') (Y' := Y')) (2 *
           k + 1))
```

B.11 Multiplicative–Additive Formulas

```
inductive Fm where
  | base (n : ℕ)
  | neg (A : Fm)
  | tens (A B : Fm)
  | wth (A B : Fm)

def web : Fm → Type
  | base n => Fin n
  | neg A => web A
  | tens A B => web A × web B
  | wth A B => web A ⊕ web B

def P : (A : Fm) → Set (web A → ℝ≥0∞)
  | base n => flat (Fin n)
  | neg A => orth (P A)
  | tens A B => orth (orth {z | ∃ x ∈ P A,
      ∃ y ∈ P B, z = tvec x y})
  | wth A B => {z | (fun a => z (Sum.inl a
      )) ∈ P A ∧ (fun b => z (Sum.inr b)) ∈
       P B}

def coh : (A : Fm) → web A → web A → Prop
  | base _ => fun x y => x = y
  | neg A => fun x y => x = y ∨ ¬ coh A x
      y
  | tens A B => fun s t => coh A s.1 t.1 ∧
      coh B s.2 t.2
  | wth A B => fun s t => match s, t with
    | Sum.inl a, Sum.inl a' => coh A a a'
    | Sum.inr b, Sum.inr b' => coh B b b'
    | _, _ => True

theorem isPCS_P : ∀ A : Fm, IsPCS (P A)

def par (A B : Fm) : Fm := neg (tens (neg
    A) (neg B))
def plus (A B : Fm) : Fm := neg (wth (neg
    A) (neg B))
def lolli (A B : Fm) : Fm := neg (tens A (
    neg B))

noncomputable def tvec {X Y : Type} (x : X
    → ℝ≥0∞) (y : Y → ℝ≥0∞) : X × Y → ℝ≥0∞
    :=
  fun ab => x ab.1 * y ab.2

noncomputable def svec {X Y : Type} (x : X
    → ℝ≥0∞) (y : Y → ℝ≥0∞) : X ⊕ Y → ℝ≥0∞
  | Sum.inl a => x a
  | Sum.inr b => y b

noncomputable def pr (s t : E) : E → ℝ≥0∞
    := Pi.single s 1 + Pi.single t 1

theorem invariants : ∀ A : Fm,
    (∀ s : web A, Pi.single s 1 ∈ P A ∧ ∀
        x ∈ P A, x s ≤ 1) ∧
    (∀ s t : web A, s ≠ t → coh A s t → pr
         s t ∈ P A) ∧
    (∀ s t : web A, s ≠ t → ¬ coh A s t →
        ∀ x ∈ P A, x s + x t ≤ 1)

theorem single_mem_P (A : Fm) (s : web A)
    : Pi.single s 1 ∈ P A

theorem pair_mem_P (A : Fm) {s t : web A}
    (hst : s ≠ t) (h : coh A s t) : pr s t
    ∈ P A

theorem sharp_isClique (A : Fm) {z : web A
    → ℝ≥0∞} (hz : z ∈ P A)
    (h01 : ∀ s, z s = 0 ∨ z s = 1) : ∀ s t
        , z s = 1 → z t = 1 → coh A s t

theorem recipe_subset_P (A : Fm) :
    Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨ z s
        = 1} ⊆ P A

theorem recipe_sharp_iff (A : Fm) {x : web
     A → ℝ≥0∞} (hx : ∀ s, x s = 0 ∨ x s =
    1) :
    x ∈ Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨
        z s = 1} ↔ x ∈ P A

theorem Mix_subset_orth {S T : Set (E → ℝ≥
    0∞)} (h : S ⊆ orth T) : Mix S ⊆ orth T

theorem P_lolli_base (n m : ℕ) : P (lolli
    (base n) (base m)) = lin (Fin n) (Fin m
    )

def second (n m n' m' : ℕ) : Fm := neg (
    tens (lolli (base n) (base m)) (lolli (
    base n') (base m')))
```

```
theorem P_second (n m n' m' : ℕ) :
    P (second n m n' m') = tD (Fin n) (Fin
        m) (Fin n') (Fin m')

def boolFin {k : ℕ} (hk : 2 ≤ k) : Bool →
    Fin k

theorem recipe_first_order (n m : ℕ) :
    P (lolli (base n) (base m)) =
        ConstructiveProb.Mix (Set.range
      (det : (Fin n → Option (Fin m)) →
          Fin n × Fin m → ℝ≥0∞))

theorem recipe_second_order {n m n' m' : ℕ
    } (hn : 0 < n) (hn' : 0 < n') :
    ConstructiveProb.Mix {z | z ∈ P (
        second n m n' m') ∧ ∀ u, z u = 0 ∨
        z u = 1}
      = sequentialG (Fin n) (Fin m) (Fin n
          ') (Fin m')

theorem recipe_second_order_ssubset {n m n
    ' m' : ℕ} (hn : 2 ≤ n) (hm : 2 ≤ m) (hn
    ' : 2 ≤ n')
    (hm' : 2 ≤ m') :
    ConstructiveProb.Mix {z | z ∈ P (
        second n m n' m') ∧ ∀ u, z u = 0 ∨
        z u = 1}
      ⊂ P (second n m n' m')

def φ : W × W → web S2 := emb (boolFin
    le_rfl) (boolFin le_rfl) (boolFin
    le_rfl) (boolFin le_rfl)

abbrev S2 : Fm := second 2 2 2 2

abbrev TT : Fm := neg (tens S2 S2)

noncomputable def shannonF : Finset (web (
    tens S2 S2)) := shannon.image fun u =>
    (φ u.1, φ u.2)

theorem shannonF_clique : ∀ u ∈ shannonF,
    ∀ v ∈ shannonF, coh TT u v

noncomputable def pentS2 : web S2 → ℝ≥0∞
    :=
  pentE (boolFin le_rfl) (boolFin le_rfl)
      (boolFin le_rfl) (boolFin le_rfl)

theorem shannonF_not_mem : (fun u => if u
    ∈ shannonF then (1 : ℝ≥0∞) else 0) ∉ P
    TT
```

B.12 Quantum Event Logic

```
def vec : Fin 18 → Fin 4 → ℤ := ![
  ![-1, 1, 1, 1], ![0, 0, 0, 1], ![0, 0,
      1, 0], ![0, 0, 1, 1], ![0, 1, -1, 0],
       ![0, 1, 0, -1],
  ![0, 1, 0, 0], ![1, -1, -1, 1], ![1, -1,
       0, 0], ![1, -1, 1, -1], ![1, 0, -1,
      0], ![1, 0, 0, -1],
  ![1, 0, 0, 1], ![1, 0, 1, 0], ![1, 1,
      -1, 1], ![1, 1, 0, 0], ![1, 1, 1,
      -1], ![1, 1, 1, 1]]

def bas : Fin 9 → Fin 4 → Fin 18 := ![
  ![1, 2, 15, 8], ![1, 6, 13, 10], ![9, 7,
       15, 3], ![9, 17, 10, 5], ![2, 6, 12,
       11],
  ![7, 17, 11, 4], ![14, 16, 8, 3], ![14,
      0, 13, 5], ![16, 0, 12, 4]]

def dot (u v : Fin 4 → ℤ) : ℤ := ∑ k, u k
    * v k

theorem basis_orth : ∀ b : Fin 9, ∀ i j :
    Fin 4, i ≠ j → dot (vec (bas b i)) (vec
     (bas b j)) = 0

theorem occ_two : ∀ v : Fin 18,
    (univ.filter (fun p : Fin 9 × Fin 4 =>
        bas p.1 p.2 = v)).card = 2

theorem no_ks : ¬ ∃ f : Fin 18 → Bool, ∀ b
    , (univ.filter (fun i => f (bas b i))).
    card = 1

def certain : Set (Fin 18 → ℝ≥0∞) :=
  {s | IsSharpVec s ∧ ∀ b, ∑ i, s (bas b i
      ) = 1}

theorem no_certain_state : certain = ∅

theorem Mix_certain_empty : Mix certain =
    ∅

theorem parseval (ψ : Fin 4 → ℝ) (b : Fin
    9) :
    ∑ i, (∑ k, ψ k * vec (bas b i) k) ^ 2
        / ∑ k, ((vec (bas b i) k : ℝ)) ^ 2
        = ∑ k, ψ k ^ 2

def states : Set (Fin 18 → ℝ≥0∞) := {x | ∀
     b, ∑ i, x (bas b i) = 1}
```

```
noncomputable def born (ψ : Fin 4 → ℝ) :
    Fin 18 → ℝ≥0∞ := fun v =>
  ENNReal.ofReal ((∑ k, ψ k * vec v k) ^ 2
      / ∑ k, ((vec v k : ℝ)) ^ 2)

theorem born_mem {ψ : Fin 4 → ℝ} (hψ : ∑ k
    , ψ k ^ 2 = 1) : born ψ ∈ states

theorem born_not_mix {ψ : Fin 4 → ℝ} (hψ :
     ∑ k, ψ k ^ 2 = 1) :
    born ψ ∈ states ∧ born ψ ∉ Mix certain
```

B.13 Finite Frames without Choice

```
def below (a : α) : Finset α := Finset.
    univ.filter (· ≤ a)

def ind (j x : α) : ℕ := if j ≤ x then 1
    else 0

def JoinPrime (j : α) : Prop := ∀ x y : α,
     j ≤ x ⊔ y → j ≤ x ∨ j ≤ y

theorem exists_joinPrime_aux : ∀ (n : ℕ) (
    a b : α), (below a).card = n → ¬ a ≤ b
    →
    ∃ j, j ≤ a ∧ ¬ j ≤ b ∧ JoinPrime j

theorem exists_sharp_separating_fin [
    BoundedOrder α] {a b : α} (hab : ¬ a ≤
    b) :
    ∃ v : α → ℕ, (∀ x, v x = 0 ∨ v x = 1)
        ∧ Monotone v ∧ v ⊥ = 0 ∧ v ⊤ = 1 ∧
      (∀ x y, v x + v y = v (x ⊔ y) + v (x
          ⊓ y)) ∧ v a = 1 ∧ v b = 0

theorem em_of_sharp_complement_fin [
    BoundedOrder α] (a c : α)
    (h : ∀ v : α → ℕ, (∀ x, v x = 0 ∨ v x
        = 1) → Monotone v → v ⊥ = 0 → v ⊤ =
         1 →
      (∀ x y, v x + v y = v (x ⊔ y) + v (x
          ⊓ y)) → v a + v c = 1) :
    a ⊔ c = ⊤
```

B.14 Choice-Free Finite Results

```
def wt (b : Bool) (m : Nat) : Nat := if b
    then m else 0

def mem : Op → Bool → Bool
  | bot, _ => false
  | halts, b => b
  | top, _ => true

def le (o o' : Op) : Bool := mem o true ≤
    mem o' true && mem o false ≤ mem o'
    false

def inf : Op → Op → Op
  | bot, _ => bot
  | _, bot => bot
  | halts, _ => halts
  | top, o => o

def val (wT wF : Nat) (o : Op) : Nat := wt
     (mem o true) wT + wt (mem o false) wF

def inClosure (x : Bool) : Bool := [bot,
    halts, top].all fun o => !mem o x ||
    mem o true

def frontier (x : Bool) : Bool :=
    inClosure x && !mem halts x

inductive Op | bot | halts | top
  deriving DecidableEq

def neg : Op → Op
  | bot => top
  | halts => bot
  | top => bot

theorem neg_spec : ∀ o o' : Op, (inf o' o
    = bot) ↔ le o' (neg o) = true

theorem neg_halts : neg halts = bot

theorem frontier_iff : ∀ x : Bool,
    frontier x = true ↔ x = false

theorem slack_eq (wT wF : Nat) :
    val wT wF top = val wT wF halts + val
        wT wF (neg halts) + wt (frontier
        false) wF

def fs : (n : Nat) → (Fin n → Nat) → Nat
  | 0, _ => 0
  | n + 1, f => f ⟨0, by omega⟩ + fs n (
      fun i => f ⟨i.val + 1, by omega⟩)

def allFin : (n : Nat) → (Fin n → Bool) →
    Bool
  | 0, _ => true
  | n + 1, p => p ⟨0, by omega⟩ && allFin
      n (fun i => p ⟨i.val + 1, by omega⟩)
```

```
def ext (ops : List (Fin n → Bool)) (U :
    Fin n → Bool) (x : Fin n) : Bool :=
  ops.any fun O => O x && allFin n (fun y
      => !(O y && U y))

def bd (ops : List (Fin n → Bool)) (U :
    Fin n → Bool) (x : Fin n) : Bool :=
  !U x && !ext ops U x

theorem sub_ext {ops : List (Fin n → Bool)
    } {U : Fin n → Bool} {O : Fin n → Bool}
    (hO : O ∈ ops) (hd : ∀ y, (!(O y && U
        y)) = true) {x : Fin n} (hx : O x =
         true) :
    ext ops U x = true

theorem slack_eq_boundary_fin (ops : List
    (Fin n → Bool)) (U : Fin n → Bool) (μ :
     Fin n → Nat) :
    fs n μ = fs n (fun x => wt (U x) (μ x)
        ) + fs n (fun x => wt (ext ops U x)
         (μ x)) +
      fs n (fun x => wt (bd ops U x) (μ x)
          )

def isPre : List Bool → List Bool → Bool
  | [], _ => true
  | _ :: _, [] => false
  | a :: u, b :: v => a == b && isPre u v

def Incomp (u v : List Bool) : Prop :=
    isPre u v = false ∧ isPre v u = false

def PF : List (List Bool) → Prop
  | [] => True
  | w :: S => (∀ v ∈ S, Incomp w v) ∧ PF S

def tails (b : Bool) : List (List Bool) →
    List (List Bool)
  | [] => []
  | [] :: S => tails b S
  | (a :: t) :: S => if a = b then t ::
      tails b S else tails b S

def ksum (L : Nat) : List (List Bool) →
    Nat
  | [] => 0
  | w :: S => 2 ^ (L - w.length) + ksum L
      S

theorem kraft : ∀ (L : Nat) (S : List (
    List Bool)), PF S → (∀ w ∈ S, w.length
    ≤ L) →
    ksum L S ≤ 2 ^ L

def Sat {n : Nat} (r : Row n) (w : Fin n →
     Int) : Prop :=
  0 ≤ dot n r.c w ∧ (r.s = true → 0 < dot
      n r.c w)

def comb {n : Nat} (p q : Int) (r₁ r₂ :
    Row n) : Row n :=
  ⟨fun i => p * r₁.c i + q * r₂.c i, (
      decide (0 < p) && r₁.s) || (decide (0
       < q) && r₂.s)⟩

def zeroS (n : Nat) : Row n := ⟨fun _ =>
    0, true⟩

def lastc (r : Row (n + 1)) : Int := r.c (
    Fin.last n)

def proj (r : Row (n + 1)) : Row n := ⟨fun
     i => r.c i.castSucc, r.s⟩

def posR (rows : List (Row (n + 1))) :=
    rows.filter fun r => decide (0 < lastc
    r)
def negR (rows : List (Row (n + 1))) :=
    rows.filter fun r => decide (lastc r <
    0)
def zerR (rows : List (Row (n + 1))) :=
    rows.filter fun r => decide (lastc r =
    0)

def pair (p q : Row (n + 1)) : Row n :=
    proj (comb (-lastc q) (lastc p) p q)

def step (rows : List (Row (n + 1))) :
    List (Row n) :=
  (zerR rows).map proj ++ (posR rows).
      flatMap fun p => (negR rows).map fun
      q => pair p q

def extd (D W : Int) (w' : Fin n → Int) :
    Fin (n + 1) → Int :=
  fun i => if h : i.val < n then D * w' ⟨i
      .val, h⟩ else W

def hrow (μ : Fin N → Int) (ν α : Int) (β
    : Fin m → Int) : Fin (N + 1) → Int :=
  fun i => if h : i.val < N then μ ⟨i.val,
       h⟩ + α + d * dot m β (S ⟨i.val, h⟩)
    else ν - α - dot m β y

def hullRows : List (Row (N + 1)) :=
  (List.finRange N).map (fun j => ⟨hrow S
      y d (fun k => if k = j then 1 else 0)
       0 0 (fun _ => 0), false⟩) ++
```

```
    [⟨hrow S y d (fun _ => 0) 1 0 (fun _ =>
        0), true⟩,
     ⟨hrow S y d (fun _ => 0) 0 1 (fun _ =>
        0), false⟩,
     ⟨hrow S y d (fun _ => 0) 0 (-1) (fun _
        => 0), false⟩] ++
    (List.finRange m).flatMap (fun e =>
      [⟨hrow S y d (fun _ => 0) 0 0 (fun k
          => if k = e then 1 else 0), false⟩,
       ⟨hrow S y d (fun _ => 0) 0 0 (fun k
          => if k = e then -1 else 0), false
          ⟩])

  structure Row (n : Nat) where
    c : Fin n → Int
    s : Bool

  inductive Der {n : Nat} (rows : List (Row
      n)) : Row n → Prop
    | base {r : Row n} : r ∈ rows → Der rows
        r
    | comb {r₁ r₂ : Row n} (p q : Int) : 0 ≤
        p → 0 ≤ q → Der rows r₁ → Der rows r
       ₂ →
        Der rows (comb p q r₁ r₂)

  theorem fm : ∀ (n : Nat) (rows : List (Row
      n)), (∃ w, ∀ r ∈ rows, Sat r w) ∨ Der
     rows (zeroS n)

  theorem complete (hd : 0 < d)
      (hlaw : ∀ (c : Fin m → Int) (k : Int),
          (∀ j, k ≤ dot m c (S j)) → k * d ≤
          dot m c y) :
      ∃ (lam : Fin N → Int) (t : Int), 0 < t
          ∧ (∀ j, 0 ≤ lam j) ∧
        dot N (fun _ => 1) lam = t ∧ ∀ e, d
            * dot N lam (fun j => S j e) = t
            * y e

  theorem sound (hd : 0 < d) (lam : Fin N →
      Int) (t : Int) (ht : 0 < t) (hl : ∀ j,
      0 ≤ lam j)
      (hsum : dot N (fun _ => 1) lam = t) (
          hx : ∀ e, d * dot N lam (fun j => S
          j e) = t * y e)
      (c : Fin m → Int) (k : Int) (hc : ∀ j,
          k ≤ dot m c (S j)) : k * d ≤ dot m
          c y

  def union {A B : Nat → Nat → Bool} (a :
      EOD A) (b : EOD B) : EOD (fun m m' => A
       m m' || B m m') where
    N

  def inter {A B : Nat → Nat → Bool} (a :
      EOD A) (b : EOD B) : EOD (fun m m' => A
       m m' && B m m') where
    N

  def compl {A : Nat → Nat → Bool} (a : EOD
      A) : EOD (fun m m' => !A m m') where
    N

  def val (b : Bool) : Nat := if b then 1
      else 0

  structure EC (S : Nat → Bool) where
    N : Nat
    v : Bool
    spec : ∀ m, N ≤ m → S m = v

  def rect {S T : Nat → Bool} (s : EC S) (t
      : EC T) : EOD (fun m m' => S m && T m')
       where
    N

  def tri : EOD (fun m m' => decide (m < m')
      ) where
    N

  structure EOD (A : Nat → Nat → Bool) where
    N : Nat
    lt : Bool
    eq : Bool
    gt : Bool
    spec : ∀ m m', N ≤ m → N ≤ m' →
      A m m' = if m < m' then lt else if m =
           m' then eq else gt

  def L {A : Nat → Nat → Bool} (w : EOD A) :
       Bool := w.lt
  /-- The iterated limit with `m → ∞` first.
       -/
  def R {A : Nat → Nat → Bool} (w : EOD A) :
       Bool := w.gt

  theorem L_modular {A B} (a : EOD A) (b :
      EOD B) :
      val (L a) + val (L b) = val (L (union
          a b)) + val (L (inter a b))

  theorem rect_L {S T} (s : EC S) (t : EC T)
       : L (rect s t) = (s.v && t.v)

  theorem rect_R {S T} (s : EC S) (t : EC T)
       : R (rect s t) = (s.v && t.v)
```

```
theorem products_differ : L tri ≠ R tri
```

B.15 Valued Frames and the Valuation Monoid

```
noncomputable def ValF : Frmᵒᵖ ⥤ Type _

abbrev SpVal := ValF.Elements

inductive Gen : Multiset P → Multiset P →
    Prop
  | modular (a b : P) : Gen ({a} + {b}) ({
      a ⊔ b} + {a ⊓ b})
  | bot : Gen {⊥} 0

def con : AddCon (Multiset P) := addConGen
    (Gen P)

abbrev V := (con P).Quotient

def ι (a : P) : V P := (con P).mk' {a}

structure Modular (f : P → M) : Prop where
  bot : f ⊥ = 0
  modular : ∀ a b, f a + f b = f (a ⊔ b) +
      f (a ⊓ b)

def lift (f : P → M) (hf : Modular f) : V
    P →+ M

theorem lift_ι (f : P → M) (hf : Modular f
    ) (a : P) : lift f hf (ι a) = f a

theorem hom_ext {h h' : V P →+ M} (H : ∀ a
    , h (ι a) = h' (ι a)) : h = h'

theorem modular_of_hom (h : V P →+ M) :
    Modular (fun a => h (ι a))

noncomputable def lvaluation_equiv {P :
    Type*} [Lattice P] [BoundedOrder P] :
    LValuation P ≃
      {h : V P →+ ℝ≥0∞ // h (ι ⊤) = 1 ∧
          Monotone fun a => h (ι a)}

theorem exists_flag (x : V P) : ∃ l : List
    P, l.Pairwise (· ≤ ·) ∧ x = (l.map ι).
    sum

theorem M3_flag_not_unique : ι M3.a + ι M3
    .o = ι M3.c + ι M3.o

def m3sep : M3 → ℝ≥0∞
  | .z => 0
  | .c => 0
  | _ => ⊤

theorem m3sep_modular : Modular m3sep

theorem M3_not_cancel : ι M3.a + ι M3.b =
    ι M3.a + ι M3.c ∧ ι M3.b ≠ ι M3.c
```

B.16 A Language for the Second-Order Type

```
inductive PEnd
  | halt
  | diverge
  | choose (k : Nat) (w : Fin k → ℝ≥0∞) (
      hw : ∑ i, w i ≤ 1) (p : Fin k → PEnd)

inductive POne (C D : Type)
  | diverge
  | choose (k : Nat) (w : Fin k → ℝ≥0∞) (
      hw : ∑ i, w i ≤ 1) (p : Fin k → POne
      C D)
  | call (c : C) (k : D → PEnd)

inductive P0 (X Y X' Y' : Type)
  | diverge
  | choose (k : Nat) (w : Fin k → ℝ≥0∞) (
      hw : ∑ i, w i ≤ 1) (p : Fin k → P0 X
      Y X' Y')
  | callF (a : X) (k : Y → POne X' Y')
  | callG (c : X') (k : Y' → POne X Y)

noncomputable def PEnd.den : PEnd → ℝ≥0∞
  | .halt => 1
  | .diverge => 0
  | .choose _ w _ p => ∑ i, w i * (p i).
      den

noncomputable def POne.den {C D : Type} [
    DecidableEq C] : POne C D → C × D → ℝ≥0
    ∞
  | .diverge => fun _ => 0
  | .choose _ w _ p => fun cd => ∑ i, w i
      * (p i).den cd
  | .call c k => fun cd => if cd.1 = c
      then (k cd.2).den else 0

noncomputable def P0.den : P0 X Y X' Y' →
    (X × Y) × (X' × Y') → ℝ≥0∞
  | .diverge => fun _ => 0
  | .choose _ w _ p => fun u => ∑ i, w i *
      (p i).den u
  | .callF a k => fun u => if u.1.1 = a
      then (k u.1.2).den u.2 else 0
  | .callG c k => fun u => if u.2.1 = c
      then (k u.2.2).den u.1 else 0
```

```
theorem P0.sound : ∀ p : P0 X Y X' Y', p.
    den ∈ sequentialG X Y X' Y'

theorem P0.den_mem_recipe [Nonempty X] [
    Nonempty X'] (p : P0 X Y X' Y') :
    p.den ∈ Mix {z | z ∈ tD X Y X' Y' ∧ ∀
        u, z u = 0 ∨ z u = 1}

def PEnd.ofBool (b : Bool) : PEnd := if b
    then .halt else .diverge

theorem P0.sharp_definable [Nonempty X] [
    Nonempty X'] {z : (X × Y) × (X' × Y') →
     ℝ≥0∞}
    (hz : z ∈ tD X Y X' Y') (h01 : ∀ u, z
        u = 0 ∨ z u = 1) : ∃ p : P0 X Y X'
        Y', p.den = z
```

B.17 The Boolean Case

```
noncomputable def comp (m : Bool → ℝ≥0∞) :
     Option Bool → ℝ≥0∞
  | none => 1 - (m true + m false)
  | some b => m b

noncomputable def bern (r : ℝ≥0∞) : Bool →
     ℝ≥0∞
  | true => r
  | false => 1 - r

abbrev Choice := (Unit → Option Bool) × (
    Bool × Bool → Option Bool) × (Bool ×
    Bool × Bool × Bool → Bool)

noncomputable def detCore (σ : Choice) (a
    b c d : Bool) : ℝ≥0∞ :=
  (if σ.1 () = some a then 1 else 0) * (if
      σ.2.1 (a, b) = some c then 1 else 0)
      *
    (if σ.2.2 (a, b, c, d) = true then 1
        else 0)

noncomputable def detF (σ : Choice) :
    FFirst where
  μ a

noncomputable def detG (σ : Choice) :
    GFirst where
  μ c

def sharpSeq : Set (W × W → ℝ≥0∞) := {z |
    z ∈ sequential ∧ ∀ pq, z pq = 0 ∨ z pq
    = 1}

structure Beh where
  μ : Bool → ℝ≥0∞
  κ : Bool → Bool → Bool → ℝ≥0∞
  ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
  hκ : ∀ a b, κ a b true + κ a b false ≤ 1
  hρ : ∀ a b c d, ρ a b c d ≤ 1

noncomputable def Beh.val (B : Beh) (a b c
     d : Bool) : ℝ≥0∞ := B.μ a * B.κ a b c
    * B.ρ a b c d

noncomputable def Beh.mass (B : Beh) : ℝ≥0
    ∞ := B.μ true + B.μ false

def seqU : Set (W × W → ℝ≥0∞) :=
  {t | ∃ BF BG : Beh, BF.mass + BG.mass ≤
      1 ∧
    t = fun pq => BF.val pq.1.1 pq.1.2 pq
        .2.1 pq.2.2 + BG.val pq.2.1 pq.2.2
        pq.1.1 pq.1.2}

theorem FFirst.den_mem_Mix (F : FFirst) :
    F.den ∈ Mix sharpSeq

theorem sequential_subset_Mix : sequential
     ⊆ Mix sharpSeq

theorem Mix_sequential : Mix sequential =
    sequential

theorem recipe_second_order_eq_sequential'
     :
    Mix {z | z ∈ tensDual ∧ ∀ pq, z pq = 0
         ∨ z pq = 1} = sequential

abbrev W2 := (W × W) × (W × W)

noncomputable def tens2 (t t' : W × W → ℝ≥
    0∞) : W2 → ℝ≥0∞ := fun u => t u.1 * t'
    u.2

def gensTT : Set (W2 → ℝ≥0∞) := {u | ∃ t ∈
     tensDual, ∃ t' ∈ tensDual, u = tens2 t
     t'}

def PTT : Set (W2 → ℝ≥0∞) := orth (orth
    gensTT)
def PTTdual : Set (W2 → ℝ≥0∞) := orth
    gensTT

def cohTT : W2 → W2 → Prop := tensC
    secondC secondC
def cohTTdual : W2 → W2 → Prop := dualC
    cohTT
```

```
def shannon : Finset W2 :=
  {(((true, true), (true, true)), ((true,
      true), (true, true))),
   (((false, true), (false, true)), ((
      false, true), (true, false))),
   (((false, true), (true, false)), ((
      false, false), (false, false))),
   (((true, false), (false, false)), ((
      false, true), (false, true))),
   (((false, false), (false, false)), ((
      true, false), (false, false)))}

theorem shannon_clique : IsClique
    cohTTdual (· ∈ shannon)

theorem shannon_not_mem :
    (fun u => if u ∈ shannon then (1 : ℝ≥0
       ∞) else 0) ∉ PTTdual

theorem recipe_eq_denotations [Nonempty X]
    [Nonempty X'] :
    Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u =
       0 ∨ z u = 1} = Set.range (P0.den (
      X := X) (Y := Y)
     (X' := X') (Y' := Y'))
```

B.18 Coordinate Retracts

```
noncomputable def push (j : E → F) (x : E
   → ℝ≥0∞) : F → ℝ≥0∞ :=
  fun a => ∑ d, if j d = a then x d else 0

structure Retract (D A : Fm) where
  j : web D → web A
  inj : Function.Injective j
  push_mem : ∀ x ∈ P D, push j x ∈ P A
  pull_mem : ∀ z ∈ P A, (fun d => z (j d))
      ∈ P D

def recipe (A : Fm) : Set (web A → ℝ≥0∞)
    :=
  ConstructiveProb.Mix {z | z ∈ P A ∧ ∀ s,
      z s = 0 ∨ z s = 1}

theorem Retract.recipe_ne {D A : Fm} (R :
    Retract D A) (h : recipe D ≠ P D) :
    recipe A ≠ P A

theorem pull_push {j : E → F} (hj :
    Function.Injective j) (x : E → ℝ≥0∞) :
    (fun d => push j x (j d)) = x

theorem pairing_push (j : E → F) (u : E →
    ℝ≥0∞) (z : F → ℝ≥0∞) :
    pairing (push j u) z = pairing u (fun
       d => z (j d))

def Retract.refl (D : Fm) : Retract D D

def Retract.trans {D A C : Fm} (R :
    Retract D A) (S : Retract A C) :
    Retract D C

def Retract.neg {D A : Fm} (R : Retract D
    A) : Retract (Fm.neg D) (Fm.neg A)

def Retract.negneg (D : Fm) : Retract D (
    Fm.neg (Fm.neg D))

def Retract.tensL (D B : Fm) (b : web B) :
    Retract D (Fm.tens D B)

def Retract.tensR (D B : Fm) (b : web B) :
    Retract D (Fm.tens B D)

def Retract.wthL (D B : Fm) : Retract D (
    Fm.wth D B)

def Retract.wthR (D B : Fm) : Retract D (
    Fm.wth B D)

theorem push_tensL {X Y : Type} [Fintype X
    ] [DecidableEq X] [DecidableEq Y] (b :
    Y)
    (x : X → ℝ≥0∞) : push (fun d => (d, b)
       ) x = tvec x (Pi.single b 1)

theorem push_tensR {X Y : Type} [Fintype X
    ] [DecidableEq X] [DecidableEq Y] (b :
    Y)
    (x : X → ℝ≥0∞) : push (fun d => (b, d)
       ) x = tvec (Pi.single b 1) x

theorem push_comp [Fintype F] (j1 : E → F)
    (j2 : F → G) (x : E → ℝ≥0∞) :
    push (fun d => j2 (j1 d)) x = push j2
       (push j1 x)

theorem push_id [DecidableEq E] (x : E → ℝ
    ≥0∞) : push (fun d : E => d) x = x

def NE : Fm → Prop
  | base n => 0 < n
  | Fm.neg A => NE A
  | tens A B => NE A ∧ NE B
  | wth A B => NE A ∧ NE B
```

```
theorem NE.point : ∀ A : Fm, NE A →
    Nonempty (web A)
  | base _, h => ⟨⟨0, h⟩⟩
  | Fm.neg A, h => NE.point A h
  | tens A B, h => by
    obtain ⟨a⟩ := NE.point A h.1
    obtain ⟨b⟩ := NE.point B h.2
    exact ⟨(a, b)⟩
  | wth A _, h => by
    obtain ⟨a⟩ := NE.point A h.1
    exact ⟨Sum.inl a⟩

/-- `Occ D p A`: `D` occurs in `A`,
    positively if `p` and negatively
    otherwise. -/
inductive Occ (D : Fm) : Bool → Fm → Prop
  | self : Occ D true D
  | neg {p : Bool} {A : Fm} : Occ D p A →
      Occ D (!p) (Fm.neg A)
  | tensL {p : Bool} {A : Fm} (B : Fm) :
      Occ D p A → Occ D p (tens A B)
  | tensR {p : Bool} {A : Fm} (B : Fm) :
      Occ D p A → Occ D p (tens B A)
  | wthL {p : Bool} {A : Fm} (B : Fm) :
      Occ D p A → Occ D p (wth A B)
  | wthR {p : Bool} {A : Fm} (B : Fm) :
      Occ D p A → Occ D p (wth B A)

/-- `D` or `D^⊥`, according to polarity.
    -/
def pol : Bool → Fm → Fm
  | true, D => D
  | false, D => Fm.neg D

/-- **Occurrences give retracts.** -/
theorem occ_retract {D : Fm} : ∀ {p : Bool
    } {A : Fm}, Occ D p A → NE A →
    Nonempty (Retract (pol p D) A)
  | _, _, Occ.self, _ => ⟨Retract.refl D⟩
  | _, _, Occ.neg (p := p) h, hA => by
    obtain ⟨R⟩ := occ_retract h hA
    cases p
    · exact ⟨(Retract.negneg D).trans R.
        neg⟩
    · exact ⟨R.neg⟩
  | _, _, Occ.tensL B h, hA => by
    obtain ⟨R⟩ := occ_retract h hA.1
    obtain ⟨b⟩ := NE.point B hA.2
    exact ⟨R.trans (Retract.tensL _ B b)⟩
  | _, _, Occ.tensR B h, hA => by
    obtain ⟨R⟩ := occ_retract h hA.2
    obtain ⟨b⟩ := NE.point B hA.1
    exact ⟨R.trans (Retract.tensR _ B b)⟩
  | _, _, Occ.wthL B h, hA => by
    obtain ⟨R⟩ := occ_retract h hA.1
    exact ⟨R.trans (Retract.wthL _ B)⟩
  | _, _, Occ.wthR B h, hA => by
    obtain ⟨R⟩ := occ_retract h hA.2
    exact ⟨R.trans (Retract.wthR _ B)⟩

/-- The second-order core `(n ⊸ m) ⊗ (n' ⊸
    m')`. -/
abbrev core (n m n' m' : ℕ) : Fm := tens (
    lolli (base n) (base m)) (lolli (base n
    ') (base m'))

/-- **The gap at every formula with a
    negative occurrence of the core.** -/
theorem gap_of_occ {n m n' m' : ℕ} (hn : 2
    ≤ n) (hm : 2 ≤ m) (hn' : 2 ≤ n') (hm'
    : 2 ≤ m')
    {A : Fm} (hocc : Occ (core n m n' m')
        false A) (hA : NE A) : recipe A ⊂ P
        A

example : recipe (lolli (core 2 2 2 2) (
    base 2)) ⊂ P (lolli (core 2 2 2 2) (
    base 2))

def partialCertain : Set (Fin 18 → ℝ≥0∞)
    :=
  {s | IsSharpVec s ∧ ∀ b, ∑ i, s (bas b i
      ) ≤ 1}

theorem sum_bases (f : Fin 18 → ℝ≥0∞) :
    ∑ b : Fin 9, ∑ i : Fin 4, f (bas b i)
        = 2 * ∑ v, f v

theorem partial_sum_le_four {s : Fin 18 →
    ℝ≥0∞} (hs : s ∈ partialCertain) : ∑ v,
    s v ≤ 4

theorem mix_sum_le_four {x : Fin 18 → ℝ≥0∞
    } (hx : x ∈ Mix partialCertain) : ∑ v,
    x v ≤ 4

theorem born_total {ψ : Fin 4 → ℝ} (hψ : ∑
     k, ψ k ^ 2 = 1) : 2 * ∑ v, born ψ v =
    9

theorem born_not_mix_partial {ψ : Fin 4 →
    ℝ} (hψ : ∑ k, ψ k ^ 2 = 1) :
    born ψ ∉ Mix partialCertain

def S4 : Finset (Fin 18) := {0, 1, 3, 11}
```

```
theorem S4_stable : ∀ b : Fin 9, (Finset.
    univ.filter (fun i : Fin 4 => bas b i ∈
     S4)).card ≤ 1

noncomputable def ind4 : Fin 18 → ℝ≥0∞ :=
    fun v => if v ∈ S4 then 1 else 0

theorem ind4_mem : ind4 ∈ partialCertain ∧
     ∑ v, ind4 v = 4
```

B.19 Four-Cycles and Closure

```
theorem push_tvec {X Y X' Y' : Type} [
    Fintype X] [Fintype Y] [DecidableEq X']
     [DecidableEq Y']
    (j : X → X') (k : Y → Y') (x : X → ℝ≥0
        ∞) (y : Y → ℝ≥0∞) :
    push (fun d : X × Y => (j d.1, k d.2))
         (tvec x y) = tvec (push j x) (push
         k y)

def Retract.tens {D A D' A' : Fm} (R :
    Retract D A) (R' : Retract D' A') :
    Retract (Fm.tens D D') (Fm.tens A A')

theorem gap_of_retracts {n m n' m' : ℕ} (
    hn : 2 ≤ n) (hm : 2 ≤ m) (hn' : 2 ≤ n')
     (hm' : 2 ≤ m')
    {X Y : Fm} (RX : Retract (lolli (base
        n) (base m)) X)
    (RY : Retract (lolli (base n') (base m
        ')) Y) :
    recipe (Fm.neg (tens X Y)) ⊂ P (Fm.neg
         (tens X Y))

theorem coh_symm : ∀ (A : Fm) {s t : web A
    }, coh A s t → coh A t s

structure C4 (X : Fm) where
  (a b c d : web X)
  hab : coh X a b
  hbc : coh X b c
  hcd : coh X c d
  hda : coh X d a
  hac : ¬ coh X a c
  hbd : ¬ coh X b d

def C4.j (Q : C4 X) : Fin 2 × Fin 2 → web
    X := fun p => ![![Q.a, Q.c], ![Q.b, Q.d
    ]] p.1 p.2

def C4.retract (Q : C4 X) : Retract (lolli
     (base 2) (base 2)) X

theorem gap_of_C4 {X Y : Fm} (QX : C4 X) (
    QY : C4 Y) :
    recipe (Fm.neg (tens X Y)) ⊂ P (Fm.neg
         (tens X Y))

theorem gap_of_C4_occ {X Y A : Fm} (QX :
    C4 X) (QY : C4 Y) (hocc : Occ (tens X Y
    ) false A)
    (hA : NE A) : recipe A ⊂ P A

def C4.lolli22 : C4 (lolli (base 2) (base
    2))

def C4.curried : C4 (Fm.neg (lolli (lolli
    (base 2) (base 2)) (base 2)))

theorem gap_curried :
    recipe (lolli (lolli (base 2) (base 2)
        ) (lolli (lolli (base 2) (base 2))
        (base 2)))
      ⊂ P (lolli (lolli (base 2) (base 2))
          (lolli (lolli (base 2) (base 2))
          (base 2)))

theorem orth_orth_subset_Mix {S : Set (E →
     ℝ≥0∞)} (hSf : S.Finite)
    (hS : ∀ s ∈ S, ConstructiveProb.
        IsSharpVec s) (h0 : (0 : E → ℝ≥0∞)
        ∈ S)
    (hdown : ∀ s ∈ S, ∀ t : E → ℝ≥0∞,
        ConstructiveProb.IsSharpVec t → t ≤
         s → t ∈ S) :
    orth (orth S) ⊆ ConstructiveProb.Mix S

theorem orth_Mix (S : Set (E → ℝ≥0∞)) :
    orth (ConstructiveProb.Mix S) = orth S

theorem recipe_orth_orth (A : Fm) : orth (
    orth (recipe A)) = recipe A

theorem recipe_tens {A B : Fm} (hA :
    recipe A = P A) (hB : recipe B = P B) :
    recipe (tens A B) = P (tens A B)

theorem recipe_wth {A B : Fm} (hA : recipe
     A = P A) (hB : recipe B = P B) :
    recipe (wth A B) = P (wth A B)

def sharpP (A : Fm) : Set (web A → ℝ≥0∞)
    := {z | z ∈ P A ∧ ∀ s, z s = 0 ∨ z s =
    1}

theorem P_neg_tens_eq {A B : Fm} (hA :
    recipe A = P A) (hB : recipe B = P B) :
```

```
    P (Fm.neg (tens A B)) = orth {z | ∃ s
        ∈ sharpP A, ∃ t ∈ sharpP B, z =
        tvec s t}

def svec' {X Y : Type} (x : X → ℝ≥0∞) (y :
    Y → ℝ≥0∞) : X ⊕ Y → ℝ≥0∞
  | Sum.inl a => x a
  | Sum.inr b => y b

theorem tvec_sum {X Y : Type} [Fintype X]
    [Fintype Y] {m n : ℕ} (w : Fin m → ℝ≥0∞
    )
    (s : Fin m → X → ℝ≥0∞) (v : Fin n → ℝ≥
        0∞) (t : Fin n → Y → ℝ≥0∞) :
    tvec (∑ i, w i • s i) (∑ j, v j • t j)
      = ∑ ij : Fin m × Fin n, (w ij.1 * v
          ij.2) • tvec (s ij.1) (t ij.2)

theorem recipe_lolli_base (n m : ℕ) :
    recipe (lolli (base n) (base m)) = P (
        lolli (base n) (base m))

theorem recipe_core (n m n' m' : ℕ) :
    recipe (core n m n' m') = P (core n m n
    ' m')

abbrev X3 : Fm := wth (base 2) (base 1)

abbrev F3 : Fm := Fm.neg (tens X3 S2)

theorem F3_eq : F3 = lolli X3 (core 2 2 2
    2) := rfl

def w0 : web S2 := ((0, 0), (0, 0))
def w1 : web S2 := ((0, 0), (0, 1))
def w2 : web S2 := ((0, 1), (1, 1))
def w3 : web S2 := ((1, 0), (0, 1))
def w4 : web S2 := ((1, 1), (1, 0))

def p0 : web X3 := Sum.inl 0
def p1 : web X3 := Sum.inl 1
def pu : web X3 := Sum.inr 0

def supp3 : Finset (web F3) := {(p0, w2),
    (p0, w4), (p1, w0), (pu, w1), (pu, w3)}

noncomputable def y3 : web F3 → ℝ≥0∞ :=
    fun v => if v ∈ supp3 then 2⁻¹ else 0

theorem y3_mem : y3 ∈ P F3

theorem sharp3_le {s : web F3 → ℝ≥0∞} (hs
    : s ∈ P F3) (h01 : ∀ v, s v = 0 ∨ s v =
     1) :
    ∑ v ∈ supp3, s v ≤ 2

theorem gap_F3 : recipe F3 ⊂ P F3

theorem X3_no_C4 (Q : C4 X3) : False
```

B.20 Second Order for Additive Types

```
inductive ACode
  | one
  | wth (a b : ACode)
  | plus (a b : ACode)

def toFm : ACode → Fm
  | one => Fm.base 1
  | wth a b => Fm.wth a.toFm b.toFm
  | plus a b => Fm.plus a.toFm b.toFm

abbrev W (a : ACode) : Type := web a.toFm

theorem recipe_plus {A B : Fm} (hA :
    recipe A = P A) (hB : recipe B = P B) :
    recipe (Fm.plus A B) = P (Fm.plus A B)

theorem recipe_code : ∀ a : ACode, recipe
    a.toFm = P a.toFm

theorem clique_mem_P : ∀ (a : ACode) (c :
    W a → ℝ≥0∞), (∀ x, c x = 0 ∨ c x = 1) →
    (∀ x y, c x = 1 → c y = 1 → ACode.coh
        a x y) → c ∈ P a.toFm

inductive TCode
  | one
  | plus (s t : TCode)
  | cone (t : TCode)

def toA : TCode → ACode
  | one => ACode.one
  | plus s t => ACode.plus s.toA t.toA
  | cone t => ACode.wth t.toA ACode.one

structure GIso (a b : ACode) where
  e : W a ≃ W b
  coh_iff : ∀ x y, ACode.coh a x y ↔ ACode
      .coh b (e x) (e y)

def GIso.retract {a b : ACode} (I : GIso a
     b) : Retract a.toFm b.toFm

def retractSwap (A B : Fm) : Retract (Fm.
    tens A B) (Fm.tens B A)

def HasC4 (a : ACode) : Prop :=
```

```
  ∃ p q r s : W a, ACode.coh a p q ∧ ACode
      .coh a q r ∧ ACode.coh a r s ∧ ACode.
      coh a s p ∧
    ¬ ACode.coh a p r ∧ ¬ ACode.coh a q s

theorem hasC4_iff (a : ACode) : HasC4 a ↔
    Nonempty (C4 a.toFm)

def Complete (a : ACode) : Prop := ∀ x y,
    ACode.coh a x y

theorem complete_or {a b : ACode} (h : ¬
    HasC4 (ACode.wth a b)) : Complete a ∨
    Complete b

theorem tree_of_noC4 : ∀ a : ACode, ¬
    HasC4 a → ∃ t : TCode, Nonempty (GIso a
     t.toA)

noncomputable def ω : (a : ACode) → (W a →
     ℝ) → ℝ
  | one, v => v pt
  | wth a b, v => ω a (fun x => v (Sum.inl
      x)) + ω b (fun x => v (Sum.inr x))
  | plus a b, v => max (ω a (fun x => v (
      Sum.inl x))) (ω b (fun x => v (Sum.
      inr x)))

abbrev N (t : TCode) : Type := ACode.W t.
    toA

def PathSums {V : Type*} [AddCommMonoid V]
     : (t : TCode) → (N t → V) → Set V
  | one, y => {0, y ACode.pt}
  | plus s t, y => PathSums s (fun x => y
      (Sum.inl x)) ∪ PathSums t (fun x => y
       (Sum.inr x))
  | cone t, y => insert 0 ((fun S => y (
      root t) + S) '' PathSums t (fun x =>
      y (Sum.inl x)))

def IsPathInd : (t : TCode) → (N t → ℝ) →
    Prop
  | TCode.one, a => a = 0 ∨ a = fun _ => 1
  | TCode.plus s u, a =>
      (∃ a1 : N s → ℝ, IsPathInd s a1 ∧ a
          = Sum.elim a1 (0 : N u → ℝ)) ∨
        (∃ a2 : N u → ℝ, IsPathInd u a2 ∧
            a = Sum.elim (0 : N s → ℝ) a2)
  | TCode.cone u, a => a = 0 ∨
      ∃ a' : N u → ℝ, IsPathInd u a' ∧ a =
          Sum.elim a' (fun _ : W ACode.one
          => (1 : ℝ))

theorem clique_le_path : ∀ (t : TCode) (c
    : N t → ℝ), (∀ x, c x = 0 ∨ c x = 1) →
    (∀ x y, c x = 1 → c y = 1 → ACode.coh
        t.toA x y) → ∃ a, IsPathInd t a ∧ c
        ≤ a

theorem mem_P_iff (t : TCode) (K : ACode)
    (Z : N t → W K → ℝ) (hZ : ∀ x k, 0 ≤ Z
    x k) :
    (fun p : N t × W K => ENNReal.ofReal (
        Z p.1 p.2)) ∈
        P (Fm.neg (Fm.tens t.toA.toFm K.
            toFm)) ↔
      ∀ S ∈ PathSums t Z, ω K S ≤ 1

def PMix (S : Set E) : Set E :=
  {x | ∃ (ι : Type) (_ : Fintype ι) (w : ι
      → ℝ) (s : ι → E),
    (∀ i, 0 < w i) ∧ ∑ i, w i = 1 ∧ (∀ i,
        s i ∈ S) ∧ x = ∑ i, w i • s i}

def IsStab (a : ACode) (v : W a → ℝ) :
    Prop := (∀ x, v x = 0 ∨ v x = 1) ∧ ω a
    v ≤ 1

def RootSet (K : ACode) (t : TCode) : Set
    ((W K → ℝ) × (N t → W K → ℝ)) :=
  {p | IsStab K p.1 ∧ 0 ≤ p.2 ∧ ∀ S ∈
      PathSums t p.2, ω K (p.1 + S) ≤ 1}

theorem root_step : ∀ (K : ACode) (t :
    TCode) (m : W K → ℝ) (y : N t → W K → ℝ
    ),
    0 ≤ m → 0 ≤ y → (∀ S ∈ PathSums t y, ω
        K (m + S) ≤ 1) → (m, y) ∈ PMix (
        RootSet K t)

def Valid (t : TCode) (K : ACode) : Set (N
     t → W K → ℝ) :=
  {σ | (∀ x h, σ x h = 0 ∨ σ x h = 1) ∧ ∀
      S ∈ PathSums t σ, IsStab K S}

theorem main : ∀ (t : TCode) (K : ACode) (
    z : N t → W K → ℝ), 0 ≤ z →
    (∀ S ∈ PathSums t z, ω K S ≤ 1) → z ∈
        PMix (Valid t K)

theorem second_order_noGap (t : TCode) (K
    : ACode) :
    recipe (Fm.neg (Fm.tens t.toA.toFm K.
        toFm)) = P (Fm.neg (Fm.tens t.toA.
        toFm K.toFm))
```

```
theorem exact_absorb : ∀ (K : ACode) {b v
    : W K → ℝ} {L : ℝ}, 0 ≤ b → 0 ≤ v → ω K
     b ≤ L →
    L ≤ ω K (b + v) → ∃ a : W K → ℝ, 0 ≤ a
        ∧ a ≤ v ∧ ω K (b + a) = L

def AbsOK (K : ACode) (L : ℝ) (m : W K → ℝ
    ) (S : (W K → ℝ) × (W K → ℝ)) : Prop :=
  ω K (m + S.1) ≤ L ∧ ω K S.2 + L ≤ max L
      (ω K (m + S.1 + S.2))

def AbsAt (K : ACode) (t : TCode) (L : ℝ)
    (m : W K → ℝ) (y : N t → W K → ℝ) :
    Prop :=
  ∃ p q : N t → W K → ℝ, 0 ≤ p ∧ 0 ≤ q ∧ p
      + q = y ∧
    ∀ S ∈ PathSums t (fun x => (p x, q x))
        , AbsOK K L m S

theorem greedy (K : ACode)
    (abs0 : ∀ (t : TCode) (m : W K → ℝ) (y
        : N t → W K → ℝ), 0 ≤ m → 0 ≤ y →
      AbsAt K t (ω K m) m y) :
    ∀ (t : TCode) (L : ℝ) (m : W K → ℝ) (y
        : N t → W K → ℝ), 0 ≤ m → 0 ≤ y →
      ω K m ≤ L →
      AbsAt K t L m y

theorem absorb : ∀ (K : ACode) (t : TCode)
     (L : ℝ) (m : W K → ℝ) (y : N t → W K →
     ℝ),
    0 ≤ m → 0 ≤ y → ω K m ≤ L → AbsAt K t
       L m y

theorem second_order_iff (a b : ACode) :
    recipe (Fm.neg (Fm.tens a.toFm b.toFm)
       ) = P (Fm.neg (Fm.tens a.toFm b.
       toFm)) ↔
      ¬ (HasC4 a ∧ HasC4 b)
```

B.21 A Gap Among Total Programs

```
abbrev L2 : Fm := lolli (base 2) (base 2)

abbrev S3 : Fm := tens (tens L2 L2) L2

abbrev T3 : Fm := Fm.neg S3

noncomputable def graph2 (f : Fin 2 → Fin
    2) : web L2 → ℝ≥0∞ := fun p => if f p.1
     = p.2 then 1 else 0

theorem graph2_mem (f : Fin 2 → Fin 2) :
    graph2 f ∈ P L2

theorem sharp_L2_le {s : web L2 → ℝ≥0∞} (
    hs : s ∈ sharpP L2) : ∃ f, s ≤ graph2 f

theorem sharp_L2L2_le {s : web (tens L2 L2
    ) → ℝ≥0∞} (hs : s ∈ sharpP (tens L2 L2)
    ) :
    ∃ f g, s ≤ tvec (graph2 f) (graph2 g)

def k3 (v : W3) : ℕ := if v = onePt then 2
     else if v ∈ halfPts then 1 else 0

noncomputable def y5 : web T3 → ℝ≥0∞ :=
    fun v => (k3 v : ℝ≥0∞) * 2^(-1)

theorem y5_total (f g h : Fin 2 → Fin 2) :
    pairing (tvec (tvec (graph2 f) (graph2
        g)) (graph2 h)) y5 = 1

theorem y5_mem : y5 ∈ P T3

theorem sharp5_le {s : web T3 → ℝ≥0∞} (hs
    : s ∈ P T3) (h01 : ∀ v, s v = 0 ∨ s v =
     1) :
    s c0 + s c1 + s c2 + s c3 + s c4 ≤ 2

theorem gap_T3 : recipe T3 ⊂ P T3
```

B.22 Local Certainty

```
def locS (C : Set (Finset E)) : Set (E → ℝ
    ≥0∞) := {x | ∀ t ∈ C, ∑ e ∈ t, x e ≤ 1}

def locN (C : Set (Finset E)) : Set (E → ℝ
    ≥0∞) := {x | ∀ t ∈ C, ∑ e ∈ t, x e = 1}

def globS (C : Set (Finset E)) : Set (E →
    ℝ≥0∞) := Mix {x | x ∈ locS C ∧
    IsSharpVec x}

def globN (C : Set (Finset E)) : Set (E →
    ℝ≥0∞) := Mix {x | x ∈ locN C ∧
    IsSharpVec x}

theorem globN_sub (C : Set (Finset E)) :
    globN C ⊆ locN C

theorem globS_sub (C : Set (Finset E)) :
    globS C ⊆ locS C

theorem globN_single : globN ({Finset.univ
    } : Set (Finset E)) = locN {Finset.univ
    }
```

```
theorem globS_single : globS ({Finset.univ
    } : Set (Finset E)) = locS {Finset.univ
    }

def loc (A : Fm) : Set (web A → ℝ≥0∞) :=
    orth (sharpP (Fm.neg A))

def dualCtx (A : Fm) : Set (Finset (web A)
    ) :=
  {t | ∃ s ∈ sharpP (Fm.neg A), t = Finset
      .univ.filter fun e => s e = 1}

theorem loc_eq_locS (A : Fm) : loc A =
    locS (dualCtx A)

theorem recipe_sub_P_sub_loc (A : Fm) :
    recipe A ⊆ P A ∧ P A ⊆ loc A

theorem P_eq_loc_iff (A : Fm) : P A = loc
    A ↔ recipe (Fm.neg A) = P (Fm.neg A)

theorem clique_mem_loc (A : Fm) {c : web A
    → ℝ≥0∞} (h01 : ∀ e, c e = 0 ∨ c e = 1)
    (hc : ∀ x y, c x = 1 → c y = 1 → coh A
        x y) : c ∈ loc A

theorem P_S2_eq_loc : P S2 = loc S2

theorem P_TT_ssubset_loc : P TT ⊂ loc TT

def ctx : Set (Finset (Fin 18)) := {t | ∃
    b, t = Finset.univ.image (bas b)}

theorem locN_cabello : locN ctx = states

theorem globN_cabello : globN ctx = ∅

theorem born_local {ψ : Fin 4 → ℝ} (hψ : ∑
    k, ψ k ^ 2 = 1) : born ψ ∈ locN ctx

theorem born_mono (ψ : Fin 4 → ℝ) : born ψ
    1 ≤ born ψ 12 + born ψ 11

def halfSet : Finset (Fin 18) := {0, 1, 2,
    6, 7, 9, 14, 16, 17}

noncomputable def half : Fin 18 → ℝ≥0∞ :=
    fun v => if v ∈ halfSet then 2⁻¹ else 0

theorem half_not_born :
    half ∈ locN ctx ∧ half ∉ Mix {x | ∃ ψ
        : Fin 4 → ℝ, ∑ k, ψ k ^ 2 = 1 ∧ x =
        born ψ}

noncomputable def unit (u : Fin 18) : Fin
    4 → ℝ :=
  fun k => (vec u k : ℝ) / Real.sqrt (dot
      (vec u) (vec u))

theorem unit_norm (u : Fin 18) : ∑ k, unit
    u k ^ 2 = 1

theorem born_ray_zero_iff (u v : Fin 18) :
    born (unit u) v = 0 ↔ dot (vec u) (vec
    v) = 0

theorem born_ray_one_iff (u v : Fin 18) :
    born (unit u) v = 1 ↔ u = v
```

B.23 Uniqueness and the Parts of Slack

```
theorem calculus_unique {S C : Set (E → ℝ≥
    0∞)} (hS : ∀ s ∈ S, IsSharpVec s) (hSC
    : S ⊆ C)
    (hconv : Mix C ⊆ C)
    (hlaws : ∀ L : AffineLaw E, (∀ s ∈ S,
        L.Holds s) → ∀ x ∈ C, L.Holds x) :
    C = Mix S

theorem calculus_laws_iff {S C : Set (E →
    ℝ≥0∞)} (hSC : S ⊆ C)
    (hlaws : ∀ L : AffineLaw E, (∀ s ∈ S,
        L.Holds s) → ∀ x ∈ C, L.Holds x) (L
        : AffineLaw E) :
    (∀ x ∈ C, L.Holds x) ↔ ∀ s ∈ S, L.
        Holds s

theorem localMix_subset_closure {S : Set (
    E → ℝ≥0∞)} {x : E → ℝ≥0∞}
    (hx : ∀ F : Finset E, restr F x ∈ Mix
        (restr F '' S)) : x ∈ closure (Mix
        S)

theorem calculus_unique_closed {S C : Set
    (E → ℝ≥0∞)} (hS : ∀ s ∈ S, IsSharpVec s
    )
    (hSC : S ⊆ C) (hconv : Mix C ⊆ C) (
        hclosed : IsClosed C)
    (hlaws : ∀ L : FinLaw E, (∀ s ∈ S, L.
        Holds s) → ∀ x ∈ C, L.Holds x) :
    C = closure (Mix S) ∧
      C = {x | ∀ L : FinLaw E, (∀ s ∈ S, L
          .Holds s) → L.Holds x}

theorem em_of_sharp_compl (a : Ω) (h : ∀ v
    : Valuation Ω, v.IsSharp → v a + v aᶜ
    = 1) :
    a ⊔ aᶜ = ⊤
```

```
theorem deMorganGap_zero_iff :
    (∀ (v : Valuation Ω) (a : Ω), v.
        deMorganGap a = 0) ↔ ∀ a : Ω, a^cc ⊔
         a^c = ⊤

theorem dnGap_zero_iff :
    (∀ (v : Valuation Ω) (a : Ω), v.dnGap
        a = 0) ↔ ∀ a : Ω, a^cc = a
```

B.24 Universal Laws

Worlds are functions `I →0` with the product topology, and `cex bad` is the event that some object has a kind in `bad` . `cexExcept bad j` is the event of a counterexample at an object other than `j`, and `obs j A` the event that object `j` has a kind in `A`.

```
theorem interior_compl_cex [Infinite I] {
    bad : Set O} (hne : bad.Nonempty) :
    interior (cex (I := I) bad)^c = ∅

theorem compl_cexOpens [Infinite I] {bad :
     Set O} (hbad : IsOpen bad) (hne : bad.
    Nonempty) :
    (cexOpens (I := I) hbad)^c = ⊥

theorem slack_cex [Infinite I] {bad : Set
    O} (hbad : IsOpen bad) (hne : bad.
    Nonempty)
    (v : Valuation (Opens (I → O))) :
    v.slack (cexOpens hbad) = 1 - v (
        cexOpens hbad)

theorem classical_law_eq_slack [Infinite I
    ] {bad : Set O} (hbad : IsOpen bad)
    (hne : bad.Nonempty) (μ : Measure (I →
         O)) [IsProbabilityMeasure μ] :
    μ (cex bad)^c = μ.toValuationOpens.
        slack (cexOpens hbad)

theorem classical_posterior_law [Infinite
    I] {bad : Set O} (hbad : IsOpen bad)
    (hne : bad.Nonempty) (μ : Measure (I →
         O)) [IsProbabilityMeasure μ] (e :
        Opens (I → O))
    (he : μ.toValuationOpens e ≠ 0) :
    μ.toValuationOpens.condVal e he (
        cexOpens hbad)^c = 0 ∧
      μ ((cex bad)^c ∩ e) / μ e = μ (
          frontier (cex (I := I) bad) ∩ e)
          / μ e

theorem posterior_cex_select {bad A : Set
    O} (hbad : IsOpen bad) (hA : IsOpen A)
    (j : I)
    (μ : Measure (I → O)) [
        IsProbabilityMeasure μ] (hsel : μ (
        obs j A) ≠ 0)
    (hind : ∀ B ∈ ({A, A ∩ bad} : Set (Set
         O)),
      μ (cexExcept bad j ∩ obs j B) = μ (
          cexExcept bad j) * μ (obs j B)) :
    μ (cex bad ∩ obs j A) / μ (obs j A)
      = μ (cexExcept bad j)
        + μ (obs j (A ∩ bad)) / μ (obs j A
            ) * μ (cexExcept bad j)^c
```

B.25 Categorical Readings

```
theorem mix_deltaPoint_inj {w w' : P → ℝ≥0
    ∞} (hw : ∑ p, w p = 1)
    (h : ∀ U : LowerSet P, ∑ q, w q *
        deltaPoint q U = ∑ q, w' q *
        deltaPoint q U) :
    w = w'

theorem lin_not_simplex :
    ∃ f g h k : Fin 2 → Option (Fin 2), f
        ≠ h ∧ f ≠ k ∧ g ≠ h ∧ g ≠ k ∧
      (fun ab => 2^-1 * det f ab + 2^-1 *
          det g ab)
        = (fun ab => 2^-1 * det h ab + 2^-1
            * det k ab)

structure HasMeets (S : Set (Ev → ℝ≥0∞)) :
     Prop where
  top : ∃ t, ∀ s ∈ S, s t = 1
  meet : ∀ a b, ∃ c, ∀ s ∈ S, (s c = 1 ↔ s
      a = 1 ∧ s b = 1)

theorem trim_eq_of_meet {S : Set (Ev → ℝ≥0
    ∞)} (hm : HasMeets S)
    (μ ν : ProbabilityMeasure (Cfg Ev))
    (hμ : ∀ F, (μ : Measure (Cfg Ev)) (bad
         S F) = 0) (hν : ∀ F, (ν : Measure
        (Cfg Ev)) (bad S F) = 0)
    (h : ∀ e, (μ : Measure (Cfg Ev)) (atom
         e) = (ν : Measure (Cfg Ev)) (atom
        e)) :
    (μ : Measure (Cfg Ev)).trim evSigma_le
         = (ν : Measure (Cfg Ev)).trim
        evSigma_le

theorem hasMeets_sharpVecs (Ω : Type) [
    Order.Frame Ω] : HasMeets (sharpVecs Ω)

theorem orth_recipe (A : Fm) : orth (
    recipe A) = loc (Fm.neg A)
```

```
theorem orth_loc (A : Fm) : orth (loc A) =
    recipe (Fm.neg A)

theorem tight_iff (A : Fm) :
    (orth (recipe A) = recipe (Fm.neg A) ∧
        orth (recipe (Fm.neg A)) = recipe
       A) ↔
      (recipe A = P A ∧ recipe (Fm.neg A)
         = P (Fm.neg A))

theorem tight_iff_recipe_eq_loc (A : Fm) :
    orth (recipe A) = recipe (Fm.neg A) ↔
        recipe A = loc A

theorem not_tight_core : orth (recipe (
    core 2 2 2 2)) ≠ recipe (Fm.neg (core 2
    2 2 2))

theorem P_lolli_eq_loc (n m : ℕ) :
    P (lolli (base n) (base m)) = loc (
        lolli (base n) (base m))

theorem tight_lolli (n m : ℕ) :
    orth (recipe (lolli (base n) (base m))
        ) = recipe (Fm.neg (lolli (base n)
        (base m)))

theorem tight_not_tensor_closed {n m n' m'
    : ℕ} (hn : 2 ≤ n) (hm : 2 ≤ m) (hn' :
   2 ≤ n')
    (hm' : 2 ≤ m') :
    orth (recipe (lolli (base n) (base m))
        ) = recipe (Fm.neg (lolli (base n)
        (base m))) ∧
    orth (recipe (lolli (base n') (base m
        '))) = recipe (Fm.neg (lolli (base
        n') (base m'))) ∧
    orth (recipe (tens (lolli (base n) (
        base m)) (lolli (base n') (base m')
        )))
      ≠ recipe (Fm.neg (tens (lolli (base
          n) (base m)) (lolli (base n') (
          base m'))))

theorem sharp_dual_exclusive {s : web (
    core 2 2 2 2) → ℝ≥0∞}
    (hs : s ∈ sharpP (Fm.neg (core 2 2 2
        2))) {e e' : BEv} (hne : e ≠ e') (
        he : s e = 1)
    (he' : s e' = 1) : ¬ cohE e e'

theorem chsh_bilinear {X Y : Fin 2 × Fin 2
    → ℝ≥0∞} (hX : ∀ a, ∑ b, X (a, b) ≤ 1)
    (hY : ∀ a, ∑ b, Y (a, b) ≤ 1) :
    ∑ e : BEv, (if win e then X e.1 * Y e
        .2 else 0) ≤ 3

theorem prBox_mem_loc : prBox ∈ loc (core
    2 2 2 2)

theorem prBox_not_mem_P : prBox ∉ P (core
    2 2 2 2)

theorem P_core_ssubset_loc : P (core 2 2 2
    2) ⊂ loc (core 2 2 2 2)

theorem em_of_complemented (h : ∀ a : Ω, ∃
    b, IsCompl a b) (a : Ω) : a ⊔ aᶜ = ⊤

structure IsPartialSumState (q : Ω → ℝ≥0∞)
    : Prop where
  bot : q ⊥ = 0
  top : q ⊤ = 1
  mono : Monotone q
  add : ∀ a b, a ⊓ b = ⊥ → q (a ⊔ b) = q a
      + q b

theorem Valuation.isPartialSumState (v :
    Valuation Ω) : IsPartialSumState (v : Ω
    → ℝ≥0∞)

theorem partialSum_not_modular :
    ∃ q : LowerSet V → ℝ≥0∞,
        IsPartialSumState q ∧
      ¬ ∀ a b, q a + q b = q (a ⊔ b) + q (
          a ⊓ b)

def windowCalc (S : Set (E → ℝ≥0∞)) : (
    Finset E)ᵒᵖ ⥤ Type where
  obj F := WinObj S F.unop
  map f := TypeCat.ofHom (winRestr S (
      leOfHom f.unop))

def closedCalc (S : Set (E → ℝ≥0∞)) : Type
    :=
  {x : E → ℝ≥0∞ // ∀ L : FinLaw E, (∀ s ∈
      S, L.Holds s) → L.Holds x}

noncomputable def closedCalcEquivLimit {S
    : Set (E → ℝ≥0∞)} (hS : ∀ s ∈ S,
    IsSharpVec s) :
    closedCalc S ≃ Limits.limit (
        windowCalc S)
```

B.26 The Axioms of a Probability Calculus

```
def Sol (Λ : Set (AffineLaw E)) : Set (E →
    ℝ≥0∞) := {x | ∀ L ∈ Λ, L.Holds x}
```

```
def ValidLaws (S : Set (E → ℝ≥0∞)) : Set (
    AffineLaw E) := {L | ∀ s ∈ S, L.Holds s
    }

theorem exists_extension_iff {S : Set (E →
     ℝ≥0∞)} {Λ : Set (AffineLaw E)} :
    (∃ C : Set (E → ℝ≥0∞), S ⊆ C ∧ Mix C ⊆
         C ∧ ∀ L ∈ Λ, ∀ x ∈ C, L.Holds x) ↔
      Λ ⊆ ValidLaws S

def FinVal : Set (E → ℝ≥0∞) := {x | ∀ e, x
     e ≠ ⊤}

theorem Mix_finVal {S : Set (E → ℝ≥0∞)} (
    hS : S ⊆ FinVal) : Mix S ⊆ FinVal

theorem existsUnique_finExtension_iff {S :
     Set (E → ℝ≥0∞)} (hS : ∀ s ∈ S,
    IsSharpVec s)
    {Λ : Set (AffineLaw E)} :
    (∃! C : Set (E → ℝ≥0∞), C ⊆ FinVal ∧ S
         ⊆ C ∧ Mix C ⊆ C ∧ ∀ L ∈ Λ, ∀ x ∈ C
        , L.Holds x) ↔
      Λ ⊆ ValidLaws S ∧ FinVal ∩ Sol Λ =
          Mix S

theorem
    existsUnique_finExtension_validLaws {S
    : Set (E → ℝ≥0∞)}
    (hS : ∀ s ∈ S, IsSharpVec s) :
    ∃! C : Set (E → ℝ≥0∞), C ⊆ FinVal ∧ S
        ⊆ C ∧ Mix C ⊆ C ∧
      ∀ L ∈ ValidLaws S, ∀ x ∈ C, L.Holds
          x

theorem existsUnique_finExtension_mono {S
    : Set (E → ℝ≥0∞)} (hS : ∀ s ∈ S,
    IsSharpVec s)
    {Λ Λ' : Set (AffineLaw E)} (h : Λ ⊆ Λ
        ') (hΛ' : Λ' ⊆ ValidLaws S)
    (hd : ∃! C : Set (E → ℝ≥0∞), C ⊆
        FinVal ∧ S ⊆ C ∧ Mix C ⊆ C ∧
      ∀ L ∈ Λ, ∀ x ∈ C, L.Holds x) :
    ∃! C : Set (E → ℝ≥0∞), C ⊆ FinVal ∧ S
        ⊆ C ∧ Mix C ⊆ C ∧
      ∀ L ∈ Λ', ∀ x ∈ C, L.Holds x

noncomputable def rmass (f : LowerSet P →
    ℝ) (p : P) : ℝ :=
  f (LowerSet.Iic p) - f (LowerSet.Iio p)

def IsModularR (f : LowerSet P → ℝ) : Prop
     :=
  ∀ U V : LowerSet P, f (U ⊔ V) + f (U ⊓ V
      ) = f U + f V

theorem eq_sum_rmass {f : LowerSet P → ℝ}
    (h0 : f ⊥ = 0) (hmod : IsModularR f)
    (U : LowerSet P) : f U = ∑ p ∈ U.
        toFinset, rmass f p

theorem monotone_of_rmass_nonneg {f :
    LowerSet P → ℝ} (h0 : f ⊥ = 0) (hmod :
    IsModularR f)
    (hloc : ∀ p, f (LowerSet.Iio p) ≤ f (
        LowerSet.Iic p)) : Monotone f

theorem frame_axioms_iff {x : LowerSet P →
     ℝ≥0∞} (hfin : ∀ U, x U ≠ ⊤) :
    (x ⊥ = 0 ∧ x ⊤ = 1 ∧ (∀ U V, x U + x V
         = x (U ⊔ V) + x (U ⊓ V)) ∧
        ∀ p, x (LowerSet.Iio p) ≤ x (
            LowerSet.Iic p)) ↔
      ∃ v : Valuation (LowerSet P), v.
          toFun = x

theorem frame_axiom_needed {p q : P} (hqp
    : q < p) :
    ∃ x : LowerSet P → ℝ≥0∞, (∀ U, x U ≠ ⊤
        ) ∧ x ⊥ = 0 ∧ x ⊤ = 1 ∧
      (∀ U V, x U + x V = x (U ⊔ V) + x (U
          ⊓ V)) ∧
      (∀ r, r ≠ p → x (LowerSet.Iio r) ≤ x
          (LowerSet.Iic r)) ∧
      ¬ ∃ v : Valuation (LowerSet P), v.
          toFun = x

abbrev W4 := (Bool × Bool) × (Bool × Bool)

abbrev exclB : W4 → W4 → Prop := excl (X
    := Bool) (Y := Bool) (X' := Bool) (Y'
    := Bool)

def chshSupp (α β γ : Bool) : Finset W4 :=
  Finset.univ.filter fun e => (e.1.2 ^^ e
      .2.2) = (((e.1.1 ^^ α) && (e.2.1 ^^ β
      )) ^^ γ)

theorem chsh_no_three : ∀ α β γ : Bool, ∀
    a ∈ chshSupp α β γ, ∀ b ∈ chshSupp α β
    γ,
    ∀ c ∈ chshSupp α β γ, a ≠ b → a ≠ c →
        b ≠ c → exclB a b ∨ exclB a c ∨
        exclB b c

theorem recipe_satisfies_laws {x : W4 → ℝ≥
    0∞}
```

```
    (hx : x ∈ Mix {z | z ∈ tD Bool Bool
        Bool Bool ∧ ∀ u, z u = 0 ∨ z u =
        1}) :
    (∀ C, IsCliqueG exclB C → ∑ e ∈ C, x e
        ≤ 1) ∧
      ∀ α β γ, ∑ e ∈ chshSupp α β γ, x e ≤
          2

def cliq (f g : Bool → Bool) : Finset W4
    :=
  Finset.univ.filter fun e => f e.1.1 = e
      .1.2 ∧ g e.2.1 = e.2.2

theorem cliq_isClique : ∀ f g : Bool →
    Bool, IsCliqueG exclB (cliq f g)

theorem cliq_maximal : ∀ f g : Bool → Bool
    , ∀ v : W4, v ∉ cliq f g →
    ∃ u ∈ cliq f g, ¬ exclB u v

theorem clique_law_needed (f g : Bool →
    Bool) : ∃ x : W4 → ℝ≥0∞,
    (∀ C, IsCliqueG exclB C → C ≠ cliq f g
        → ∑ e ∈ C, x e ≤ 1) ∧
    (∀ α β γ : Bool, ∑ e ∈ chshSupp α β γ,
        x e ≤ 2) ∧
    1 < ∑ e ∈ cliq f g, x e

theorem chsh_law_needed (α β γ : Bool) : ∃
     x ∈ tD Bool Bool Bool Bool,
    (∀ C, IsCliqueG exclB C → ∑ e ∈ C, x e
        ≤ 1) ∧
    (∀ α' β' γ' : Bool, (α', β', γ') ≠ (α,
        β, γ) → ∑ e ∈ chshSupp α' β' γ', x
        e ≤ 2) ∧
    2 < ∑ e ∈ chshSupp α β γ, x e

theorem tD_violates_chsh (α β γ : Bool) :
    ∃ x ∈ tD Bool Bool Bool Bool, 2 < ∑ e
        ∈ chshSupp α β γ, x e
```

C The Aczél Development

This appendix sketches the from-scratch construction behind the product-rule half of Theorem 31. The target shape is Aczél's associativity theorem [Aczél 1966]: a suitably regular associative $F : \mathbb{R} \to \mathbb{R} \to \mathbb{R}$ regraduates to multiplication, $G(F\,x\,y) = G(x) \cdot G(y)$ for all x, y and a strictly monotone G. mathlib has no associativity representation, no Hölder-style embedding of ordered semigroups on intervals, and no Cauchy functional equation under monotonicity (only under continuity).

Hypotheses. A `Scale` bundles: associativity of F, *divisibility* $\forall c\, \exists x,\ F\,x\,x = c$, positivity on the cone $x < F\,x\,c$, joint order-preservation, joint continuity, a unit u, and identity at $-\infty$, meaning $F\,x\,z \to x$ as $z \to -\infty$. Order-preservation is required by the additivity proof below. Divisibility yields the 2^n-th roots used in the dyadic construction below. Without it the generator is not definable. It is assumed directly in the bundle, though its interval form follows from the intermediate value theorem (`exists_diag_eq`). Non-vacuity: *log-sum-exp*, $F\,x\,y = \log(e^x + e^y)$ with $u = 0$, satisfies every field of the bundle (`logSumExpScale`), with intended generator globally exp.

Construction. Write $a^{(n)}$ for n right-associated copies of a under F (`Fpow`). Associativity alone makes $n \mapsto a^{(n)}$ a semigroup morphism from $(\mathbb{N}, +)$. Commutativity is not assumed, and follows instead as a corollary of the generator. Unboundedness of the iterates is derived: were they bounded, monotone convergence would give a fixed point $F\,L\,a = L$, contradicting positivity, and unboundedness yields the Archimedean property. Divisibility yields a tower of 2^n-th roots $u \succ u^{1/2} \succ u^{1/4} \succ \cdots$ (`droot`), where 2^ℓ copies of the finer root compose under F to one copy of the coarser root. Counting the number of copies of the n-th root not exceeding b (`dcount`, well defined by the Archimedean property) gives dyadic approximants $g_n(b) = \mathrm{dcount}(b, n)/2^n$. The refinement inequality $2N + 1 \le N' \le 2N + 2$, where N and N' are the counts at consecutive refinement levels n and $n + 1$, makes (g_n) monotone and Cauchy, and the generator is its limit, $g(b) = \bigsqcup_n g_n(b)$.

Results (all `sorry`-free, standard axioms). On the cone $\{x \mid u \le x\}$ we prove three things. Normalization: $g(u) = 1$. Additivity: $g(F\,x\,y) = g(x) + g(y)$, since $\mathrm{dcount}\,x + \mathrm{dcount}\,y + 1 \le \mathrm{dcount}(F\,x\,y) \le \mathrm{dcount}\,x + \mathrm{dcount}\,y + 3$ and the 2^{-n} error term vanishes. And strict monotonicity, proved without any continuity of g: a root inserted between $x < y$ contributes a full extra block of 2^{n-M} copies, M being the refinement level of the inserted root. Exponentiating gives the multiplicative form, a strictly monotone G with $G(F\,x\,y) = G(x)\,G(y)$ on the cone (`exists_mul_generator`), the conclusion of Aczél's theorem restricted to the cone (`aczelStatement_cone`). The reorientation $\bar{G} = \exp(-g)$ gives the $[0, 1]$-valued form used in Section 6.

Remaining gap. The constructed g is discontinuous at the unit (it is 0 below u), so matching the classical statement verbatim needs the off-cone extension (a group completion) and continuity of the extension. This is open. It is not vacuous: the log-sum-exp example admits a global continuous generator (`hasOrderedGenerator_logSumExp`). Of Hölder's classical hypotheses, the Archimedean property is proved, and is what enables the embedding into $\mathbb{R}$. The interval form of divisibility follows from the intermediate value theorem. Only the embedding's global regularity remains.